ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI: ỨNG DỤNG DỮ LIỆU MẠNG XÃ HỘI ĐỂ NÂNG CAO HIỆU QUẢ TRONG LOGISTICS NHÂN ĐẠO

Môn học: Lập trình hướng đối tượng

Nhóm: 05

Mã lớp: 167919

Giảng viên hướng dẫn: TS. Trịnh Tuấn Đạt

Sinh viên tham gia:

Vũ Lê Dũng – 202416900

Phạm Minh Hùng – 202416928

Nguyễn Thị Thanh Trúc – 202417052

Hà Nội, tháng 6, năm 2026

LỜI NÓI ĐẦU

Trong những năm gần đây, nhân loại đang phải đối mặt với những thách thức to lớn từ sự biến đổi khí hậu khắc nghiệt. Các thảm họa thiên nhiên như bão lũ, hạn hán hay đại dịch không chỉ gia tăng về tần suất mà còn để lại những hậu quả tàn khốc: tước đi sinh mạng, phá hủy cơ sở hạ tầng và làm gián đoạn nghiêm trọng nền kinh tế - xã hội. Trước những thời khắc khủng hoảng đó, Logistics nhân đạo (Humanitarian Logistics) nổi lên như một "mạch máu" sống còn, quyết định trực tiếp đến tốc độ cứu trợ và khả năng phục hồi của toàn bộ khu vực bị ảnh hưởng.

Tuy nhiên, thực tiễn từ các đợt thiên tai quy mô lớn cho thấy, công tác cứu trợ thường xuyên vấp phải một rào cản chí mạng: sự đứt gãy và nhiễu loạn thông tin. Trong bối cảnh đó, mạng xã hội không còn đơn thuần là không gian giao tiếp, mà đã chuyển mình thành một "trạm cảm biến" khổng lồ, cập nhật liên tục nhu cầu kêu cứu và tình trạng thiệt hại theo thời gian thực. Việc thu thập, bóc tách và giải mã khối lượng dữ liệu khổng lồ này chính là chìa khóa để điều hướng dòng chảy cứu trợ đến đúng nơi, đúng lúc và đúng người.

Nhận thức sâu sắc được tính cấp bách đó, cùng với mong muốn đóng góp năng lực công nghệ vào công cuộc giải quyết các bài toán thiết thực của xã hội, nhóm chúng em đã quyết định thực hiện đề tài: **"Ứng dụng dữ liệu mạng xã hội để nâng cao hiệu quả trong Logistics nhân đạo"**.

Lấy bối cảnh từ sức tàn phá của các đợt thiên tai nghiêm trọng (tiêu biểu như siêu bão Yagi), báo cáo tập trung xây dựng quy trình thu thập và tiền xử lý dữ liệu đa nguồn từ các nền tảng mạng xã hội. Dựa trên nguồn dữ liệu đã chuẩn hóa, nhóm tiến hành giải quyết các bài toán phân tích nghiệp vụ cốt lõi bao gồm: theo dõi diễn biến tâm lý công chúng theo thời gian, phân loại và đánh giá các danh mục thiệt hại (nhà cửa, con người, hạ tầng...), đồng thời đo lường mức độ đáp ứng các nhu yếu phẩm thiết yếu.

Thông qua báo cáo này, nhóm hy vọng sẽ cung cấp một góc nhìn định lượng trực quan, hỗ trợ đắc lực cho các cơ quan và tổ chức nhân đạo trong việc ra quyết định và ưu tiên phân bổ nguồn lực.

MỤC LỤC

I. THÔNG TIN CHUNG VỀ NHÓM 7

1.1. Danh sách thành viên 7
1.2. Nhật ký làm việc nhóm 7
1.3. Phân công nhiệm vụ & mức độ đóng góp của từng thành viên 12

II. GÓC NHÌN CHUNG VỀ BÀI TOÁN 13

2.1. Yêu cầu bài toán 13
2.2. Mở rộng yêu cầu 14
2.2.1. Bài toán 4 (Phân tích điểm nóng thiên tai - Hotspot Locations) 14
2.2.2. Tích hợp Trí tuệ nhân tạo (AI Generative Summary) 14
2.2.3. Đồng bộ hóa giao diện đa trạng thái (Theme Toggle) 14
2.2.4. Hệ thống báo cáo chuyên nghiệp 14
2.3. Tổng quan hoạt động hệ thống 15

III. PHÂN TÍCH & THIẾT KẾ HỆ THỐNG 16

3.1. Cấu trúc dữ liệu và lưu trữ tệp (CSV/JSON) 16
3.1.1. Cấu trúc dữ liệu trong bộ nhớ (In-memory Data Structure) 16
3.1.2. Lưu trữ tệp tin CSV (Định dạng phân cấp) 16
3.1.3. Xử lý cấu trúc dữ liệu JSON từ API 17
3.1.4. Bóc tách cấu trúc dữ liệu từ HTML (Web Scraping) 17
3.2. Biểu đồ phụ thuộc gói (UML) 18
3.3. Biểu đồ lớp (UML) & phân tích chi tiết lớp 19
3.3.1. Các lớp cơ sở độc lập & cấu trúc nội bộ trong từng gói 19
3.3.1.1. model 19
3.3.1.2. data 22
3.3.1.3. exception 26
3.3.1.4. analysis 27
3.3.1.5. preprocessor 30
3.3.1.6. service 33
3.3.1.7. app 36
3.3.1.8. ui 38

3.3.2. Mối quan hệ tương tác tổng thể giữa các gói với nhau.....41
3.3.2.1. app & ui 41
3.3.2.2. app & preprocessor.....42
3.3.2.3. app & analysis 43
3.3.2.4. app & data.....44
3.2.2.5. app & model 45
3.2.2.6. ui & analysis 46
3.2.2.7. ui & service.....47
3.2.2.8. preprocessor & model.....47
3.2.2.9. analysis & model 48
3.2.2.10. service & model.....49
3.2.2.11. data & model.....50
3.2.2.12. data & exception.....51
3.2.2.13. service & exception 52
3.2.2.14. app & exception.....53
IV. BẢNG THỐNG KÊ VỀ DỮ LIỆU ĐÃ THU THẬP 54
4.1. Bão yagi – Dân Trí News 54
4.2. Bão bualoi – VNExpress News.....55
4.3. Bão yagi – VietNamNet News 56
4.4. Bão yagi – Youtube 57
4.5. Bão matmo – TikTok.....58
V. CÁC CÔNG NGHỆ & KỸ THUẬT LẬP TRÌNH ĐƯỢC ÁP DỤNG 59
5.1. Trong lập trình hướng đối tượng 59
5.1.1. Tính Đóng Gói (Encapsulation).....59
5.1.1.1. Tính đóng gói trong preprocessor.....59
5.1.1.2. Tính đóng gói trong service.....60
5.1.1.3. Tính đóng gói trong model 60
5.1.1.4. Tính đóng gói trong data 61
5.1.1.5. Tính đóng gói trong analysis 62
5.1.2. Tính Kế Thừa (Inheritance) 63
5.1.2.1. Kế thừa trong exception & các package liên quan.....63
5.1.2.2. Kế thừa trong ui.....63

5.1.2.3. Kế thừa trong model 64
5.1.2.4. Kế thừa trong data 65
5.1.2.5. Kế thừa từ analysis 65
5.1.3. Tính Đa Hình (Polymorphism) 66
5.1.3.1. Đa hình trong preprocessor 66
5.1.3.2. Đa hình trong service 67
5.1.3.3. Đa hình trong analysis 67
5.1.3.4. Đa hình trong model 67
5.1.3.5. Đa hình trong data 68
5.1.4. Tính Trừu Tượng (Abstraction) 69
5.1.4.1 Trừu tượng trong analysis 69
5.1.4.2. Trừu tượng trong model 70
5.1.4.3. Trừu tượng trong data 70
5.1.5. Các mẫu thiết kế áp dụng (Design Patterns) 71
5.2. Các công nghệ, thuật toán hay khác 72
5.2.1. Máy trạng thái phân tích CSV chuẩn RFC 4180 (Custom CSV State Machine) 72
5.2.2. Thuật toán trích lọc bối cảnh thiên tai (Contextual Filtering) 72
5.2.3. Duyệt đối sánh từ khóa thông minh bằng Regex biên từ (Unicode Word Boundary) 73
5.2.4. Tối ưu hóa hạn ngạch gọi mạng (Batch Stats Request) 73
5.2.5. Đa luồng bất đồng bộ chống đóng băng giao diện (Asynchronous Multithreading) 73

VI. TRIỂN KHAI, HƯỚNG DẪN SỬ DỤNG & MINH HỌA CHƯƠNG TRÌNH 74

6.1. Môi trường triển khai và yêu cầu hệ thống 74
6.2. Hướng dẫn sử dụng chi tiết 75
6.3. Minh họa và thuyết minh chương trình 77
6.3.1. Giao diện màn hình Trang chủ (HomeView) 77
6.3.2. Giao diện bảng điều khiển trung tâm (DashboardView) 78
6.3.3. Kết quả thu thập và lưu trữ báo cáo ngoại tuyến 79
6.3.4. Trực quan hóa đồ thị và Phân tích tóm tắt bằng Generative AI 80

VII. KẾT LUẬN, MỞ RỘNG & HƯỚNG PHÁT TRIỂN TƯƠNG LAI 82

7.1. Kết luận 82
7.2. Hạn chế của hệ thống 82
7.3. Hướng phát triển tương lai 83

TÀI LIỆU THAM KHẢO 84

I. Thông tin chung về nhóm

1.1. Danh sách thành viên

Họ và tên	MSSV	Mail
Phạm Minh Hùng	202416928	Hung.PM2416928@sis.hust.edu.vn
Nguyễn Thị Thanh Trúc	202417052	Truc.NTT2417052@sis.hust.edu.vn
Vũ Lê Dũng	202416900	Dung.VL2416900@sis.hust.edu.vn

1.2. Nhật ký làm việc nhóm

Thời gian	Giai đoạn	Hoạt động chính & Phân công
14/03 - 28/03	Khởi động & Khảo sát	**- 14/03: Họp nhóm khởi động** • Nội dung: Phân tích các yêu cầu từ giảng viên hướng dẫn, thảo luận các ý tưởng ban đầu. **- 15/03 - 18/03: Khảo sát và tìm kiếm đề tài** • Nội dung: Cả nhóm tham khảo sản phẩm/đề tài của các nhóm khác và khóa trước để tìm hiểu xu hướng và yêu cầu kỹ thuật. **- 19/03: Họp nhóm chọn chủ đề** • Nội dung: Thống nhất chọn chủ đề chính thức cho dự án dựa trên kết quả khảo sát. **- 20/03 - 27/03: Lập kế hoạch và thiết kế hệ thống sơ bộ** • **Hùng:** Lập kế hoạch chi tiết, viết mô tả bài toán và xác định các yêu cầu chức năng. • **Cả nhóm:** Nghiên cứu công nghệ áp dụng, phác thảo kiến trúc hệ thống sơ bộ. **- 28/03: Họp nhóm thông qua kế hoạch** • Nội dung: Đánh giá lại tính khả thi của kế hoạch trước khi bắt tay vào code thử nghiệm.

29/03 – 14/04	Thử nghiệm đầu tiên & Thay đổi định hướng	**- 29/03 - 11/04: Xây dựng phiên bản Demo thử nghiệm** • **Hùng:** Thực hiện chia việc cho cả nhóm, xác định rõ ai code phần nào và phụ trách package nào trong mã nguồn để tránh xung đột code (conflict). • **Cả nhóm:** Đồng loạt tiến hành code demo các tính năng cốt lõi theo thiết kế ban đầu đã thống nhất. • **Hùng:** Chủ trì hỗ trợ kiểm thử các tính năng thô, tổng hợp lỗi và chuẩn bị các tài liệu kỹ thuật liên quan để nhóm đánh giá khả năng vận hành hệ thống. **- 12/04 - 13/04: Đánh giá phiên bản Demo** • Cả nhóm chạy thử và nhận thấy khả năng vận hành không ổn định, công nghệ ban đầu chưa tối ưu hoặc quá phức tạp. **- 14/04: Họp nhóm khẩn cấp** • Nội dung: Phân tích nguyên nhân thất bại của bản demo, quyết định thay đổi phương án và thiết kế lại hệ thống.
15/04 – 21/05	Thiết kế lại & Chỉnh sửa code	**- 15/04 - 20/04: Thiết kế lại luồng xử lý hệ thống** • **Trúc:** Nghiên cứu cấu trúc dữ liệu đầu vào và thiết kế sơ bộ mô hình (model & data). • **Hùng:** Thiết kế các bước tiền xử lý dữ liệu (preprocessor) cần thiết cho mô hình. • **Dũng:** Thiết kế các phương pháp và chỉ số phân tích dữ liệu đầu ra (analysis). **- 21/04: Họp nhóm thống nhất luồng xử lý mới** • Nội dung: Rà soát lại tính logic của luồng đi từ Mô hình → Dữ liệu → Tiền xử lý → Phân tích để đảm bảo tính khả thi trước khi code. **- 22/04 - 29/04: Phát triển phần model & data**

		• **Trúc:** Tập trung hoàn thiện phần thu thập/chuẩn bị dữ liệu và xây dựng mô hình (model & data). • **Hùng & Dũng:** Hỗ trợ chuẩn bị môi trường lập trình và viết các hàm bổ trợ. **- 02/05 - 07/05: Phát triển phần Tiền xử lý (preprocessor)** • **Hùng:** Nhận phần dữ liệu và mô hình từ Trúc để xây dựng module tiền xử lý dữ liệu (preprocessor). • **Dũng:** Chuẩn bị các thuật toán phân tích dựa trên dữ liệu đã qua tiền xử lý. • **Trúc:** Viết tài liệu mô tả về cấu trúc dữ liệu và mô hình đã hoàn thành ở bước trước. **- 09/05 - 15/05: Phát triển phần Phân tích (analysis)** • **Dũng:** Tiến hành code module phân tích dữ liệu (analysis) dựa trên kết quả tiền xử lý của Hùng. • **Hùng & Trúc:** Hỗ trợ Dũng kiểm tra tính chính xác của các thuật toán phân tích. **- 16/05 - 20/05: Xây dựng Giao diện và Luồng xử lý chính (UI & Main)** • **Hùng & Dũng:** Phối hợp thiết kế giao diện (UI) và tích hợp tất cả các module vào tệp chạy chính (Main). • **Trúc:** Kiểm thử tổng thể luồng chạy dữ liệu từ đầu vào đến đầu ra trên giao diện để phát hiện lỗi. **- 21/05: Họp nhóm nghiệm thu phần khung của hệ thống** • Nội dung: Đánh giá độ ổn định của luồng xử lý (model → data → preprocessor → analysis → ui/main), chuẩn bị tinh chỉnh giao diện chi tiết ở giai đoạn tiếp theo.

		• Đồng thời các thành viên trong nhóm sẽ tìm cách tối ưu hơn nữa.
26/05 – 31/05	Cải tiến mã nguồn	**- 26/05 – 28/05: Tái cấu trúc mã nguồn (Refactor)** • **Công việc:** Cả nhóm (Trúc, Hùng, Dũng) tập trung chia nhỏ các lớp lớn thuộc phần mình phụ trách thành các lớp đơn nhiệm gọn gàng hơn. **- 29/05: Họp và viết Package Exception** • **Công việc:** Giải thích phần phân chia lại package của mỗi người sau khi tái cấu trúc lại & cả nhóm thống nhất và xây dựng package exception để xử lý tập trung các lỗi phát sinh. • **Hùng:** Triển khai phần exception. **- 30/05: Hùng bắt đầu sửa giao diện và tích hợp AI** • **Hùng:** Bắt đầu chỉnh sửa giao diện theo cấu trúc code mới và tiến hành tích hợp tính năng AI vào UI. **- 31/05: Hùng hoàn thiện phần AI trên giao diện** • **Hùng:** Tiếp tục hoàn thiện phần kết nối và hiển thị tính năng AI trên giao diện. • **Trúc & Dũng:** Hỗ trợ kiểm thử tổng thể giao diện có tích hợp AI của Hùng để đảm bảo hệ thống chạy ổn định, không có lỗi xung đột.
01/06 – 13/06	Viết báo cáo & Hoàn thiện hệ thống	**- 01/06: Bắt đầu viết báo cáo & Comment code** • **Cả nhóm:** Thực hiện viết chú thích (comment) rõ ràng trong mã nguồn và note lại tài liệu kỹ thuật của các module đã refactor xong. **- 02/06 – 06/06: Vẽ biểu đồ lớp, package trên astah**

		• **Cả nhóm:** Tiến hành vẽ biểu đồ lớp chi tiết dựa trên cấu trúc các class mới đã được chia nhỏ và tối ưu ở Giai đoạn 4. • **Hùng:** Tiến hành gộp lại các phần của các thành viên và thêm vào báo cáo đồng thời vẽ biểu đồ package. • **Trúc:** Quay video demo, cắt ghép, chỉnh sửa và xuất bản video hoàn chỉnh. • **Dũng:** Làm slide thuyết trình. **- 07/06 - 08/06: Viết phần còn lại của báo cáo** • **Cả nhóm:** Tập trung viết các chương còn lại của báo cáo (kết quả đạt được, hướng phát triển) và định dạng lại tài liệu theo đúng chuẩn. **- 09/06 - 10/06: Kiểm tra cuối cùng** • Rà soát lại toàn bộ sản phẩm: Code sạch, File báo cáo hoàn chỉnh, Video demo, slide để sẵn sàng báo cáo trước hội đồng. **- 11/06 – 12/06: Luyện tập** • Cả nhóm đọc lại báo cáo, slide, luyện tập thuyết trình, chuẩn bị kỹ phần của mình & góp ý với nhau. **- 13/06: Cả nhóm vào họp lần cuối và luyện tập báo cáo trước thầy**

1.3. Phân công nhiệm vụ & mức độ đóng góp của từng thành viên

Họ và Tên	Nhiệm vụ	Mức độ đóng góp
Phạm Minh Hùng	- Tổ chức, lên ý tưởng, phân chia, tổng hợp công việc, nhiệm vụ của các thành viên trong nhóm - Lên ý tưởng, code chính các packages: preprocessor, exception, service, ui - Vẽ biểu đồ UML cho các packages trên, biểu đồ gói, biểu đồ quan hệ giữa các gói - Làm, chỉnh sửa báo cáo chính, slides.	**100%**
Nguyễn Thị Thanh Trúc	- Lên ý tưởng, code chính các packages: model, data - Vẽ biểu đồ UML cho các packages trên, hỗ trợ vẽ biểu đồ quan hệ giữa các gói. - Hỗ trợ làm, chỉnh sửa báo cáo. - Quay video demo, edit video.	**100%**
Vũ Lê Dũng	- Lên ý tưởng, code chính các packages: app, analysis - Vẽ biểu đồ UML cho các packages trên. - Hỗ trợ làm, chỉnh sửa báo cáo. - Làm slide thuyết trình.	**100%**

II. Góc nhìn chung về bài toán

2.1. Yêu cầu bài toán

Trong quản lý thảm họa, **Logistics nhân đạo (Humanitarian Logistics)** thường xuyên phải đối mặt với thách thức lớn về sự đứt gãy và nhiễu loạn thông tin. Trong các đợt thiên tai quy mô lớn, tiêu biểu là siêu bão Yagi (tháng 9/2024) hay bão Bualoi (tháng 9/2025),.. việc thu thập nhu cầu cứu trợ và phân loại thiệt hại theo thời gian thực là vô cùng cấp bách.

Bài toán đặt ra yêu cầu xây dựng một ứng dụng cho phép:

- **Thu thập dữ liệu đa nguồn:** Khai thác thông tin từ báo điện tử lớn (VnExpress, Dân Trí, VietNamNet) và các mạng xã hội phổ biến (YouTube, TikTok) trong khung thời gian thảm họa.

- **Tiền xử lý dữ liệu:** Làm sạch dữ liệu chữ thô, xử lý nhiễu (emoji, viết tắt, từ vô nghĩa) để chuẩn hóa nội dung đầu vào.

- **Phân tích dữ liệu:** Bóc tách và cấu trúc hóa các luồng văn bản phi cấu trúc thông qua việc trích xuất từ khóa, nhận diện thực thể (địa điểm, thời gian, loại nhu yếu phẩm) và phân loại luồng thông tin cứu trợ

- **Giải quyết các bài toán phân tích nghiệp vụ:**
    - *Bài toán 1 (Theo dõi tâm lý theo thời gian):* Thống kê số lượng bài đăng/bình luận tích cực và tiêu cực theo từng ngày để theo dõi diễn biến tâm lý xã hội.
    - *Bài toán 2 (Phân loại thiệt hại phổ biến):* Phân nhóm nội dung thành các danh mục thiệt hại chính (Con người, Hạ tầng, Nhà cửa, Tài sản, Kinh tế).
    - *Bài toán 3 (Đánh giá mức độ hài lòng về cứu trợ):* Phân tích phản hồi tích cực/tiêu cực của công chúng đối với từng nhóm nhu yếu phẩm cứu trợ thiết yếu (Lương thực, Y tế, Chỗ ở, Vận chuyển, Tiền mặt...).

2.2. Mở rộng yêu cầu

2.2.1. Bài toán 4 (Phân tích điểm nóng thiên tai - Hotspot Locations)

Để nâng cao năng lực định vị rủi ro và tổng hợp thông tin, hệ thống được mở rộng thêm module chiến lược với thiết kế tối ưu: “Định vị điểm nóng thiên tai (Hotspot Analysis): Lọc ngữ cảnh (Contextual Filtering)”. Hệ thống chỉ ghi nhận một khu vực là "điểm nóng" **nếu** câu văn đi kèm với các từ khóa thiệt hại hoặc sắc thái tiêu cực. Logic này giúp loại bỏ triệt để dữ liệu rác, mang lại bản đồ phân bố rủi ro có độ tin cậy tuyệt đối để lực lượng cứu hộ khoanh vùng chi viện.

2.2.2. Tích hợp Trí tuệ nhân tạo (AI Generative Summary)

Sử dụng các mô hình ngôn ngữ lớn (LLM) như Google Gemini và Llama-3.1 qua Groq để tự động đọc dữ liệu biểu đồ phân tích và đưa ra các bản tóm tắt, nhận xét định lượng và đề xuất giải pháp phục hồi khách quan.

2.2.3. Đồng bộ hóa giao diện đa trạng thái (Theme Toggle)

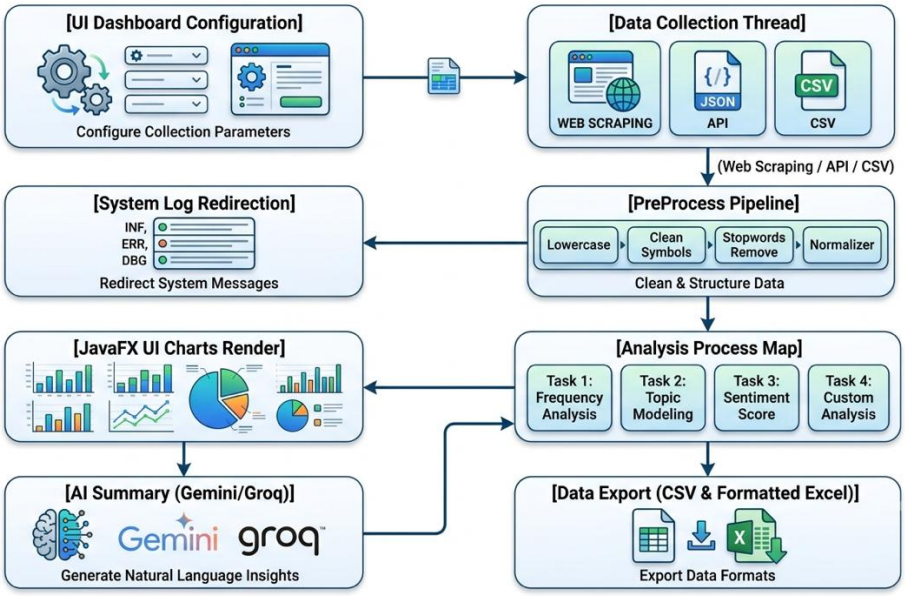
Thiết kế hệ thống cho phép chuyển đổi mượt mà giữa chế độ tối (Dark Mode) và sáng (Light Mode), tự động điều chỉnh cả màu sắc phân vùng biểu đồ.

2.2.4. Hệ thống báo cáo chuyên nghiệp

Cho phép xuất dữ liệu phân tích ra báo cáo Excel định dạng bảng biểu phân cấp, dễ dàng nắm được các thông số quan trọng trên biểu đồ, giúp người xem có một góc nhìn trực quan khác về thảm họa xảy ra.

2.3. Tổng quan hoạt động hệ thống

Quy trình vận hành của hệ thống được tổ chức khép kín qua các giai đoạn:



1. Cấu hình & Thu thập: Người dùng nhập cấu hình trên UI (nguồn, từ khóa, khoảng ngày, số lượng tối đa). Hệ thống khởi chạy luồng thu thập bất đồng bộ (Thread) tương ứng với Collector được chọn để lấy dữ liệu.

2. Lọc & Tiền xử lý: Toàn bộ nội dung thô (bài đăng, bình luận) được đưa qua một đường ống lọc (PreProcessPipeline) để chuẩn hóa chữ thường, xóa ký tự đặc biệt/emoji, loại bỏ từ dừng và chuẩn hóa teencode.

3. Phân tích tích lũy: Dữ liệu sạch được phân phối tới các tác vụ phân tích (AnalysisTask). Kết quả được tích lũy vào các bản đồ kết quả chuyên biệt (Map<String, AnalysisResult>).

4. Trực quan hóa & Xuất bản: Hệ thống dựng biểu đồ trực quan, hỗ trợ gọi API AI tóm tắt nội dung thô của biểu đồ, đồng thời hỗ trợ xuất file CSV lưu trữ gốc hoặc báo cáo Excel tổng hợp.

III. Phân tích & Thiết kế hệ thống

3.1. Cấu trúc dữ liệu và lưu trữ tệp (CSV/JSON)

3.1.1. Cấu trúc dữ liệu trong bộ nhớ (In-memory Data Structure)

- Hệ thống biểu diễn thực thể mạng xã hội qua mối quan hệ Aggregation 1 - N giữa Post và Comment, kế thừa từ lớp cha trừu tượng SocialItem. Một đối tượng Post duy trì một danh sách các đối tượng bình luận qua thuộc tính private final List<Comment> comments.

- Đối với dữ liệu phân tích, cấu trúc AnalysisResult làm cơ sở để phân nhánh thành CountNum (lưu trữ số lượng đếm nguyên) và SentimentResult (lưu trữ cặp số lượng tích cực và tiêu cực song song).

3.1.2. Lưu trữ tệp tin CSV (Định dạng phân cấp)

- Để lưu trữ mối quan hệ phân cấp post-comment vào một tệp phẳng CSV mà vẫn đảm bảo tính toàn vẹn khi đọc lại ngoại tuyến (offline), nhóm thiết kế cấu trúc cột phân dòng thông qua thuộc tính định danh hàng Type theo chuẩn RFC 4180:

Type	ID	ParentID	Date	Content	Likes	Comments
POST	1830219		2024-09-08 14:20:00	"Nội dung bài viết..."	120	5
COMMENT	9021831	1830219	2024-09-08 14:22:15	"Nội dung bình luận..."	12	0

    - Dòng có Type = POST đại diện cho một bài viết thô. Cột ParentID được để trống.

- Dòng có Type = COMMENT đại diện cho một bình luận.
  Cột ParentID bắt buộc phải chứa giá trị khớp với ID của bài viết cha chứa nó.

- Khi nạp dữ liệu bằng FileCollector, hệ thống sử dụng cấu trúc Map<String, Post> để tạm lưu trữ các bài viết cha theo ID. Khi gặp dòng COMMENT, hệ thống chỉ việc truy vấn nhanh trong bản đồ để lấy đối tượng bài viết cha tương ứng và gọi phương thức addComment() để tái thiết lập liên kết phân cấp trong bộ nhớ.

3.1.3. Xử lý cấu trúc dữ liệu JSON từ API

- Trong BaseRestApiCollector, thư viện Gson được sử dụng để phân tích cú pháp chuỗi JSON tĩnh nhận về từ các API của Google YouTube hoặc RapidAPI TikTok.

- Hệ thống chuyển đổi JSON sang các cấu trúc JsonObject và JsonArray trung gian trước khi trích xuất giá trị trường dữ liệu gốc (như trường publishedAt kiểu ISO 8601, trường digg_count kiểu số nguyên).

3.1.4. Bóc tách cấu trúc dữ liệu từ HTML (Web Scraping)

- Đối với các nguồn báo điện tử (VnExpress, Dân Trí, VietNamNet), hệ thống sử dụng thư viện **Jsoup** để phân tích cú pháp cấu trúc cây DOM HTML thông qua lớp BaseHtmlScraper với các bước xử lý tối giản:

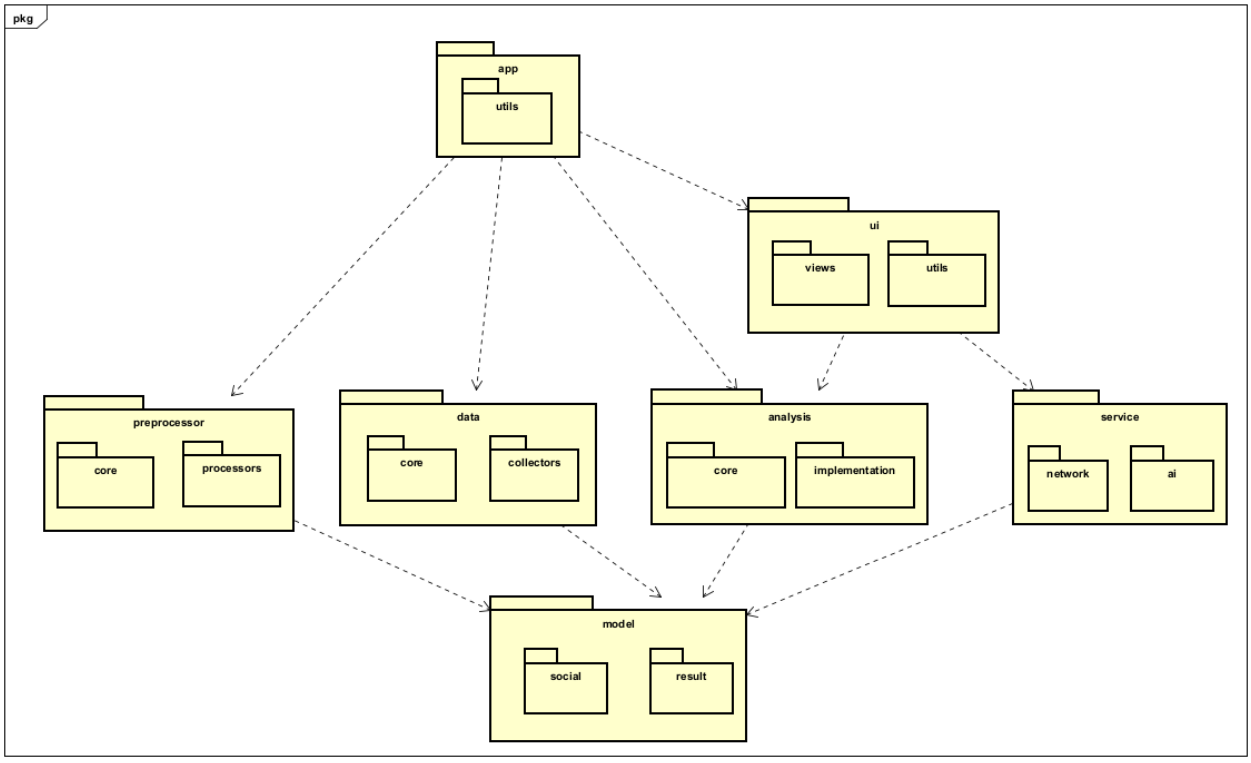
  - **Lọc liên kết:** Sử dụng CSS Selector động từ getArticleLinkSelector() để tìm các thẻ <a> và bóc tách URL chi tiết tuyệt đối (abs:href).

  - **Bóc tách nội dung:** Sử dụng các bộ chọn CSS chuyên biệt (getTitleSelector(), getSapoSelector(), getContentSelector()) để lấy văn bản Tiêu đề, Sapo và Nội dung chi tiết. Ba phần này được ghép lại thành trường dữ liệu rawContent hoàn chỉnh.

  - **Xử lý ngày và ID:** Định vị thẻ chứa ngày qua getDateSelector(), dùng biểu thức chính quy (Regex) để trích lọc chuỗi ngày tháng

dạng dd/MM/yyyy HH:mm. Mã bài viết (Post ID) được cắt trực tiếp từ slug của URL bài viết, hoặc dùng mã băm hashCode() của URL làm ID dự phòng để đảm bảo tính duy nhất.

3.2. Biểu đồ phụ thuộc gói (UML)



- Sơ đồ thể hiện rõ cấu trúc phân tầng bậc thang với luồng phụ thuộc (các mũi tên nét đứt) tuân thủ tuyệt đối quy tắc một chiều (Unidirectional Dependency) – chỉ hướng từ tầng cao xuống tầng thấp, không có phụ thuộc ngược hay phụ thuộc vòng (Circular Dependency). Sơ đồ này phản ánh cực kỳ xuất sắc sự giao thoa giữa hai tư tưởng thiết kế hiện đại: **Kiến trúc Phân tầng (Layered Architecture)** và **Đóng gói theo Tính năng (Package by Feature)**.

- **Tầng Ứng dụng & Giao diện (Top Layer):** Bao gồm gói app (điều phối luồng khởi chạy, tiện ích chung) và gói ui (hiển thị giao diện người dùng).

Đây là các thành phần giao tiếp trực tiếp với người dùng và đóng vai trò gọi lệnh xuống các tầng dưới.

- **Tầng Nghiệp vụ & Hạ tầng (Middle Layer):** Gồm các gói preprocessor, data, analysis, và service. Tầng này chứa toàn bộ "não bộ" xử lý của hệ thống: từ việc thu thập dữ liệu (data), làm sạch văn bản (preprocessor), phân tích chuyên sâu (analysis), cho đến giao tiếp với mạng và AI bên ngoài (service).

- **Tầng Dữ liệu cốt lõi (Bottom Layer - Domain Model):** Gói model nằm ở dưới cùng của biểu đồ và **không có bất kỳ mũi tên nào trỏ ra ngoài**. Nó chứa các thực thể nền tảng (như Post, Comment, Result). Mọi gói khác đều phụ thuộc vào model, nhưng model hoàn toàn độc lập, không biết đến sự tồn tại của bất kỳ module nào khác. Điều này bảo vệ cấu trúc dữ liệu cốt lõi khỏi những thay đổi từ giao diện hay thuật toán bên ngoài.

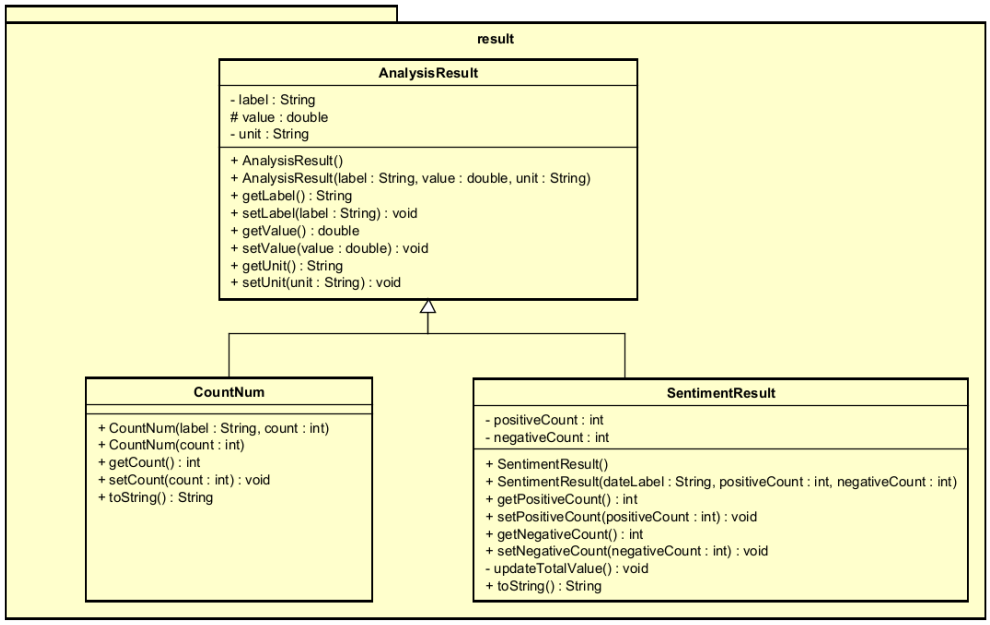
3.3. Biểu đồ lớp (UML) & phân tích chi tiết lớp

Việc trình bày biểu đồ lớp được nhóm thực hiện theo cách tiếp cận từ dưới lên (bottom-up): bắt đầu phân tích từ các lớp cơ sở độc lập -> tiếp đến là cấu trúc nội bộ trong từng gói (package) -> cuối cùng là làm rõ mối quan hệ tương tác tổng thể giữa các gói với nhau.

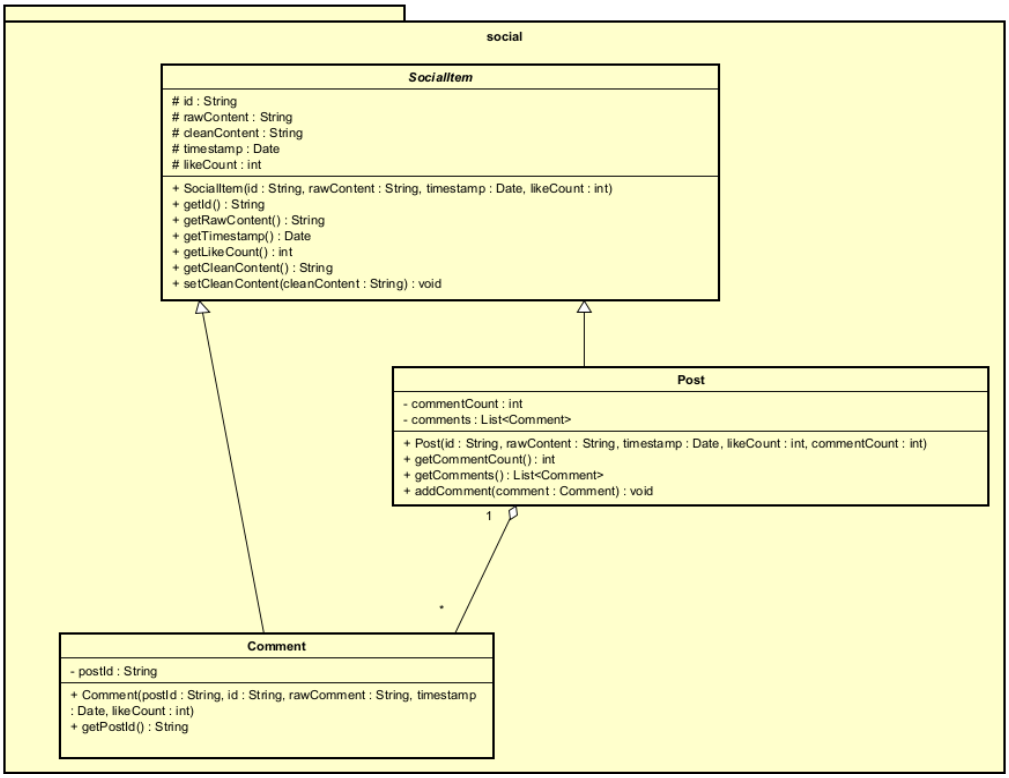
3.3.1. Các lớp cơ sở độc lập & cấu trúc nội bộ trong từng gói

3.3.1.1. model

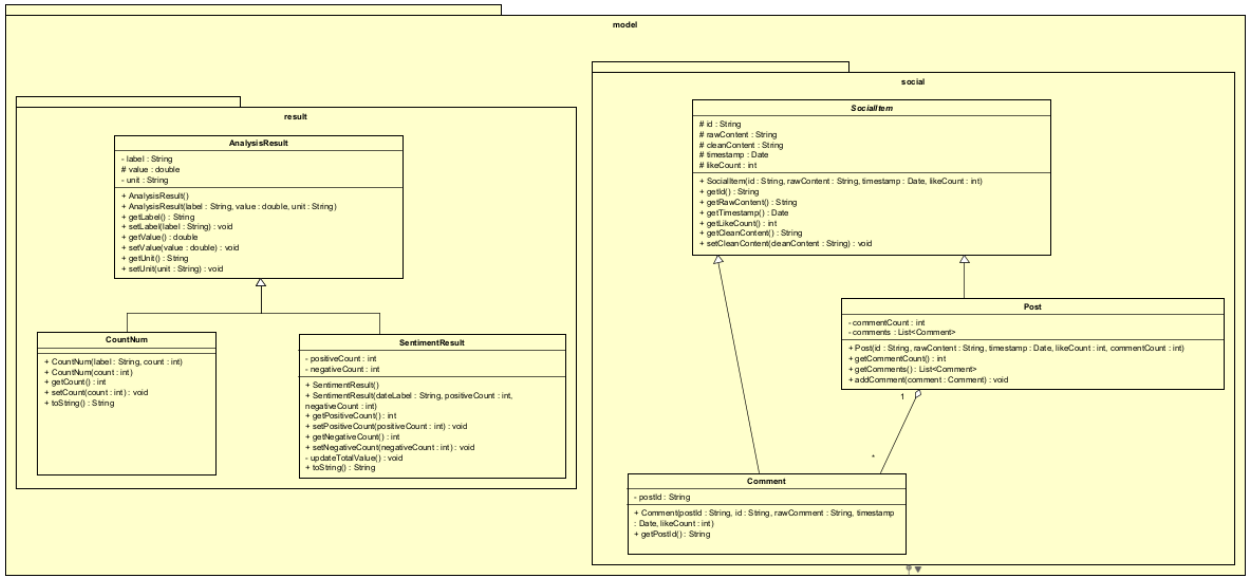
- **model.result**



- **model.social**



- **model**



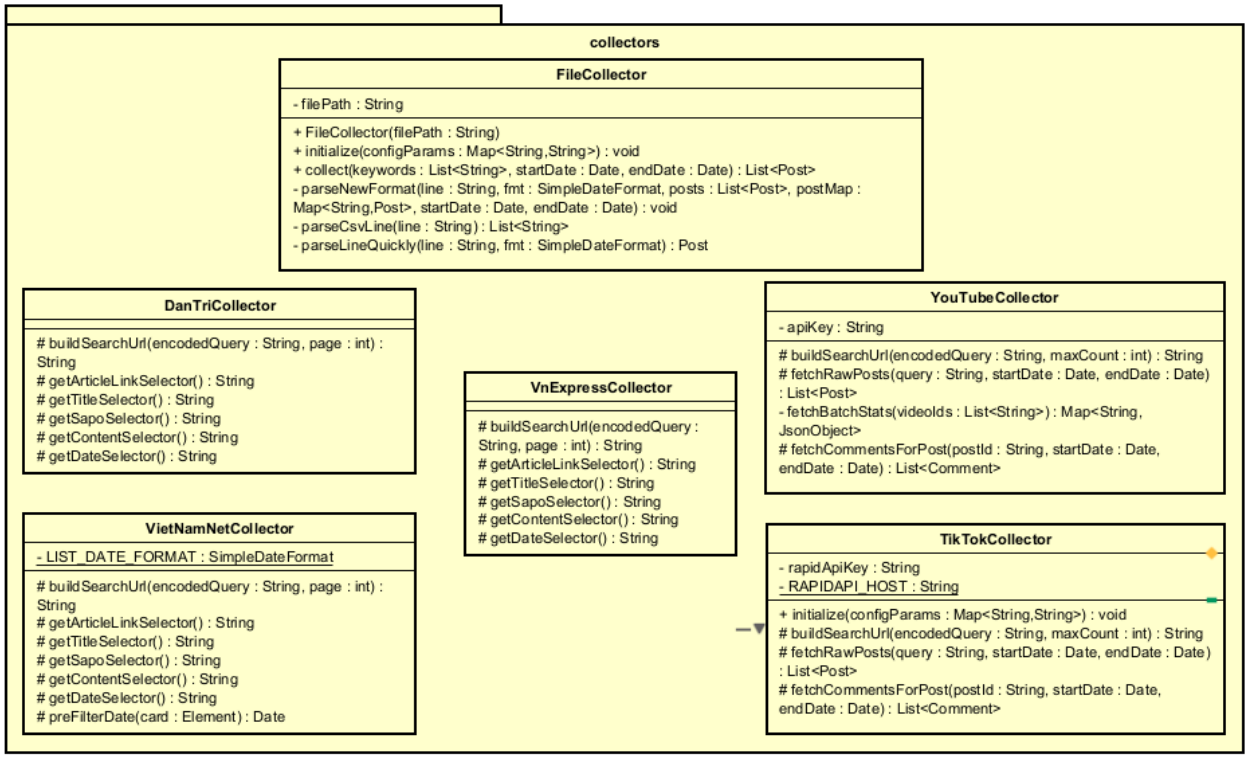
model: Tầng thực thể (Entity/Domain Layer), nơi định nghĩa các đối tượng cốt lõi phản ánh chính xác nghiệp vụ của hệ thống. Tầng này được thiết kế hoàn toàn độc lập, không phụ thuộc vào giao diện (UI) hay các thuật toán phức tạp bên ngoài, đảm bảo tính toàn vẹn của dữ liệu.

- **result:** Gói chứa các cấu trúc dữ liệu chuẩn hóa, làm nhiệm vụ đóng gói và vận chuyển kết quả đầu ra từ tầng phân tích lên tầng giao diện để vẽ biểu đồ.
  - **AnalysisResult:** Lớp cơ sở (Base Class) mang tính tổng quát, đóng gói (Encapsulation) các thuộc tính chung nhất cho mọi loại báo cáo (như nhãn label, giá trị value, đơn vị unit).
  - **CountNum:** Lớp con kế thừa từ AnalysisResult, chuyên đặc tả các kết quả phân tích mang tính chất thống kê, đếm số lượng thuần túy (ví dụ: đếm tần suất xuất hiện của từ khóa, tổng số lượng nhà sập, số người cần cứu trợ …).
  - **SentimentResult:** Lớp con kế thừa từ AnalysisResult, được thiết kế chuyên biệt cho bài toán phân tích tâm lý đám đông. Lớp này bổ sung thêm các trường dữ liệu đặc thù để lưu trữ chi tiết số lượng tương tác tích cực (positiveCount) và tiêu cực (negativeCount).

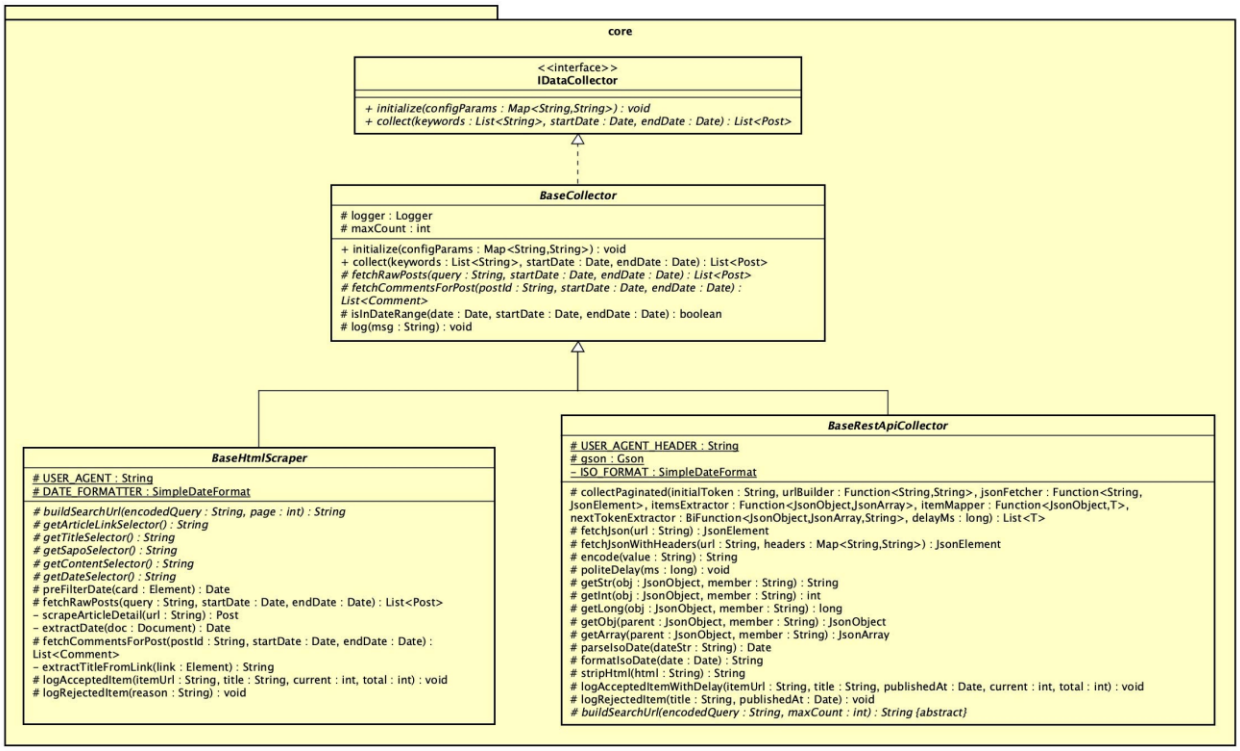
- **social:** Gói mô hình hóa các đối tượng dữ liệu văn bản thô được thu thập (crawl) về từ các nền tảng mạng xã hội.
    - **SocialItem:** Lớp cha (Superclass) đóng vai trò nền tảng, chứa đựng các thuộc tính cốt lõi mà bất kỳ nội dung mạng xã hội nào cũng bắt buộc phải có (như id, rawContent, cleanContent, timeStamp, likeCount). Việc gom nhóm này tuân thủ tuyệt đối nguyên lý DRY (Don't Repeat Yourself), giúp các lớp con kế thừa không bị trùng lặp mã nguồn.
    - **Post:** Lớp đại diện cho một bài đăng gốc. Điểm nhấn kiến trúc ở đây là mối quan hệ Tập hợp (Aggregation loại 1-n) với lớp **Comment**. Một đối tượng **Post** sẽ chứa một danh sách (**List<Comment>**) để phản ánh đúng cấu trúc phân cấp thực tế: một bài đăng luôn chứa nhiều bình luận bên trong nó.
    - **Comment:** Lớp đại diện cho các dòng phản hồi của người dùng. Nhờ kế thừa từ **SocialItem**, nó tự động thừa hưởng toàn bộ cấu trúc văn bản của lớp cha, đồng thời chỉ cần định nghĩa thêm duy nhất thuộc tính **postId** để làm khóa ngoại ánh xạ ngược trở lại bài viết gốc.

3.3.1.2. data

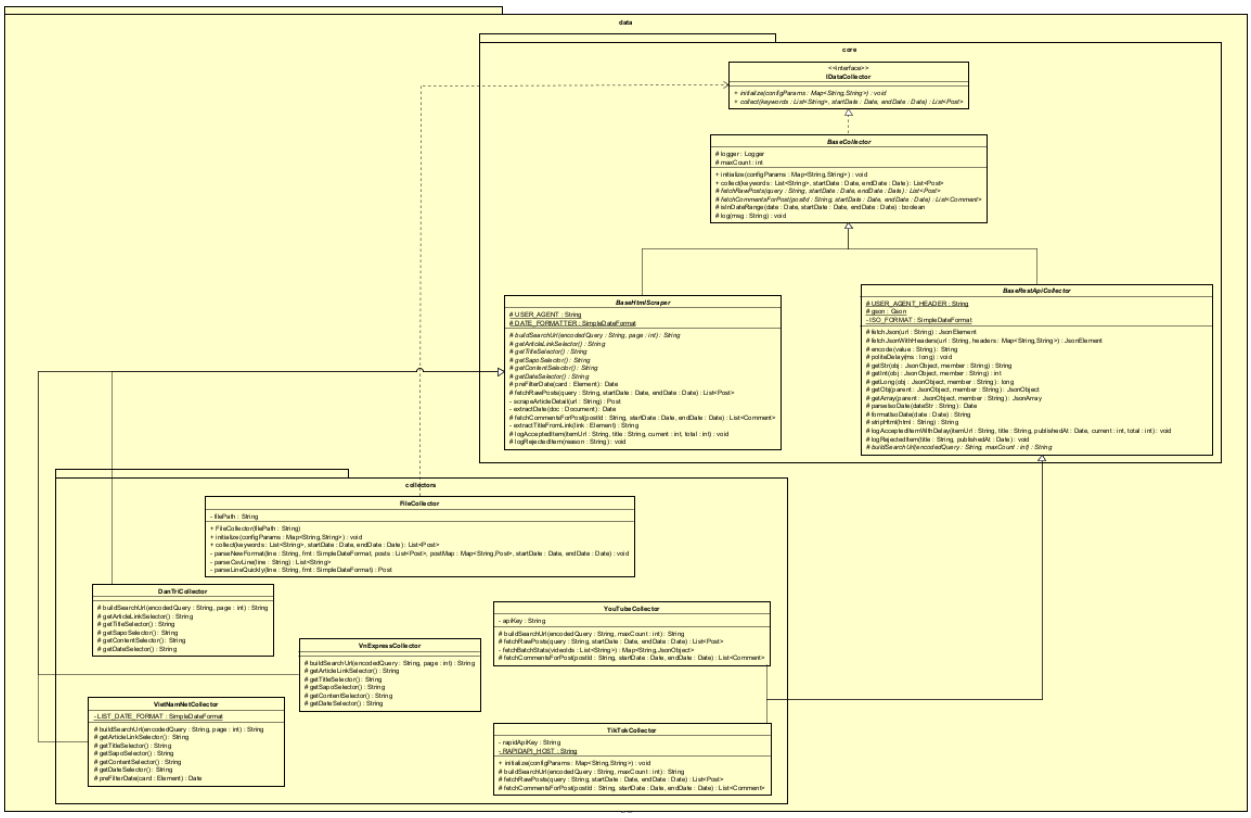
- **data.collectors**



- **data.core**



- **data**



data: Tầng Thu thập Dữ liệu (Crawling Layer), chịu trách nhiệm trích xuất và nạp dữ liệu thô. Tầng này được thiết kế theo nguyên lý Mở/Đóng (Open/Closed Principle) – cực kỳ dễ dàng cắm thêm các nguồn dữ liệu mới mà không làm hỏng cấu trúc cũ.

- **core:** Gói nền tảng (Core Framework), cung cấp bộ khung tiêu chuẩn và các logic dùng chung cho toàn bộ quá trình thu thập.
    - **IDataCollector:** Giao diện (Interface) gốc, đóng vai trò như một "hợp đồng" (Contract) bắt buộc mọi bộ cào phải tuân thủ thông qua hai phương thức cốt lõi là initialize() và collect().
    - **BaseCollector:** Lớp trừu tượng (Base Class). Bằng việc ứng dụng mẫu thiết kế **Template Method**, lớp này đã định hình sẵn một quy trình thu thập chuẩn mực, loại bỏ hoàn toàn việc lặp lại mã nguồn (tuân thủ nguyên lý DRY).

- **BaseHtmlScraper:** Lớp trừu tượng chuyên trách nhánh Web Scraping. Lớp này gồm sẵn các công cụ mạnh mẽ để phân tích bóc tách cú pháp HTML. Nó ép các lớp con chỉ việc khai báo các Selector tương ứng mà không phải tự viết lại logic bóc tách.

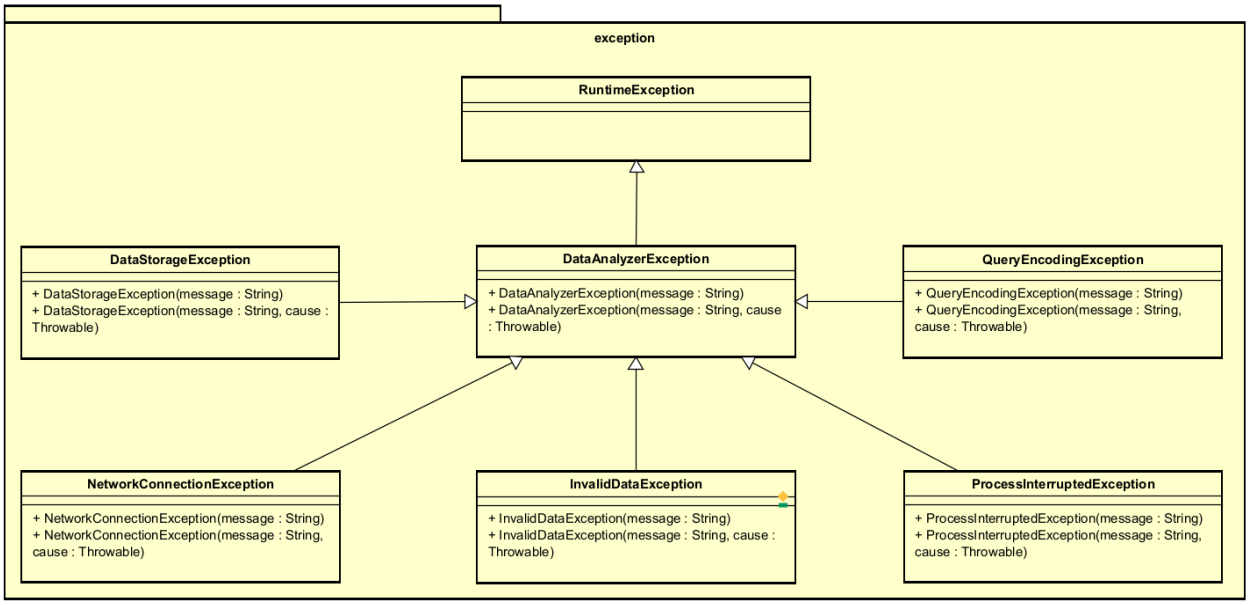
- **BaseRestApiCollector:** Lớp trừu tượng chuyên trách nhánh HTTP/RESTful. Lớp này cung cấp sẵn các tiện ích để xử lý luồng mạng (Network Header, API Key), mã hóa URL và bóc tách cấu trúc dữ liệu JSON phức tạp.

- **collectors:** Gói chứa các lớp thực thi chi tiết. Mỗi lớp ở đây tuân thủ tuyệt đối nguyên lý Đơn trách nhiệm (Single Responsibility Principle - SRP), chỉ giải quyết việc lấy dữ liệu cho đúng một nền tảng duy nhất.
  - **FileCollector:** Bộ nạp dữ liệu ngoại tuyến (Offline). Hỗ trợ đọc, phân giải và giả lập dữ liệu mạng xã hội trực tiếp từ các tệp CSV cục bộ, cực kỳ hữu ích cho việc lấy dữ liệu từ dự án cũ hoặc khi không có Internet.

  - **VnExpressCollector, DanTriCollector, VietNamNetCollector:** Nhóm bộ cào báo điện tử truyền thống. Kế thừa từ BaseHtmlScraper, các lớp này có cấu trúc siêu mỏng. Chúng chỉ làm duy nhất một việc là khai báo (override) các đường dẫn XPath/CSS Selector (ví dụ: getTitleSelector, getContentSelector) để tương thích với giao diện riêng của từng trang báo.

  - **YouTubeCollector, TikTokCollector:** Nhóm bộ thu thập mạng xã hội hiện đại. Trực tiếp kế thừa từ BaseRestApiCollector, các lớp này chịu trách nhiệm đóng gói tham số truy vấn, xác thực API Key và chuyển hóa các chuỗi JSON khổng lồ trả về từ máy chủ thành các đối tượng Post và Comment có cấu trúc.

3.3.1.3. exception



exception: Gói quản lý và chuẩn hóa các ngoại lệ (Custom Exception) của toàn bộ hệ thống. Việc xây dựng một cơ chế xử lý lỗi riêng biệt giúp ứng dụng dễ dàng khoanh vùng sự cố, đồng thời cung cấp các thông điệp báo lỗi rõ ràng cho người dùng thay vì để ứng dụng bị "văng" (crash) đột ngột.

- **DataAnalyzerException:** Lớp ngoại lệ cơ sở nền tảng của dự án, kế thừa trực tiếp từ RuntimeException của Java. Bằng cách thiết kế theo hướng Unchecked Exception, hệ thống giúp mã nguồn trở nên "sạch" và gọn gàng hơn, giải phóng các lớp nghiệp vụ khỏi việc phải khai báo throws hay lồng ghép các khối try-catch dư thừa khắp mọi nơi.

- **DataStorageException:** Lớp ngoại lệ chuyên biệt dùng để bắt và thông báo các sự cố liên quan đến thao tác I/O (Input/Output), điển hình như lỗi không thể đọc/ghi tệp tin CSV, thiếu quyền truy cập thư mục hoặc tràn bộ nhớ lưu trữ.

- **NetworkConnectionException:** Lớp chịu trách nhiệm kiểm soát các rủi ro về đường truyền. Nó sẽ được ném ra (throw) khi hệ thống gặp sự cố mất kết nối mạng, máy chủ API của mạng xã hội hoặc AI từ chối phản hồi, hoặc yêu cầu bị quá hạn thời gian (timeout)

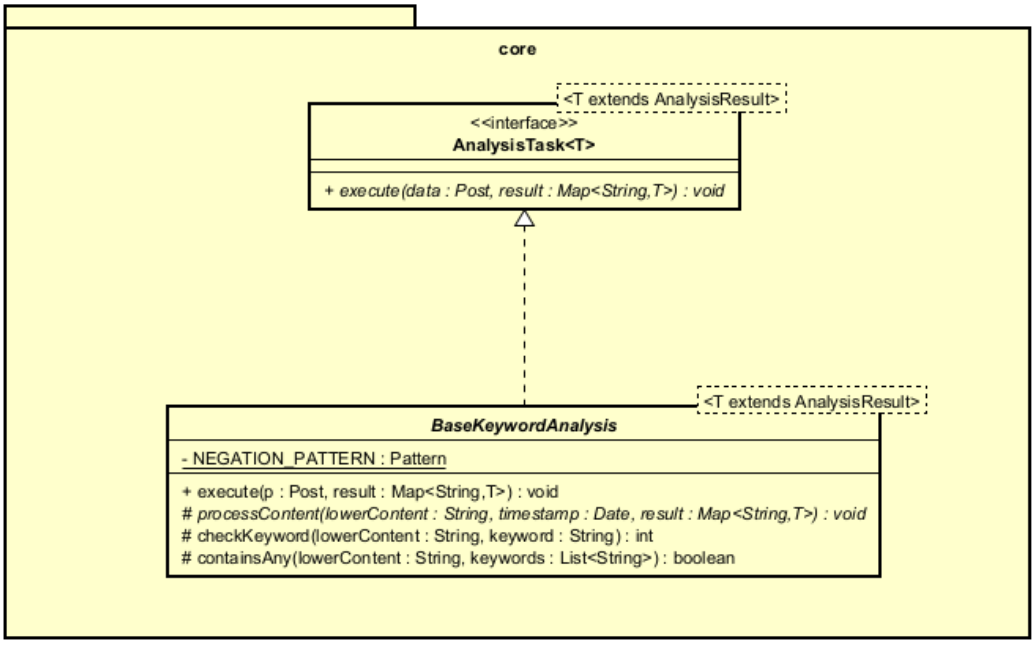
- **InvalidDataException:** Lớp xử lý các vi phạm về tính toàn vẹn dữ liệu. Nó được kích hoạt khi hệ thống nhận phải dữ liệu đầu vào rỗng (null), sai cấu trúc định dạng hoặc không vượt qua được các ràng buộc kiểm duyệt của nghiệp vụ.

- **ProcessInterruptedException:** Lớp ngoại lệ báo hiệu sự cố liên quan đến luồng (Thread). Thường được sử dụng để bắt các trường hợp một tiến trình phân tích đang chạy thì bị gián đoạn đột ngột do lỗi hệ thống hoặc do người dùng chủ động can thiệp (ví dụ: nhấn nút Stop trên giao diện).

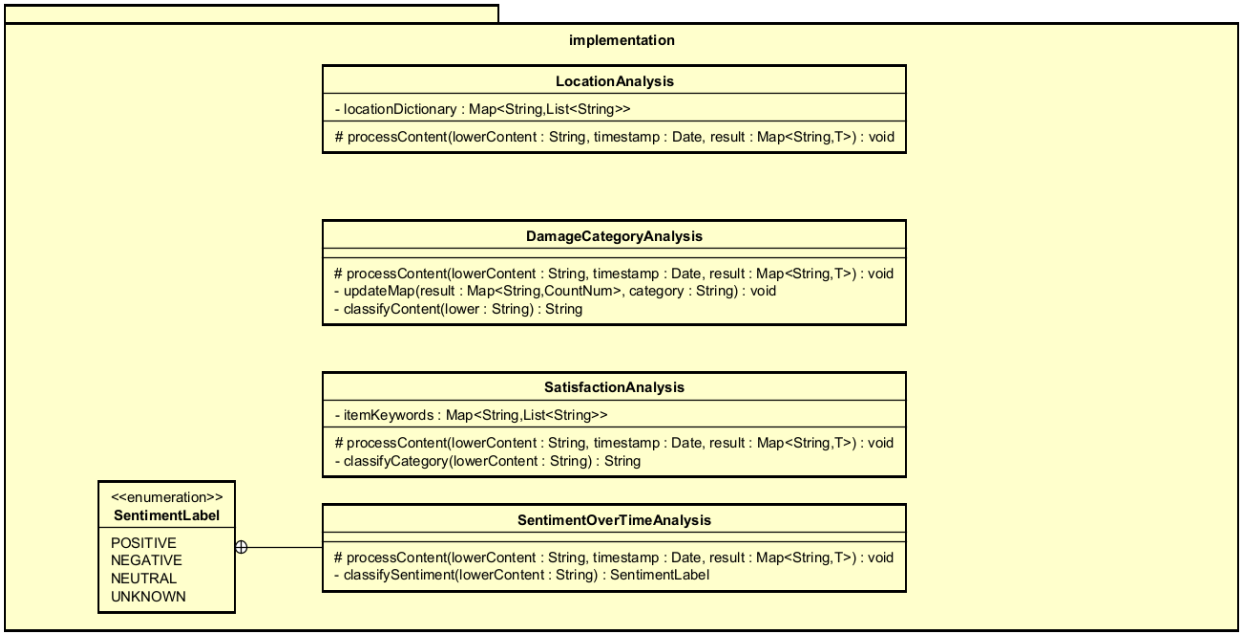
- **QueryEncodingException:** Lớp xử lý các lỗi đặc thù phát sinh trong quá trình biên dịch và mã hóa chuỗi ký tự (URL Encoding). Cực kỳ quan trọng để đảm bảo các từ khóa tìm kiếm (chứa dấu tiếng Việt, ký tự đặc biệt) được chuyển đổi an toàn trước khi gắn vào các đường dẫn API mạng xã hội.

3.3.1.4. analysis

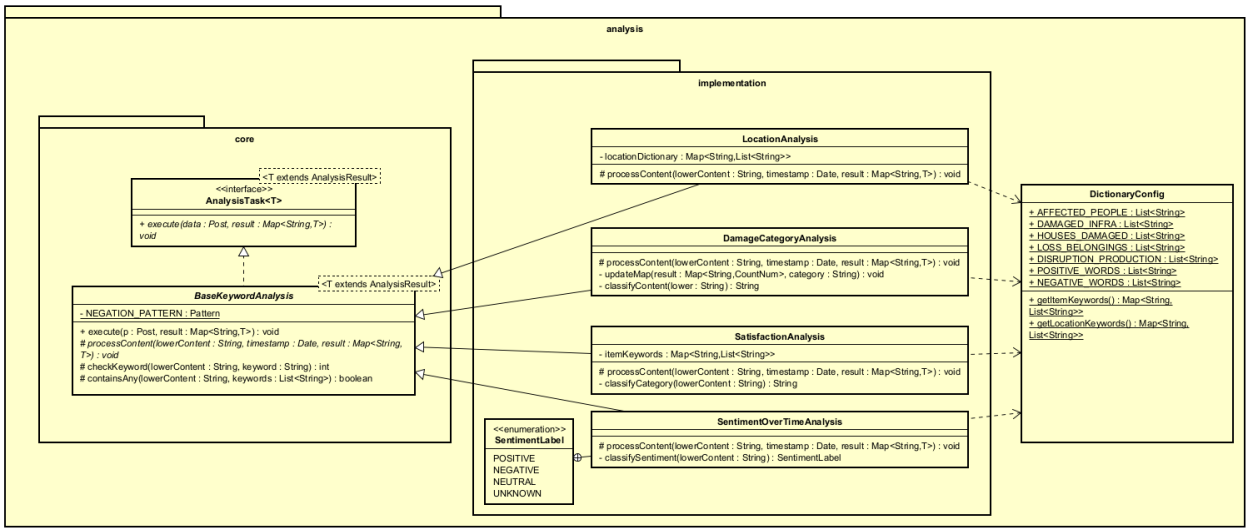
- **analysis.core**



- **analysis.implementation**



- **analysis**



analysis: Tầng xử lý nghiệp vụ cốt lõi (Business Logic Layer), đảm nhiệm toàn bộ các thuật toán bóc tách, phân loại và đánh giá dữ liệu. Kiến trúc tầng này được thiết kế theo hướng module hóa, tối ưu để dễ dàng cắm thêm (plug-in) các bài toán phân tích mới trong tương lai mà không làm vỡ cấu trúc cũ.

- **core:** Gói nền tảng, định nghĩa bộ khung chuẩn mực cho mọi tác vụ phân tích.

- **AnalysisTask<T>:** Giao diện (Interface) gốc ứng dụng Generics (<T AnalysisResult extends>). Lớp này đảm bảo tính Đa hình (Polymorphism), ép buộc mọi thuật toán phân tích đều phải tuân thủ chung một "hợp đồng" là hàm thực thi execute(), giúp hệ thống đồng nhất chuẩn giao tiếp đầu vào và đầu ra.

- **BaseKeywordAnalysis:** Lớp trừu tượng (Abstract Class) đóng vai trò là "xương sống" của module. Bằng cách áp dụng mẫu thiết kế **Template Method Pattern**, lớp này cài đặt sẵn các logic phức tạp dùng chung.

- **implementation:** Gói thực thi, chứa các thuật toán giải quyết trực tiếp 4 bài toán nghiệp vụ của đề tài. Nhờ kế thừa từ BaseKeywordAnalysis, các lớp này có cấu trúc rất "sạch", chỉ cần tập trung ghi đè (override) phương thức processContent() để xử lý logic đặc thù của riêng nó.
  - **LocationAnalysis:** Lớp nghiệp vụ chuyên đối chiếu từ điển để bóc tách và thống kê các địa danh, khu vực chịu ảnh hưởng bởi thảm họa.

  - **DamageCategoryAnalysis:** Lớp đảm nhiệm việc phân tích văn bản để phân loại và đếm tần suất các báo cáo thiệt hại (nhà cửa, con người, cơ sở hạ tầng, tài sản).

  - **SatisfactionAnalysis:** Lớp phân tích đo lường mức độ hài lòng hoặc bức xúc của công chúng đối với từng hạng mục cứu trợ thiết yếu.

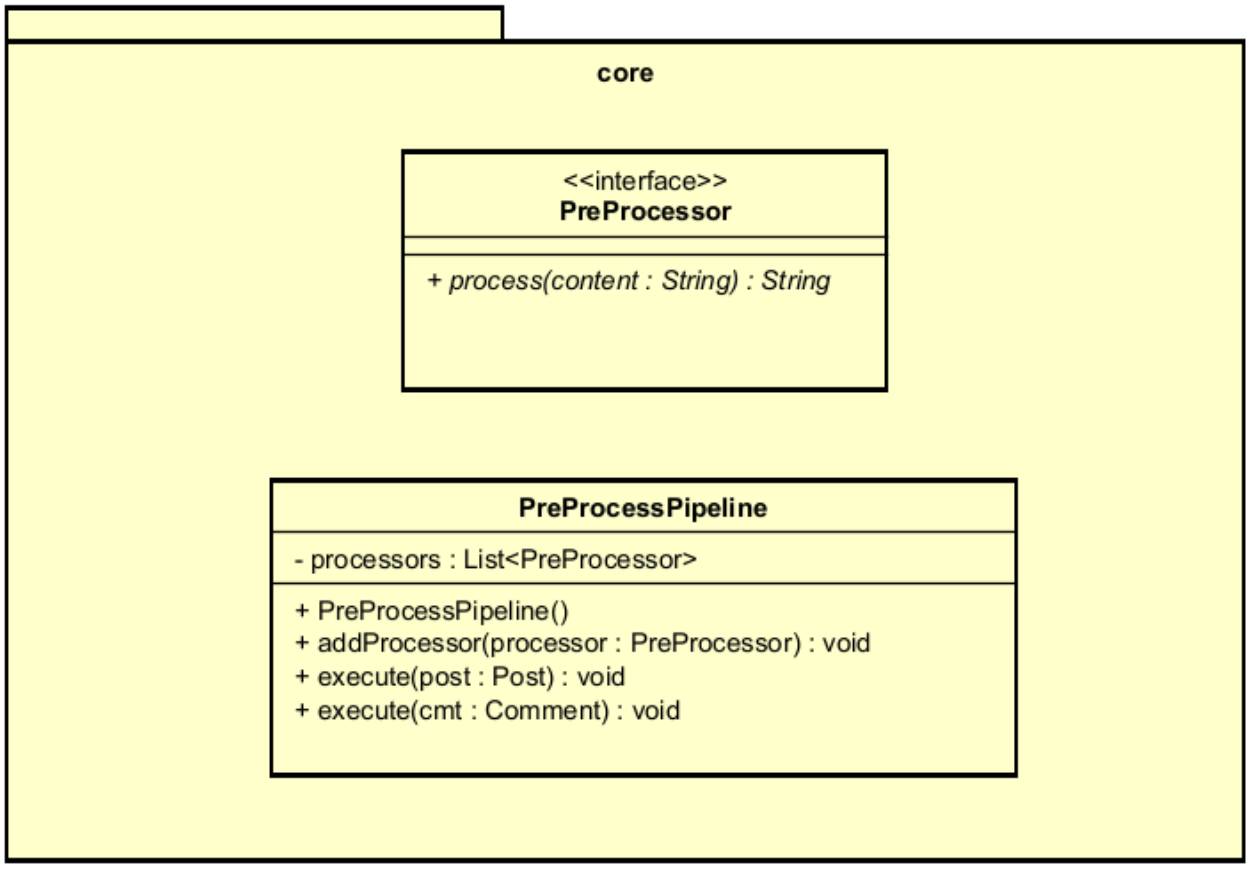
  - **SentimentOverTimeAnalysis:** Lớp chuyên trách phân loại câu từ vào các nhóm cảm xúc để theo dõi sự dịch chuyển tâm lý đám đông theo trục thời gian thực.

- **SentimentLabel:** Kiểu dữ liệu liệt kê (Enum) định nghĩa chặt chẽ các nhãn trạng thái cảm xúc (POSITIVE, NEGATIVE, NEUTRAL, UNKNOWN).

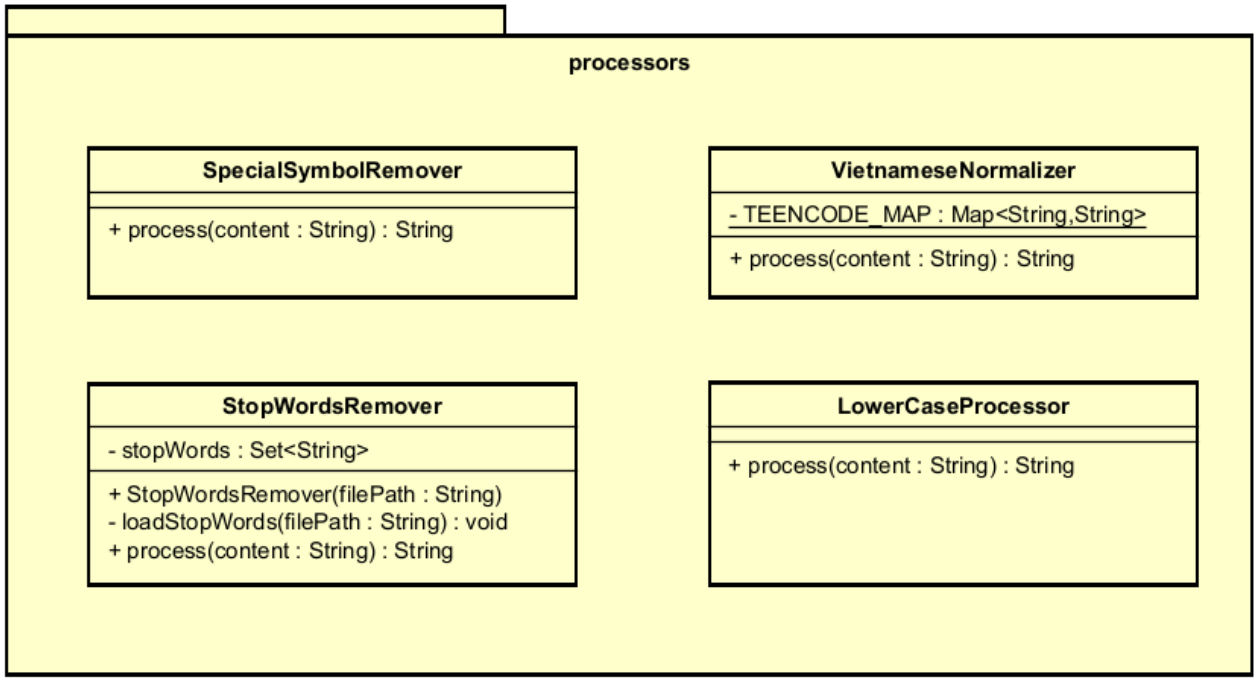
- **DictionaryConfig:** Lớp cấu hình độc lập đóng vai trò như một kho lưu trữ từ điển tĩnh (Static Registry). Lớp này cung cấp các bộ từ khóa đặc tả được phân loại sẵn (ví dụ: AFFECTED_PEOPLE, NEGATIVE_WORDS) để các lớp trong gói implementation gọi ra sử dụng.

3.3.1.5. preprocessor

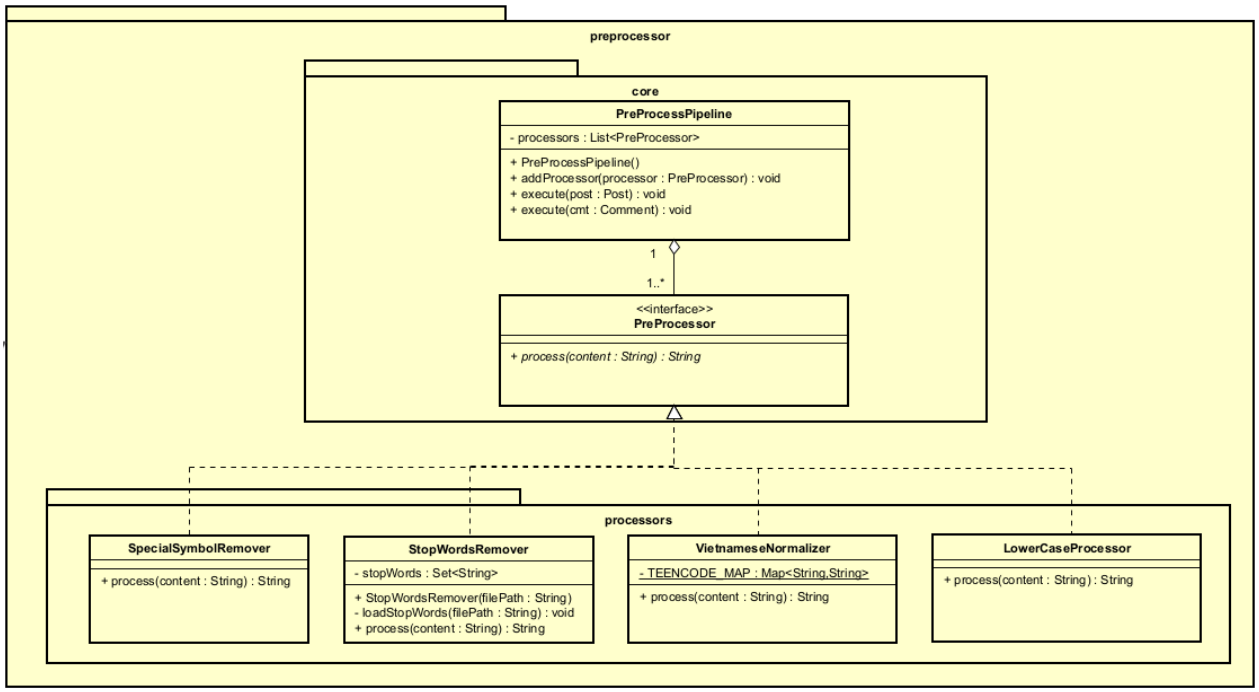
- **preprocessor.core**



- **preprocessor.processors**



- **preprocessor**



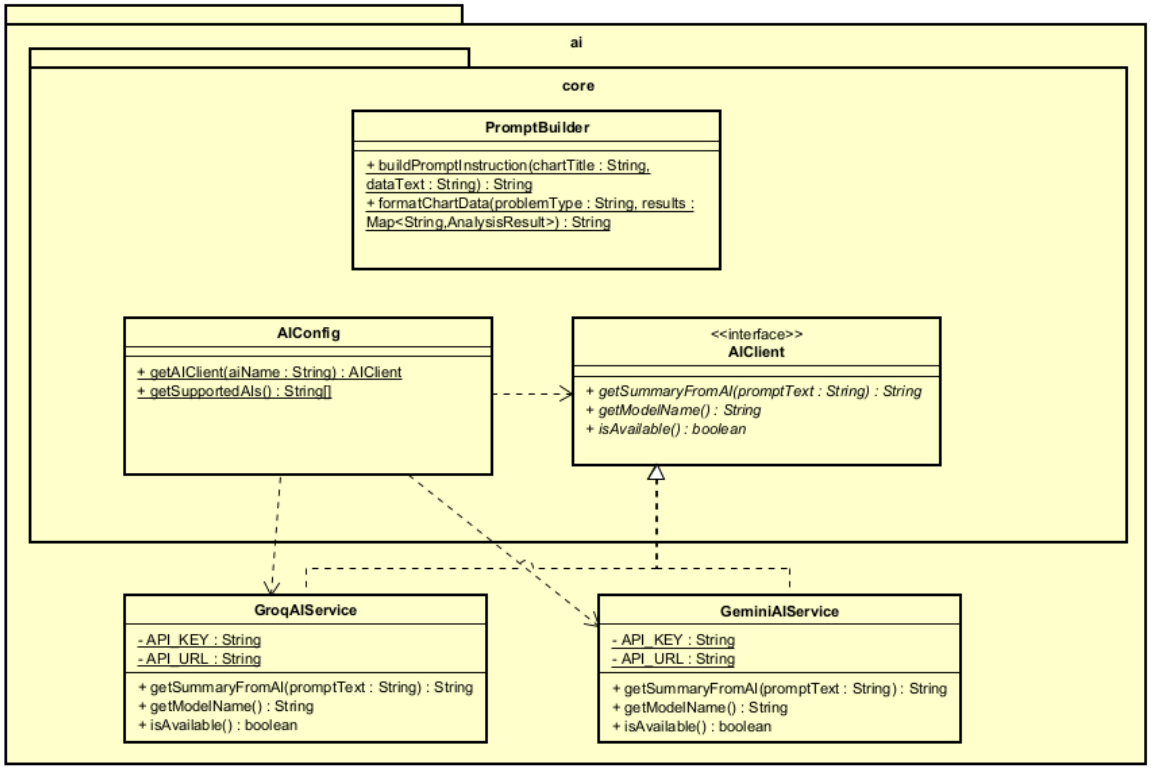
preprocessor: Tầng tiền xử lý dữ liệu, đóng vai trò như một màng lọc, chịu trách nhiệm làm sạch và chuẩn hóa các luồng văn bản thô (raw text) từ mạng xã hội trước khi đưa vào các thuật toán phân tích chuyên sâu.

- **core:** Gói lõi quy định kiến trúc tổng thể của hệ thống tiền xử lý, được thiết kế theo mô hình **Pipeline (Đường ống)** để tối ưu hóa sự linh hoạt.
    - **PreProcessor:** Giao diện (Interface) nền tảng, định nghĩa một "hợp đồng" với phương thức process(). Việc sử dụng interface này đảm bảo tính Đa hình (Polymorphism), cho phép hệ thống gọi chung một hàm xử lý mà không cần quan tâm đến chi tiết logic bên trong của từng bộ lọc.
    - **PreProcessPipeline:** Đóng vai trò là bộ quản lý trung tâm của đường ống xử lý. Lớp này chứa một danh sách (List) các đối tượng PreProcessor và thực thi chúng theo tuần tự. Thiết kế này giúp nhóm dễ dàng "tháo lắp", thêm mới hoặc thay đổi thứ tự các bước làm sạch dữ liệu (như một dây chuyền nhà máy) mà không làm ảnh hưởng đến mã nguồn.
- **processors:** Gói chứa các lớp triển khai (implement) thực tế. Mỗi lớp ở đây tuân thủ nghiêm ngặt nguyên lý Đơn trách nhiệm (Single Responsibility Principle), chỉ giải quyết duy nhất một khâu trong quá trình dọn dẹp văn bản.
    - **LowerCaseProcessor:** Đảm nhiệm việc chuyển đổi toàn bộ chuỗi văn bản về định dạng chữ thường (lowercase) để đồng nhất hóa dữ liệu đầu vào.
    - **SpecialSymbolRemover:** Bộ lọc chuyên bóc tách và loại bỏ các ký tự đặc biệt, dấu câu phức tạp hoặc các biểu tượng không mang lại giá trị ngữ nghĩa cho bài toán phân tích.
    - **StopWordsRemover:** Chịu trách nhiệm tải tệp cấu hình từ khóa và loại bỏ các "từ dừng" (stop words - những từ xuất hiện với tần suất dày đặc nhưng vô nghĩa như "là", "thì", "mà", "ở") để làm gọn dữ liệu.

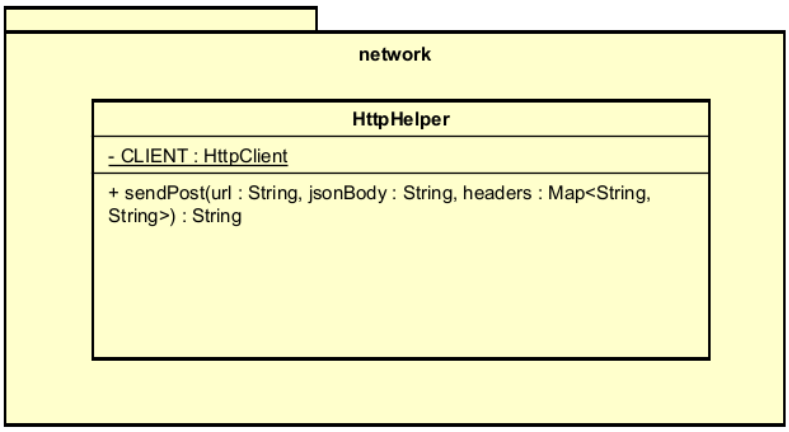
- **VietnameseNormalizer:** Bộ chuẩn hóa ngôn ngữ chuyên biệt, sử dụng cơ chế ánh xạ (TEENCODE_MAP) để dịch và đưa các từ lóng, teencode, hoặc từ viết tắt trên mạng xã hội về định dạng tiếng Việt chuẩn mực.

3.3.1.6. service

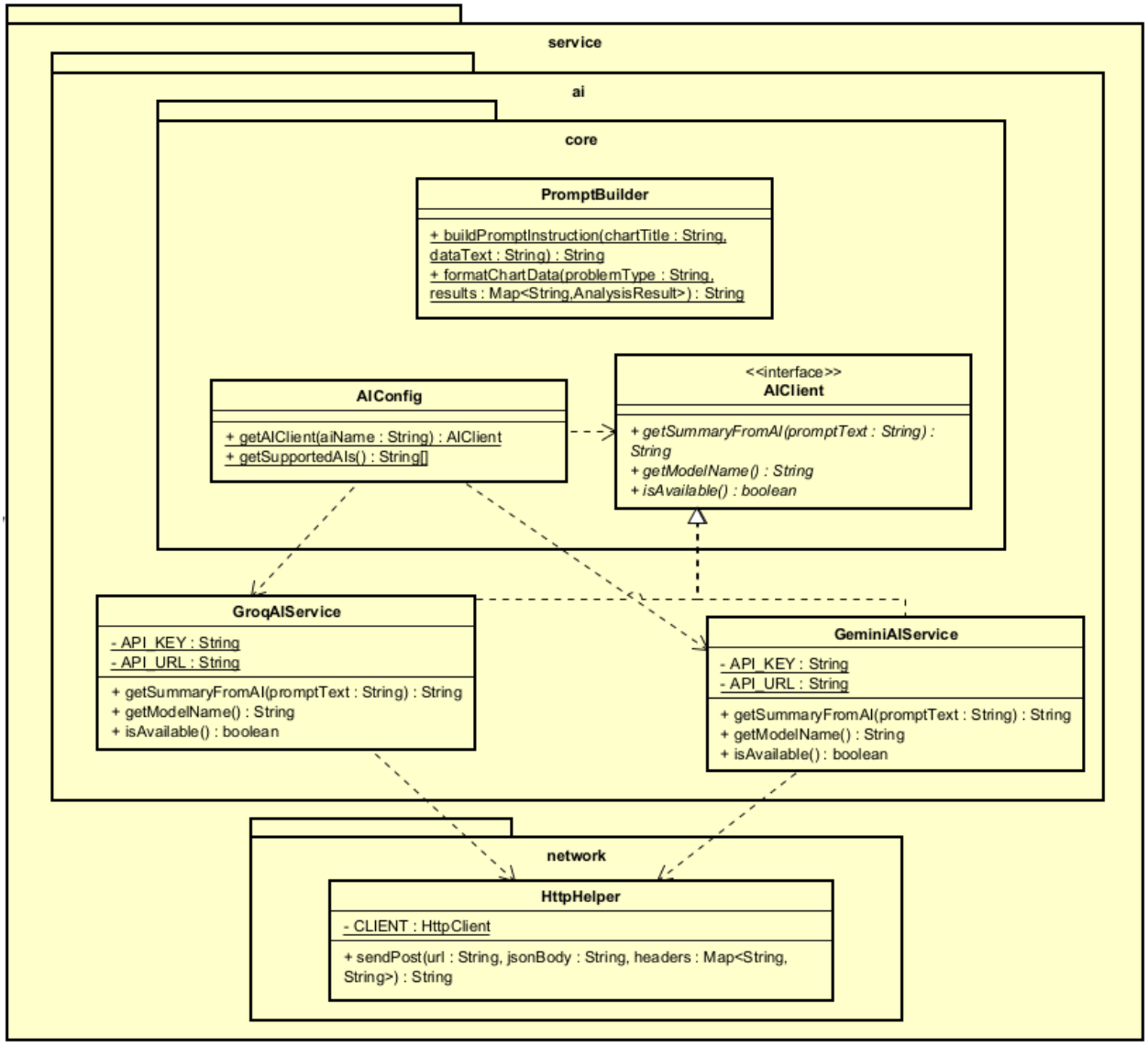
- **service.ai.core**



- **service.network**



- **service**



service: Đóng vai trò là trung tâm giao tiếp của hệ thống, nơi tập trung chủ yếu vào việc tích hợp và khai thác sức mạnh của Trí tuệ nhân tạo (AI).

- **ai.core:** Gói nền tảng, định nghĩa bộ khung kiến trúc chuẩn hóa cho toàn bộ luồng xử lý AI.
    - **AIClient:** Giao diện (Interface) định nghĩa các hành vi tiêu chuẩn bắt buộc cho mọi dịch vụ AI (như chức năng tóm tắt nội dung, kiểm tra trạng thái hoạt động, truy xuất tên model AI).

- **AIConfig:** Đóng vai trò như một Nhà máy (Factory), quản lý cấu hình và tự động khởi tạo kết nối đến đúng mô hình AI qua mà người dùng lựa chọn qua UI.

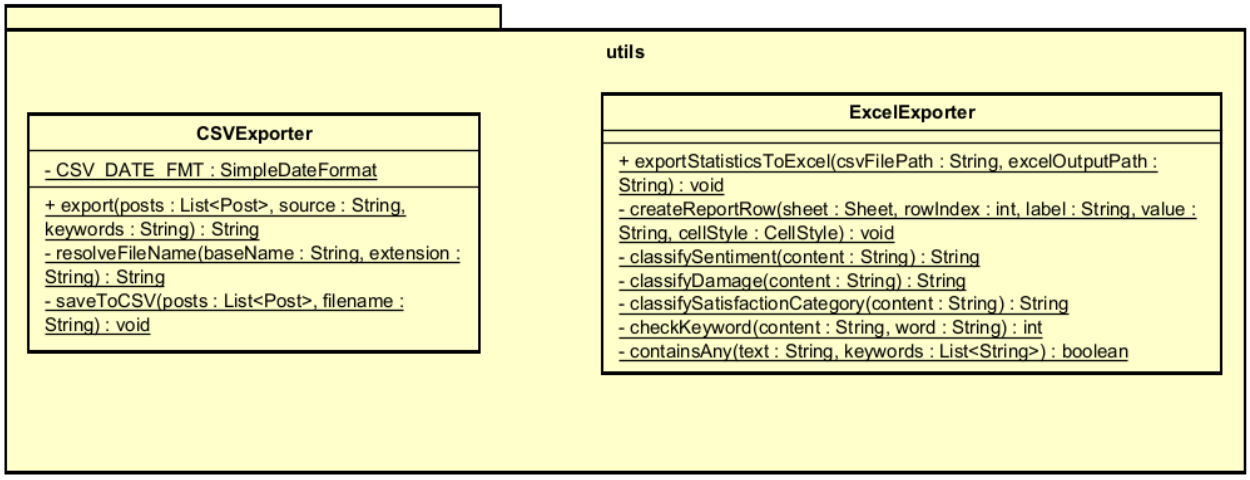
- **PromptBuilder:** Lớp tiện ích nghiệp vụ chuyên đảm nhiệm việc cấu trúc hóa dữ liệu thô và xây dựng các câu lệnh (prompt) tối ưu nhất trước khi gửi cho các mô hình ngôn ngữ lớn (LLM).

- **ai:** Gói thực thi tổng quát, nơi triển khai sức mạnh của các mô hình AI cụ thể.
  - **GeminiAIService, GroqAIService:** Các lớp triển khai chi tiết từ AIClient, chịu trách nhiệm quản lý vòng đời kết nối và trực tiếp gọi API của Google Gemini cùng Groq để xử lý ngôn ngữ tự nhiên.

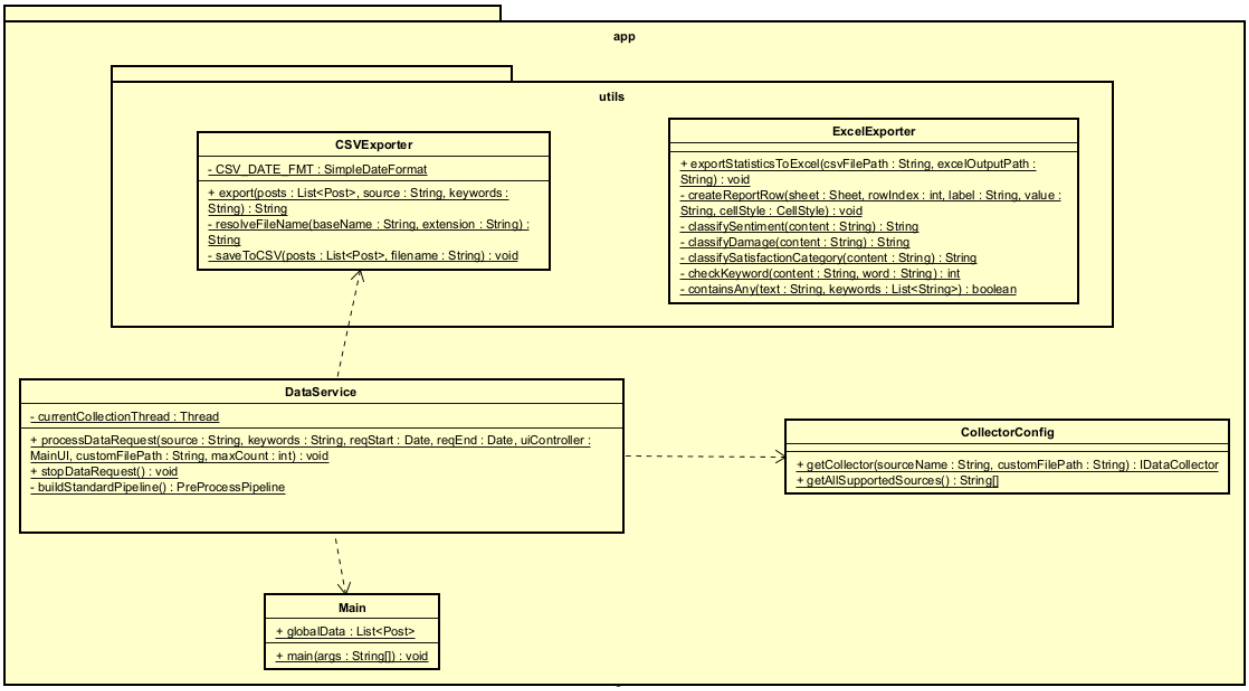
- **network:** Gói hạ tầng mạng, đóng vai trò hỗ trợ đắc lực để các dịch vụ AI có thể giao tiếp với thế giới bên ngoài.
  - **HttpHelper:** Lớp tiện ích cốt lõi xử lý các giao thức truyền tải (như HTTP POST), đảm bảo việc đóng gói và vận chuyển các chuỗi JSON (chứa prompt) từ hệ thống đến các máy chủ AI một cách an toàn và thông suốt.

3.3.1.7. app

- **app.utils**



- **app**



app: Tầng Ứng dụng (Application Layer), nơi chứa điểm neo khởi chạy và các lớp dịch vụ trung tâm. Tầng này làm nhiệm vụ kết dính tất cả các module rời rạc (UI, Data, Analysis, Preprocessor) lại thành một luồng thực thi thống nhất.

- **Main:** Lớp khởi chạy hệ thống (Entry Point). Ngoài việc chứa hàm main() để kích hoạt ứng dụng, lớp này còn đóng vai trò như một kho lưu trữ trạng

thái toàn cục (với biến tĩnh globalData), giúp duy trì và chia sẻ tập dữ liệu bài viết xuyên suốt giữa các màn hình và các tiến trình phân tích.

- **DataService:** Đây là lớp dịch vụ cốt lõi, nó che giấu đi sự phức tạp của hệ thống bên dưới bằng cách cung cấp một hàm duy nhất là processDataRequest(). Lớp này trực tiếp điều phối toàn bộ vòng đời của một phiên làm việc: gọi nhà máy tạo bộ cào dữ liệu, nạp dữ liệu vào đường ống tiền xử lý (PreProcessPipeline), và đặc biệt là quản lý luồng chạy nền (Thread) để quá trình thu thập khối lượng dữ liệu lớn không làm đơ giao diện người dùng.

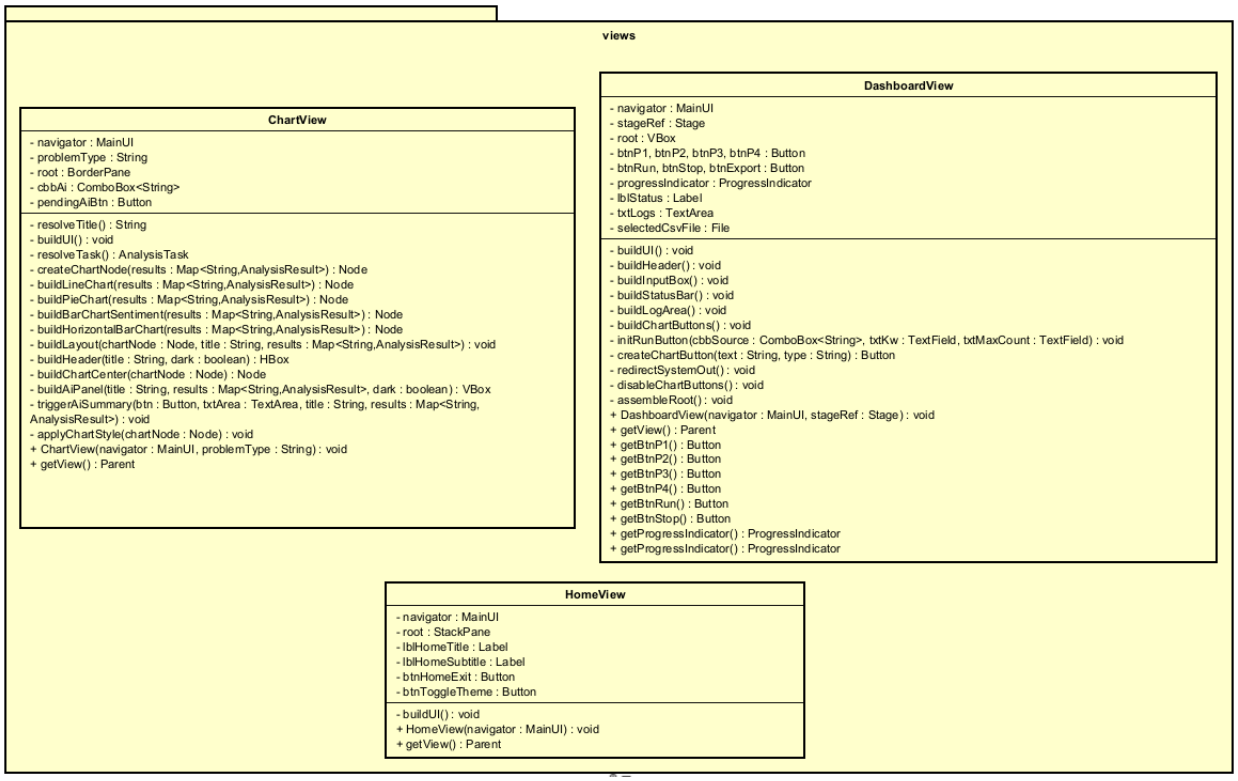
- **CollectorConfig:** Lớp cấu hình áp dụng triệt để **Simple Factory Pattern** (Mô hình Nhà máy), tách biệt hoàn toàn logic khởi tạo đối tượng ra khỏi logic sử dụng.

- **utils:** Gói chứa các công cụ hỗ trợ xuất file độc lập, giúp hệ thống dễ dàng lưu trữ và chia sẻ kết quả ra môi trường bên ngoài.
    - **CSVExporter:** Lớp chuyên trách việc ghi xuất dữ liệu mạng xã hội (cả bản thô và bản đã làm sạch) thành định dạng tệp CSV. Đảm bảo dữ liệu thu thập được lưu trữ an toàn, phục vụ cho các lần phân tích ngoại tuyến (offline) sau này.

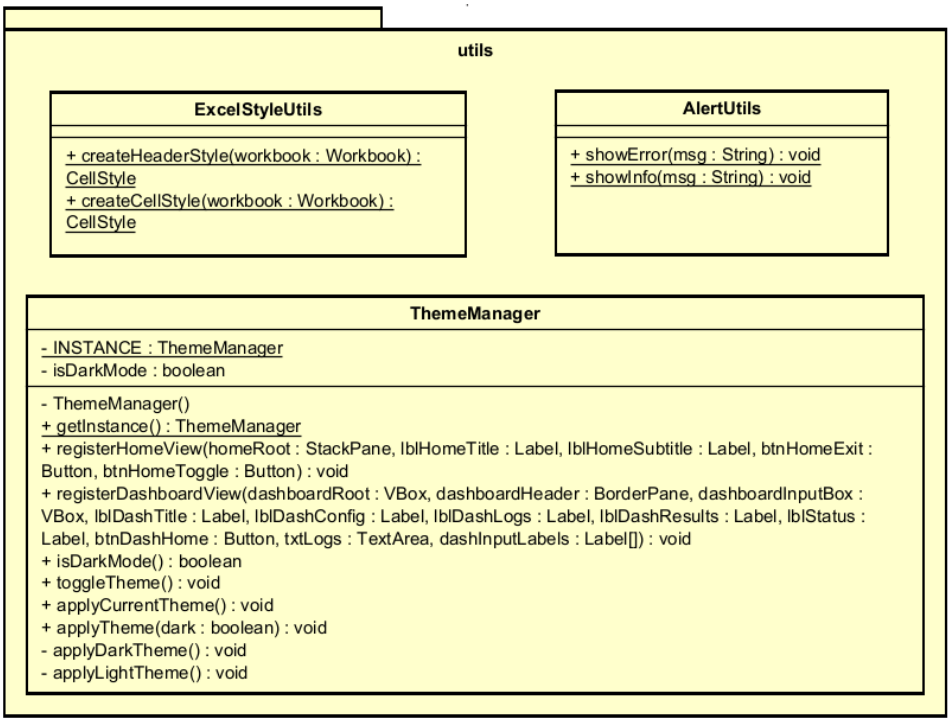
    - **ExcelExporter:** Lớp tiện ích báo cáo cấp cao. Không chỉ dừng lại ở việc xuất chữ, lớp này còn tích hợp các logic định dạng cell, tổng hợp số liệu và phân loại để tạo ra một bản báo cáo thống kê định dạng Excel hoàn chỉnh, trực quan và chuyên nghiệp.

3.3.1.8. ui

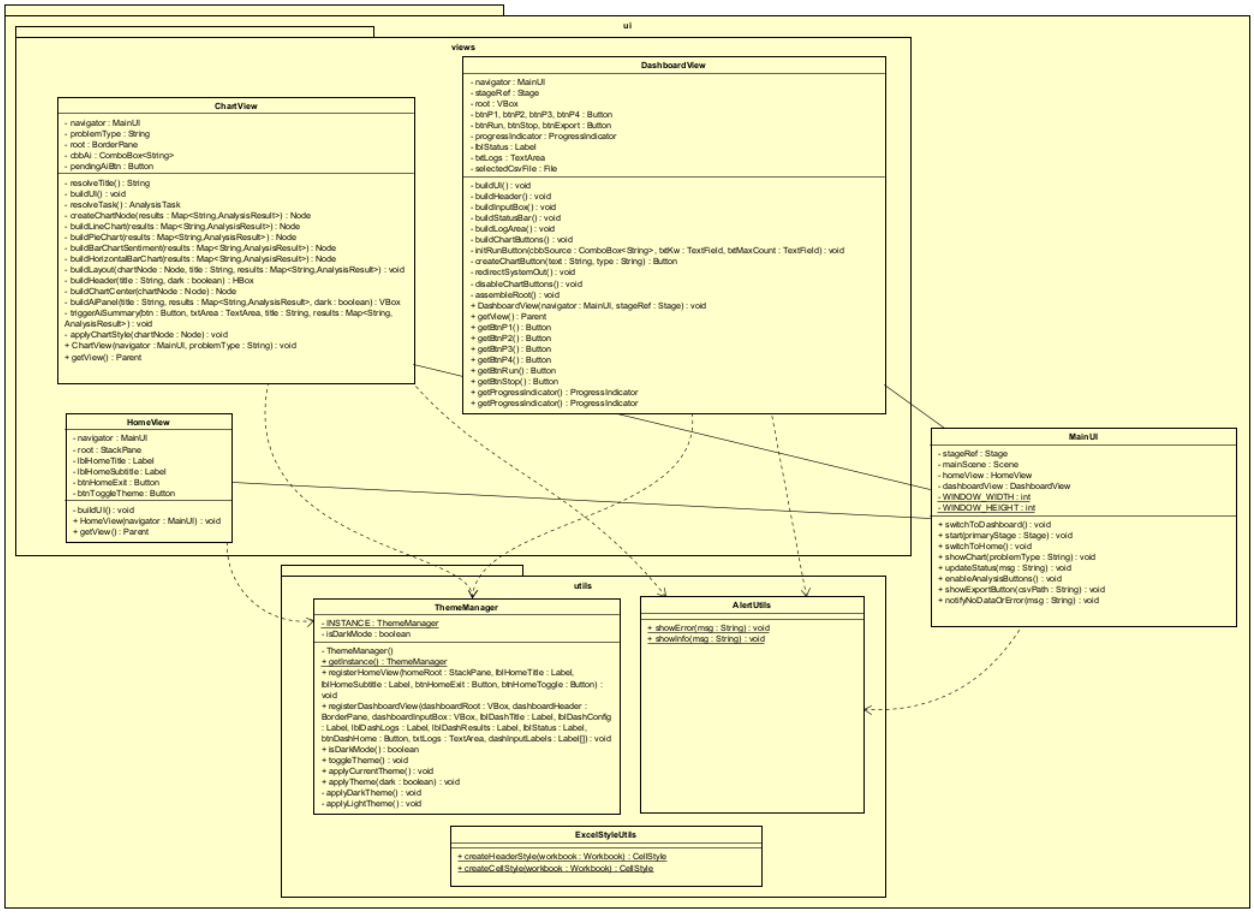
- **ui.views**



- **ui.utils**



- **ui**



- **ui:** Tầng giao diện người dùng (Presentation Layer), đảm nhiệm vai trò tương tác trực tiếp, tiếp nhận luồng sự kiện và hiển thị kết quả trực quan cho người dùng.
    - **MainUI:** Đóng vai trò là "nhạc trưởng" (Bộ điều hướng - Router/Controller) của toàn bộ hệ thống giao diện. Lớp này nắm giữ cửa sổ làm việc chính (Stage) và kiểm soát logic chuyển đổi qua lại giữa các màn hình (như Home, Dashboard, Chart).
    - **views:** Gói chứa các lớp định nghĩa cấu trúc và bố cục (layout) của từng màn hình cụ thể.

- **HomeView:** Màn hình khởi động (Landing page), cung cấp giao diện chào mừng, chứa các thao tác điều hướng cơ bản và nút chuyển đổi giao diện Sáng/Tối.

- **DashboardView:** Màn hình không gian làm việc trung tâm. Đây là nơi tổ chức giao diện phức tạp nhất bao gồm: khu vực nhập liệu, thanh tiến trình (ProgressIndicator), bảng log theo dõi hệ thống và cụm nút điều khiển các tiến trình phân tích (P1, P2, P3, P4).

- **ChartView:** Màn hình chuyên trách trực quan hóa dữ liệu. Lớp này có khả năng tự động "lắp ráp" các loại biểu đồ khác nhau (PieChart, BarChart, LineChart) tùy theo nghiệp vụ, đồng thời tích hợp khung hiển thị kết quả báo cáo tóm tắt từ AI.

- **utils:** Gói chứa các công cụ tiện ích hỗ trợ nhằm giảm tải logic cho các lớp View.

  - **ThemeManager:** Triển khai theo mẫu thiết kế Singleton, đảm nhiệm việc quản lý trạng thái và áp dụng mượt mà các tệp CSS để thay đổi giao diện Sáng/Tối (Light/Dark mode) cho toàn bộ ứng dụng.

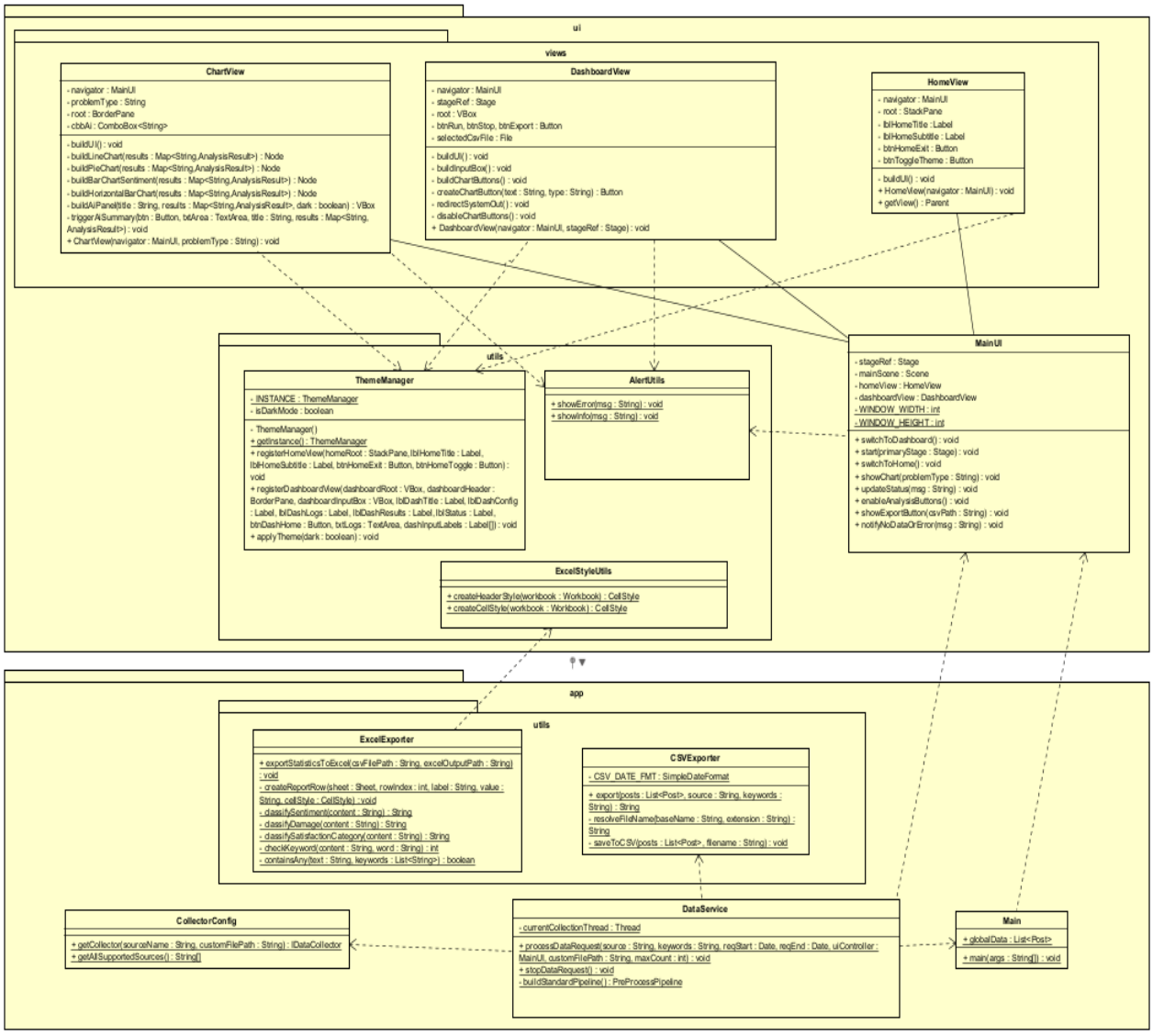
  - **AlertUtils:** Lớp tiện ích đóng gói sẵn các hộp thoại (Dialog/Popup), giúp hệ thống dễ dàng bật các thông báo thành công (Info) hoặc cảnh báo lỗi (Error) đến người dùng một cách nhất quán.

  - **ExcelStyleUtils:** Lớp tiện ích cấu hình, chịu trách nhiệm định dạng (màu sắc, font chữ, viền) cho các ô tính khi ứng dụng xuất kết quả phân tích ra tệp Excel.

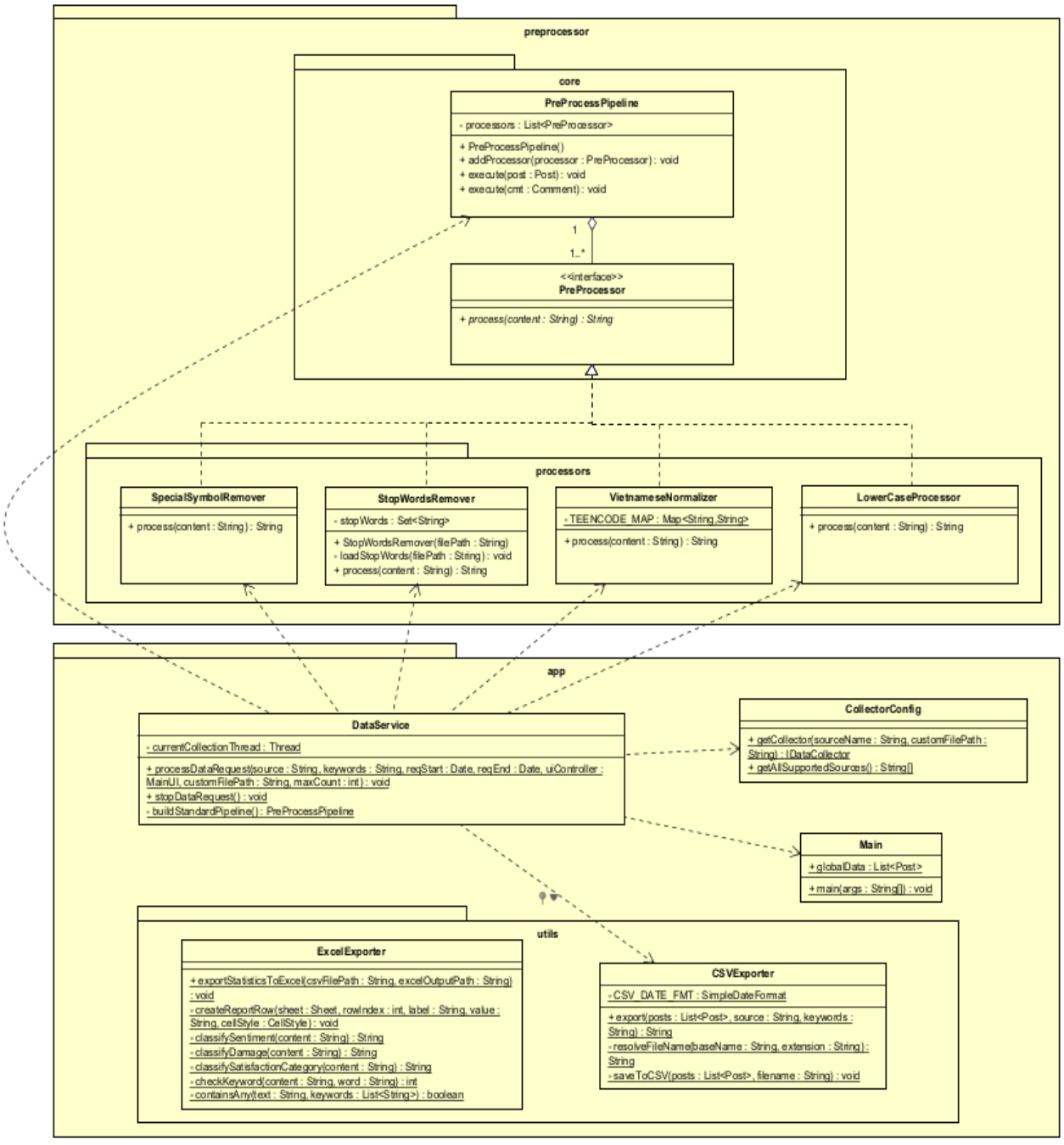
3.3.2. Mối quan hệ tương tác tổng thể giữa các gói với nhau

(Ở đây nhóm xin lược bớt một số các phương thức, thuộc tính đã liệt kê ở trên, không quá đóng góp nhiều ý nghĩa về mối quan hệ tương tác trong một số lớp ở mục 3.3.2 này, mục đích để biểu đồ nhìn đỡ rối. Song bên cạnh đó vẫn còn một số quan hệ hơi khó nhìn, mời thầy có thể zoom chi tiết hơn những quan hệ này trong folder UML nhóm đã up lên drive ạ)

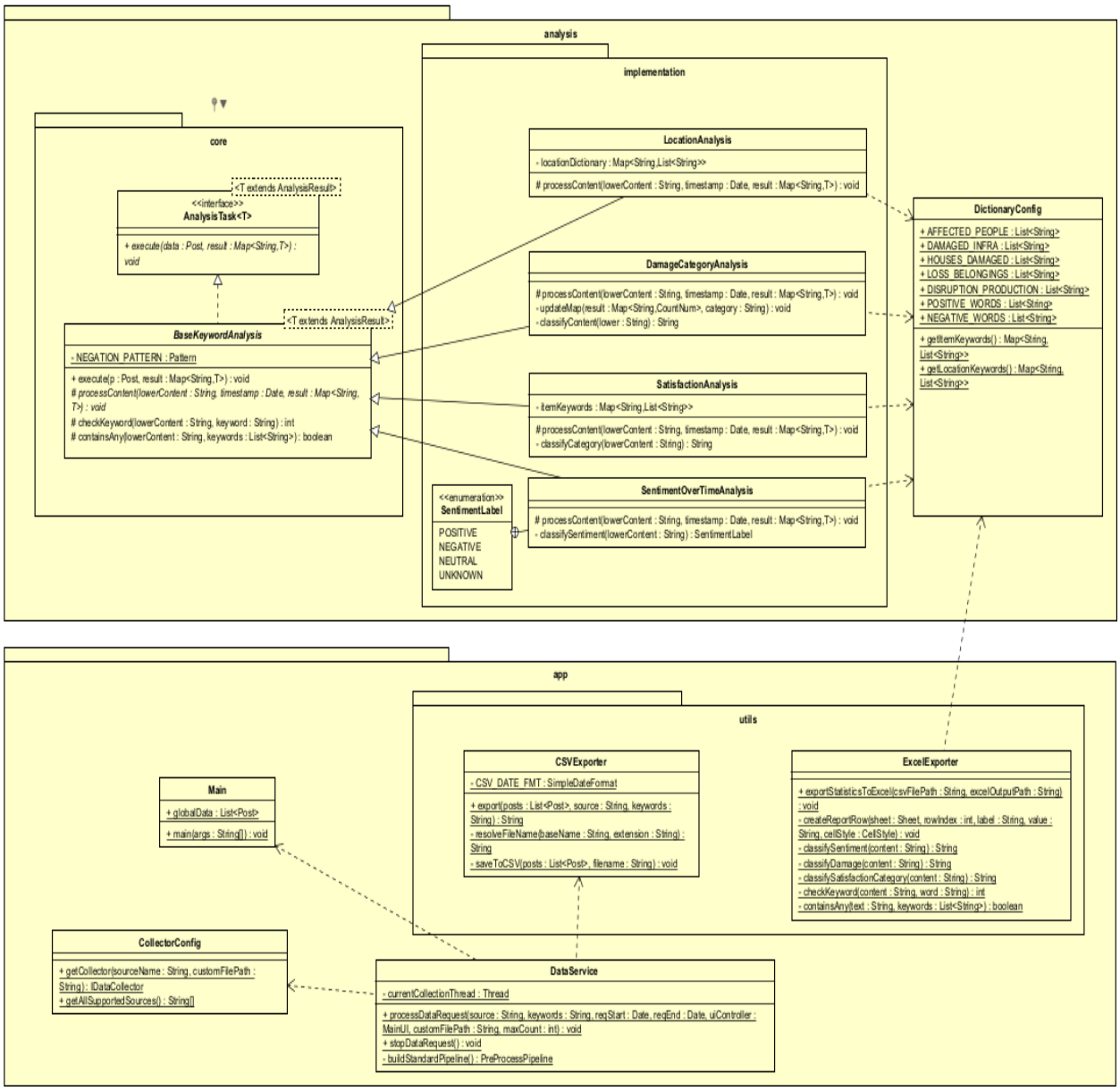
3.3.2.1. app & ui



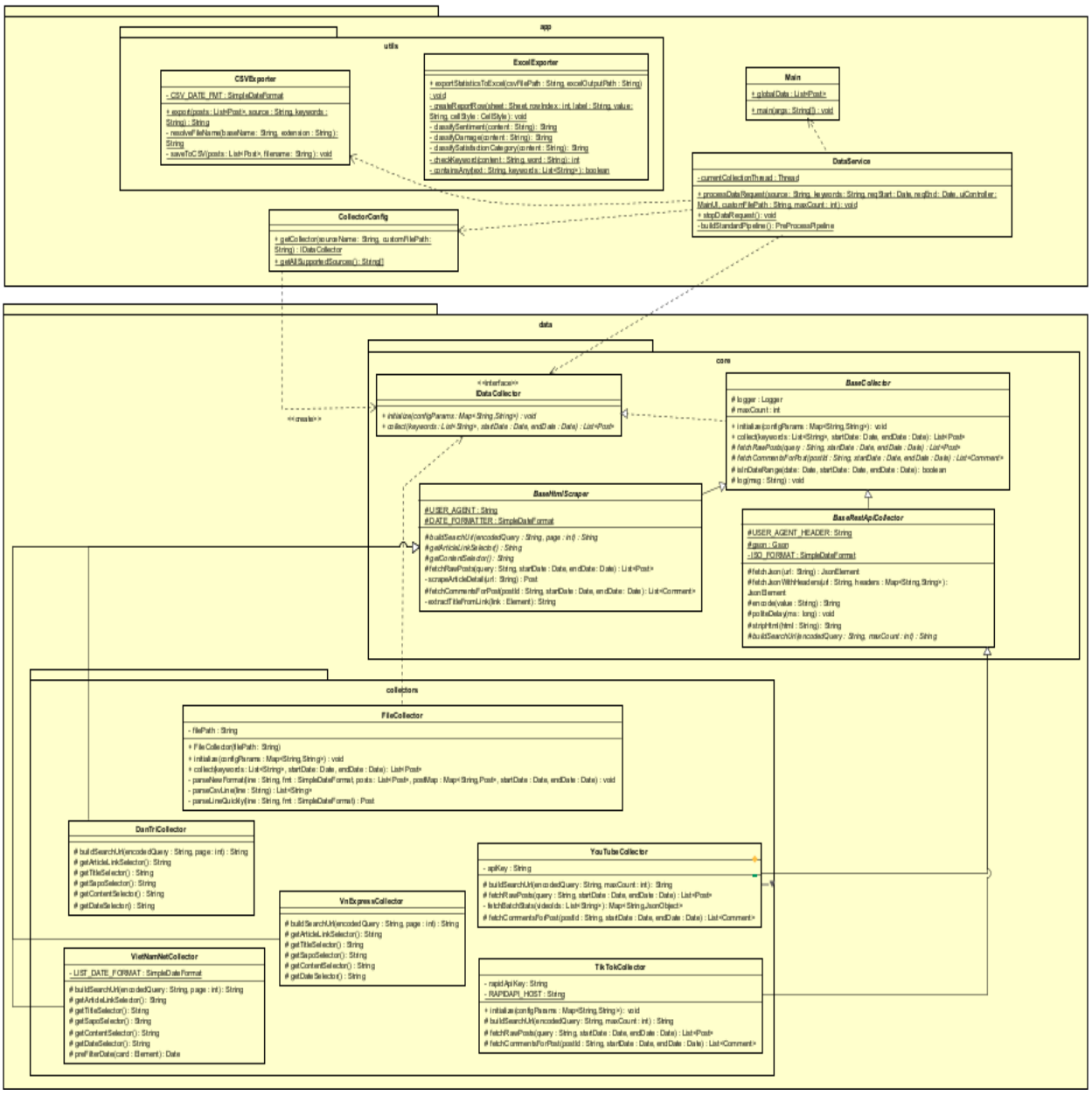
3.3.2.2. app & preprocessor



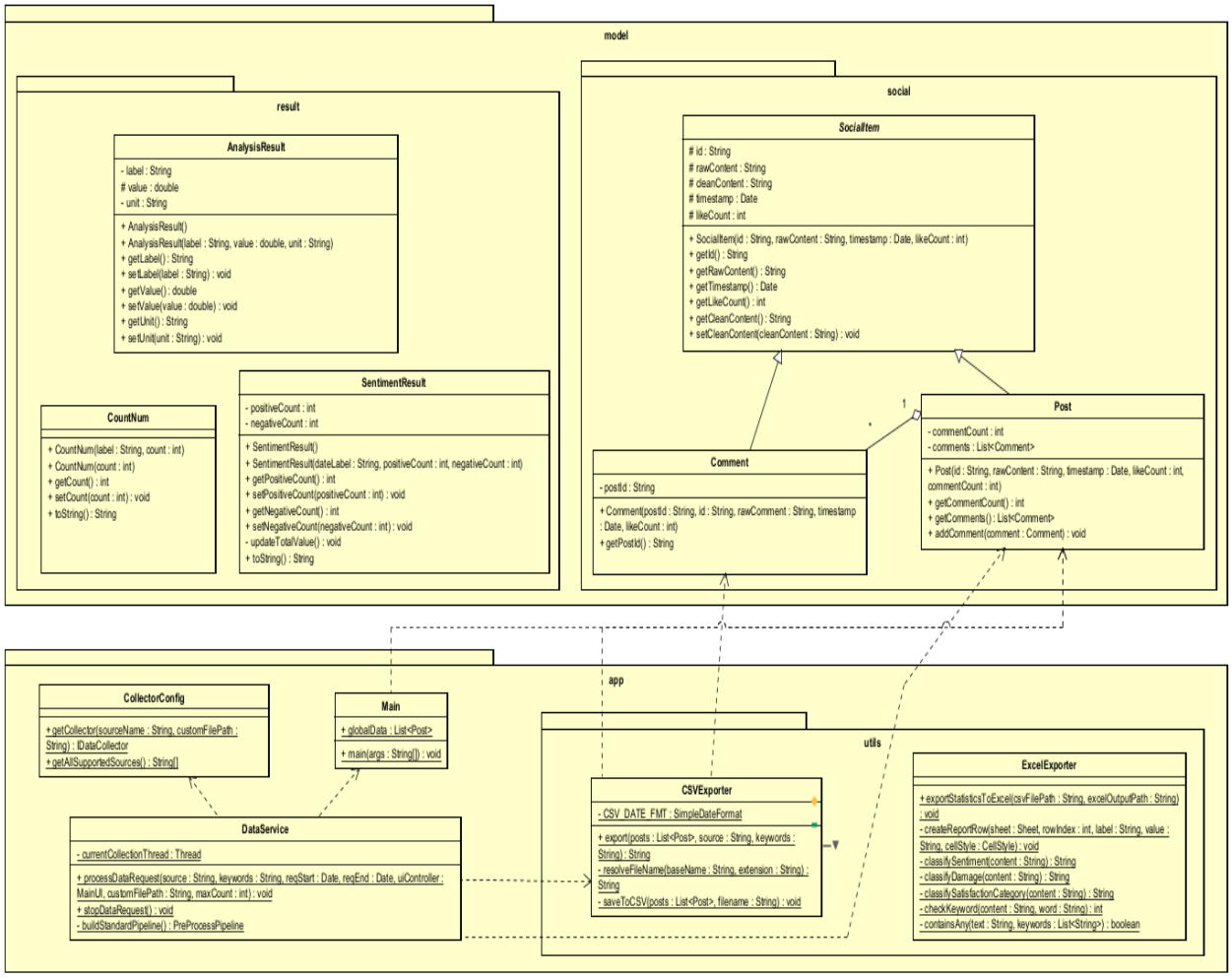
3.3.2.3. app & analysis



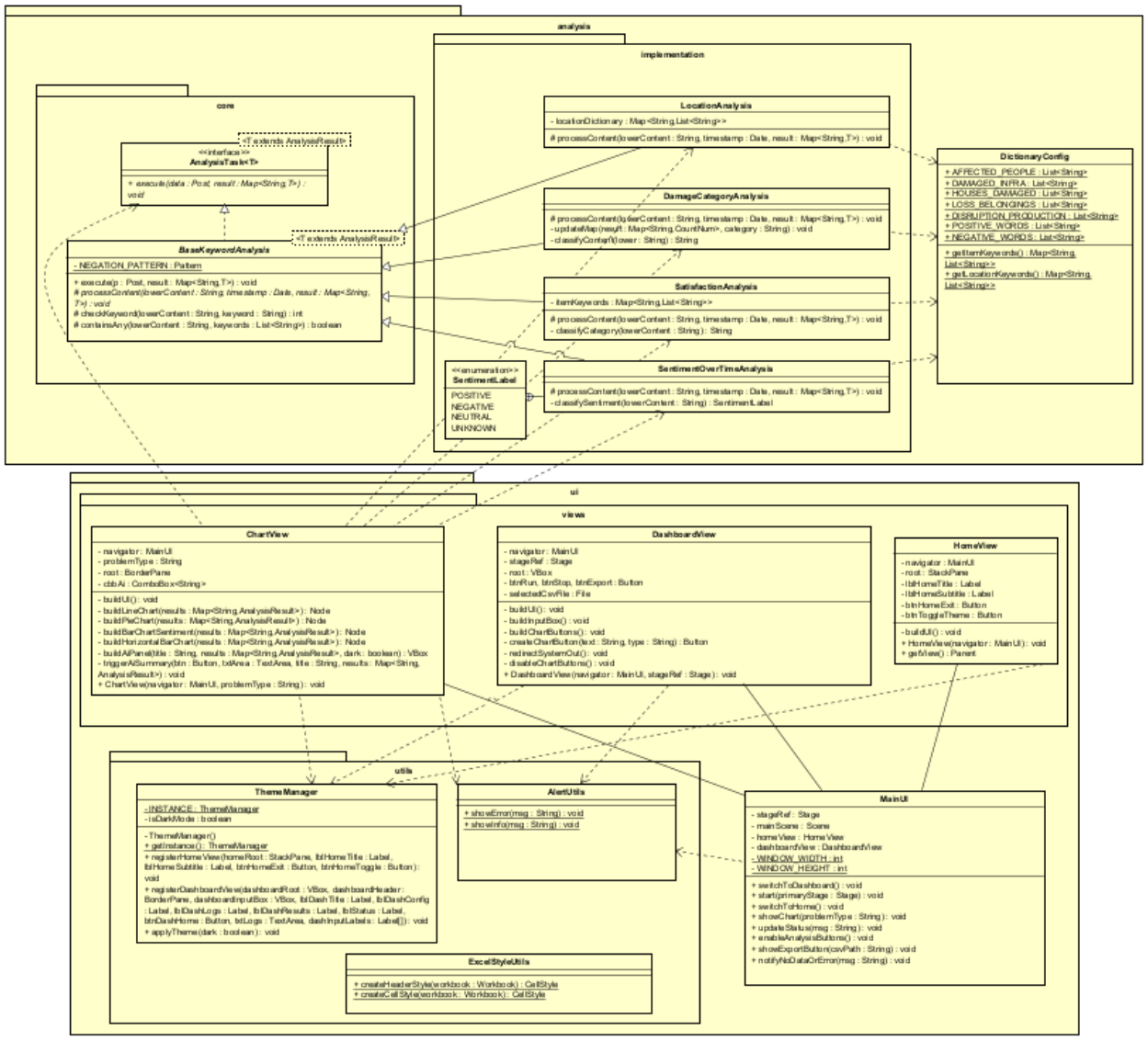
3.3.2.4. app & data



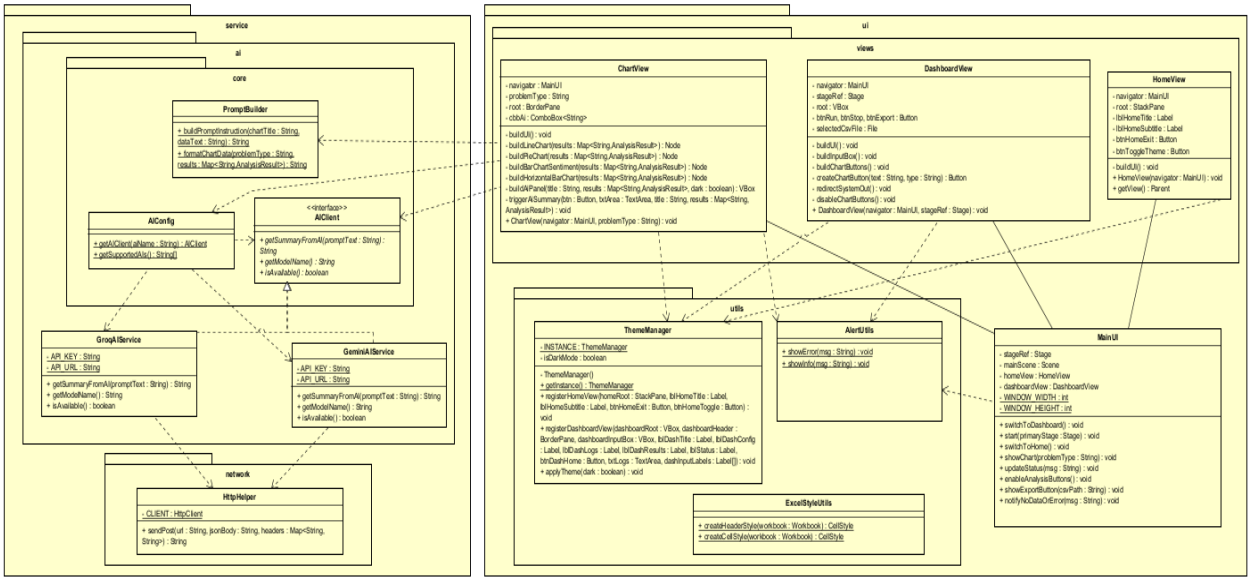
3.2.2.5. app & model



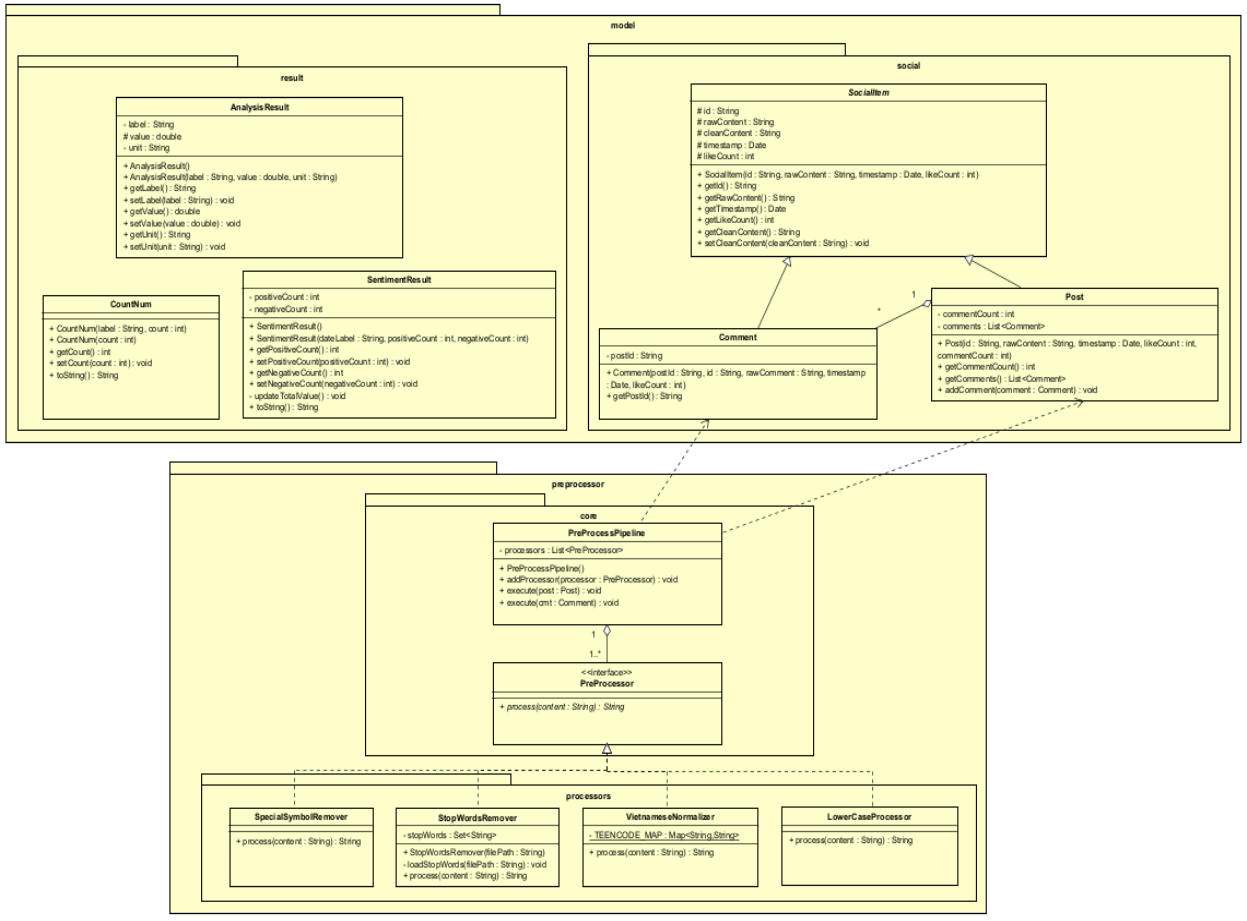
3.2.2.6. ui & analysis



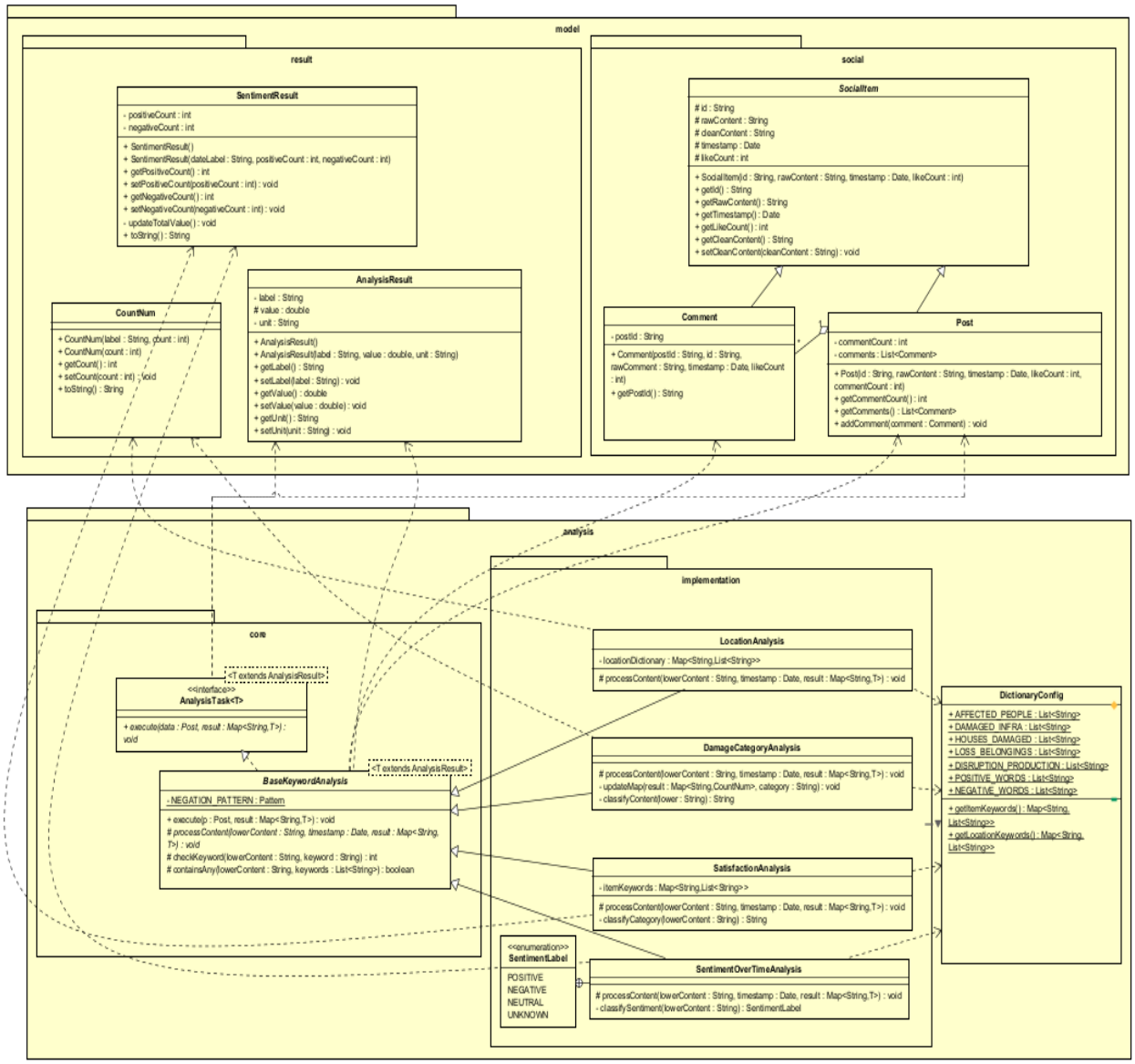
3.2.2.7. ui & service



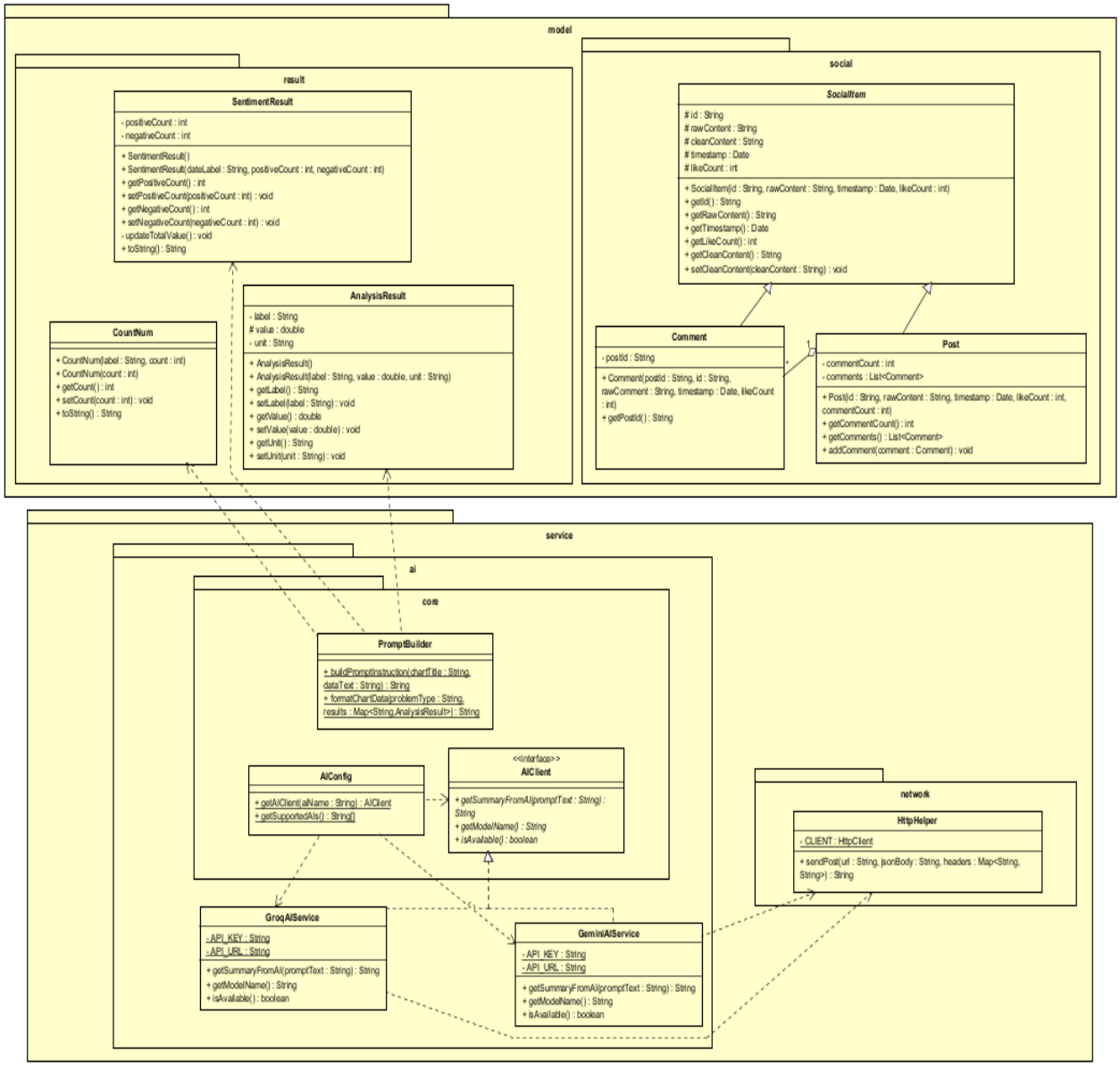
3.2.2.8. preprocessor & model



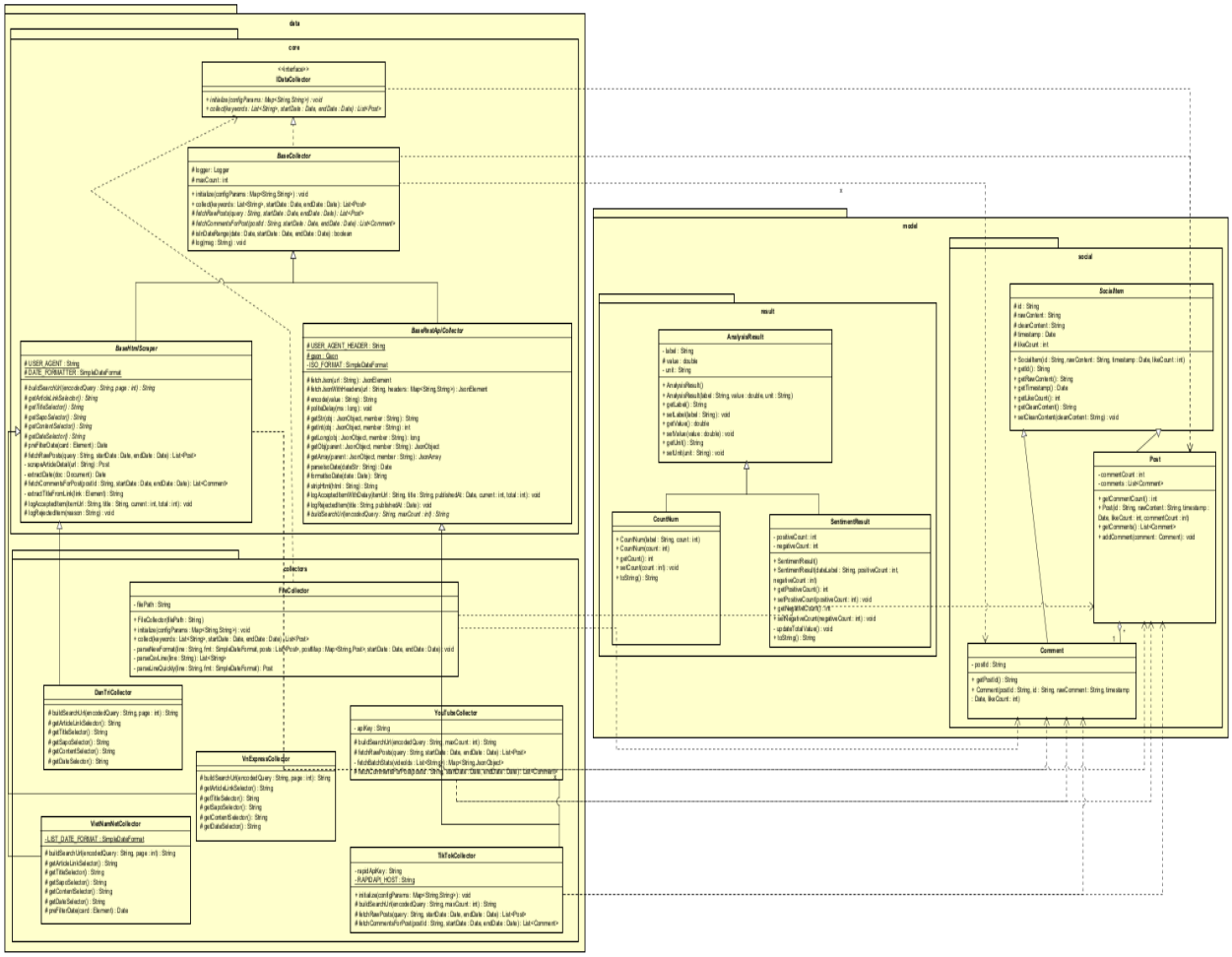
3.2.2.9. analysis & model



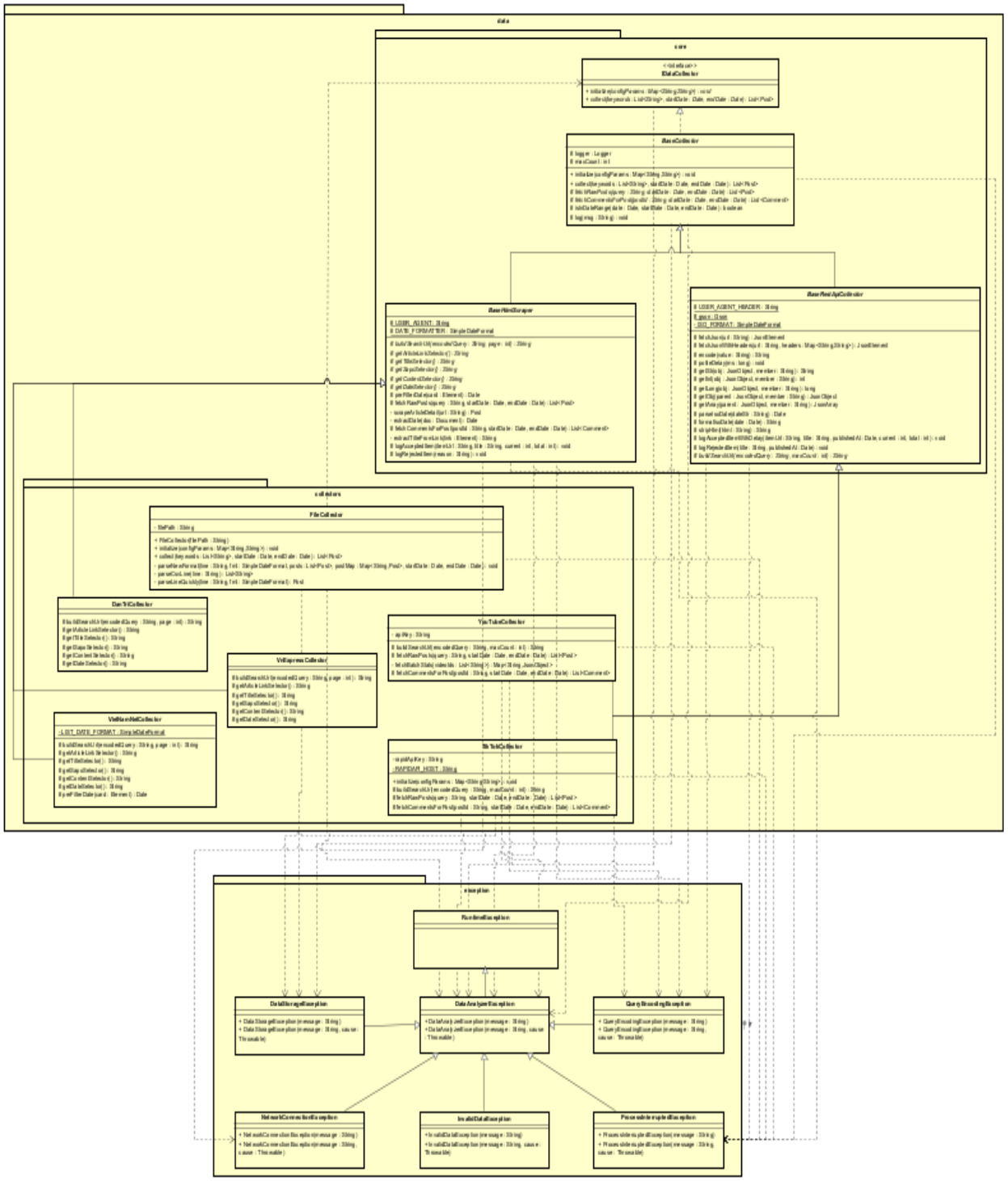
3.2.2.10. service & model



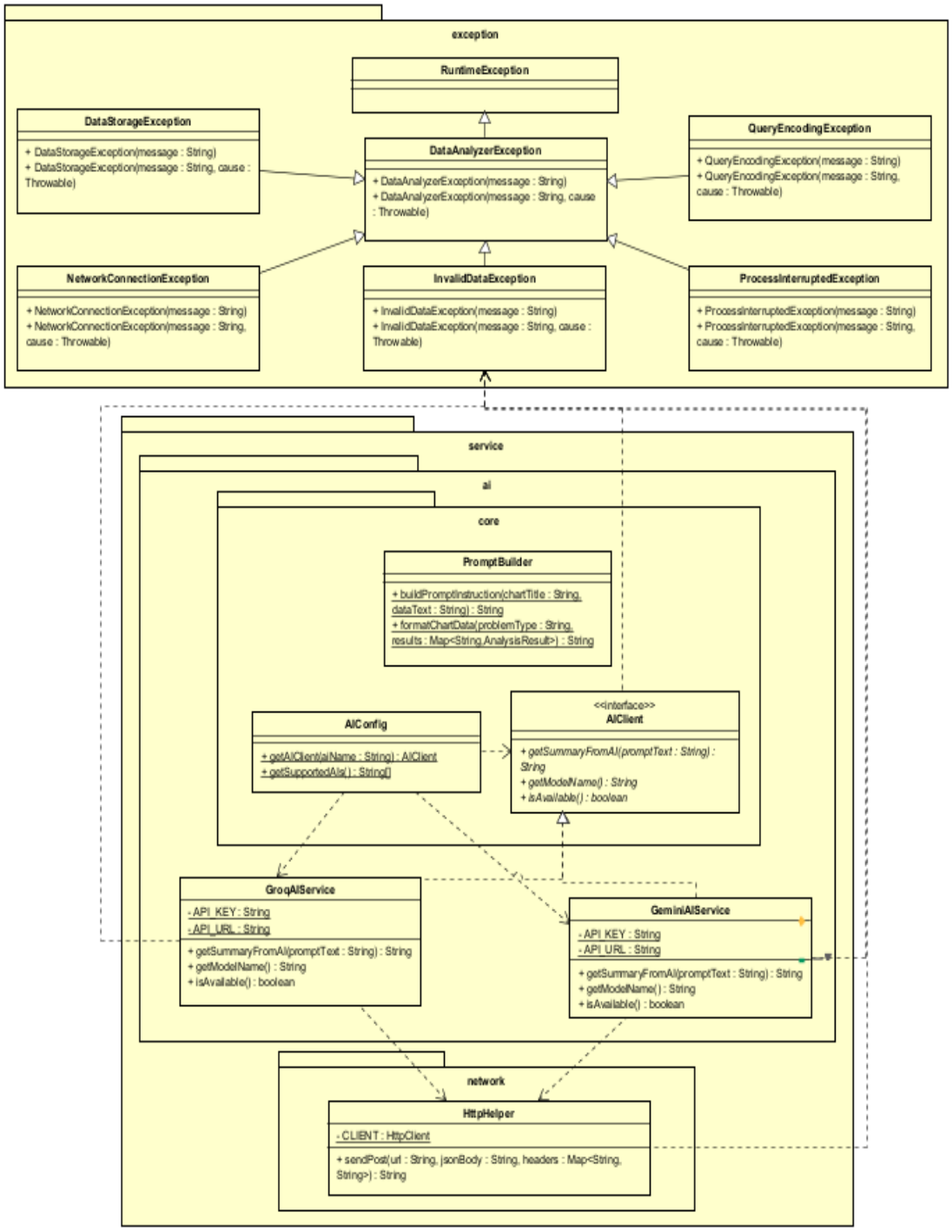
3.2.2.11. data & model



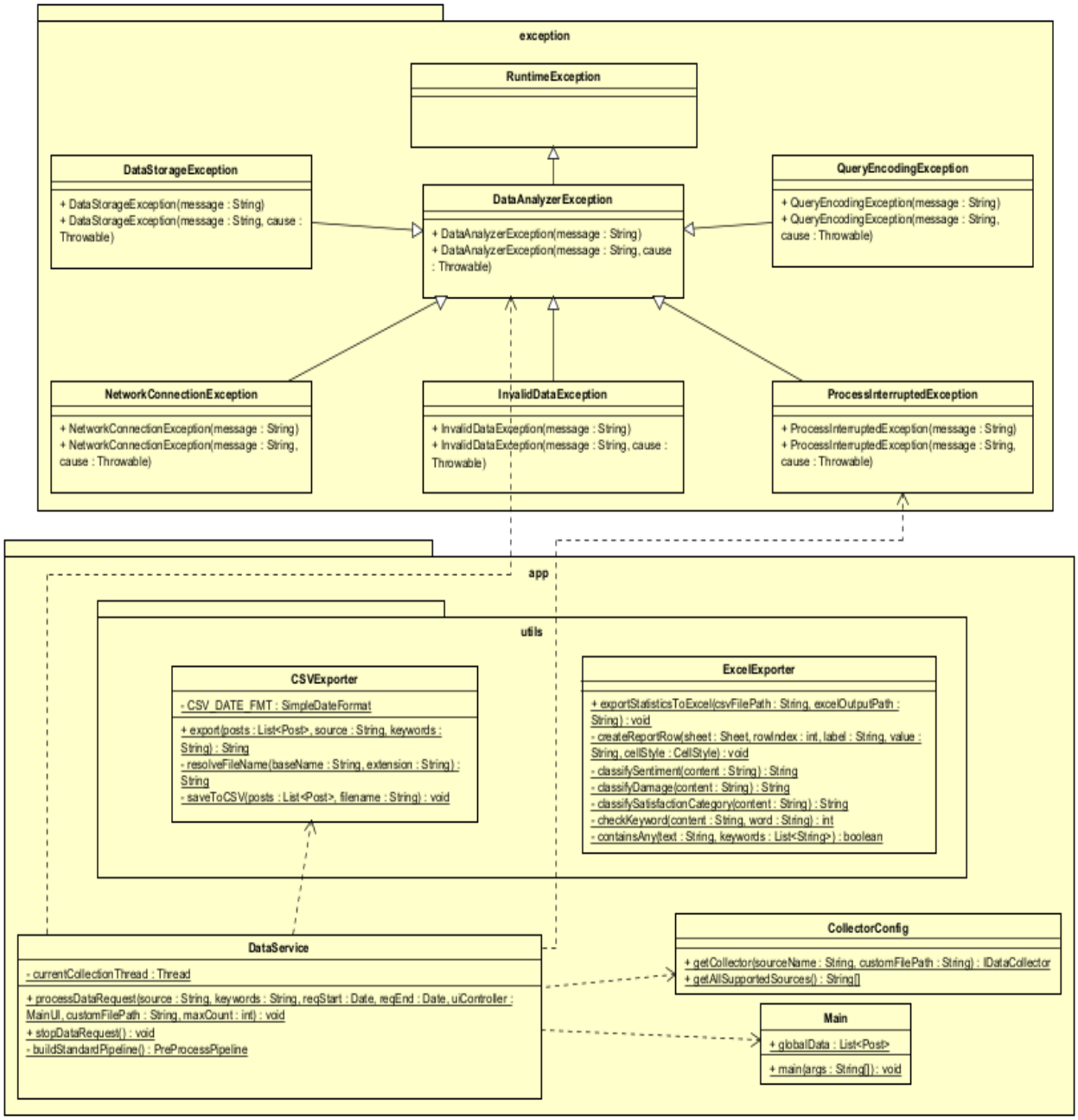
3.2.2.12. data & exception



3.2.2.13. service & exception



3.2.2.14. app & exception



IV. Bảng thống kê về dữ liệu đã thu thập

4.1. Bão yagi – Dân Trí News

THÔNG TIN DỰ ÁN	
Tên thảm họa	yagi
Từ khóa	bão yagi
Khoảng thời gian	2024-09-02 21:29:00 đến 2024-09-27 13:27:00
Nền tảng	DanTri
THỐNG KÊ DỮ LIỆU	
Tổng số bài viết	150
Tổng số bình luận	0
PHÂN BỐ CẢM XÚC	
Tích cực	24 (16,0%)
Trung lập	9 (6,0%)
Tiêu cực	117 (78,0%)
PHÂN LOẠI THIỆT HẠI	
Con người (People)	44
Kinh tế (Economy)	3
Nhà cửa (Housing)	16
Tài sản (Belongings)	3
Hạ tầng (Infrastructure)	42
PHÂN LOẠI HÀI LÒNG	
Liên lạc (Communication)	Tích cực: 147 | Tiêu cực: 345
Chỗ ở (Shelter)	Tích cực: 145 | Tiêu cực: 242
Lương thực (Food)	Tích cực: 36 | Tiêu cực: 54
Cứu hộ (Rescue)	Tích cực: 14 | Tiêu cực: 29
Vận chuyển (Transport)	Tích cực: 5 | Tiêu cực: 17
ĐIỂM NÓNG THIÊN TAI	
Quảng Ninh	84
Hà Nội	65
Hải Phòng	64
Lào Cai	15
Bắc Ninh	13
Yên Bái	12
Thái Nguyên	11
Cao Bằng	8
Phú Thọ	7
Tuyên Quang	6

4.2. Bão bualoi – VNExpress News

THÔNG TIN DỰ ÁN	
Tên thảm họa	bualoi
Từ khóa	bão bualoi
Khoảng thời gian	2025-09-24 11:04:00 đến 2025-10-30 12:01:00
Nền tảng	VnExpress
THỐNG KÊ DỮ LIỆU	
Tổng số bài viết	131
Tổng số bình luận	0
PHÂN BỐ CẢM XÚC	
Tích cực	15 (11,5%)
Trung lập	22 (16,8%)
Tiêu cực	94 (71,8%)
PHÂN LOẠI THIỆT HẠI	
Con người (People)	62
Kinh tế (Economy)	1
Nhà cửa (Housing)	14
Tài sản (Belongings)	0
Hạ tầng (Infrastructure)	33
PHÂN LOẠI HÀI LÒNG	
Liên lạc (Communication)	Tích cực: 348 | Tiêu cực: 639
Chỗ ở (Shelter)	Tích cực: 205 | Tiêu cực: 317
Lương thực (Food)	Tích cực: 15 | Tiêu cực: 23
Vận chuyển (Transport)	Tích cực: 6 | Tiêu cực: 5
ĐIỂM NÓNG THIÊN TAI	
Hà Nội	57
Lào Cai	29
Tuyên Quang	24
Phú Thọ	20
Thái Nguyên	18
Quảng Ninh	17
Hải Phòng	11
Cao Bằng	10
Yên Bái	9
Bắc Ninh	9

4.3. Bão yagi – VietNamNet News

THÔNG TIN DỰ ÁN	
Tên thảm họa	yagi
Từ khóa	bão yagi
Khoảng thời gian	2024-09-03 09:48:00 đến 2024-09-28 13:28:00
Nền tảng	VietNamNet
THỐNG KÊ DỮ LIỆU	
Tổng số bài viết	150
Tổng số bình luận	0
PHÂN BỐ CẢM XÚC	
Tích cực	24 (16,0%)
Trung lập	23 (15,3%)
Tiêu cực	103 (68,7%)
PHÂN LOẠI THIỆT HẠI	
Con người (People)	44
Kinh tế (Economy)	4
Nhà cửa (Housing)	7
Tài sản (Belongings)	1
Hạ tầng (Infrastructure)	31
PHÂN LOẠI HÀI LÒNG	
Liên lạc (Communication)	Tích cực: 150 | Tiêu cực: 304
Chỗ ở (Shelter)	Tích cực: 142 | Tiêu cực: 248
Lương thực (Food)	Tích cực: 51 | Tiêu cực: 70
Cứu hộ (Rescue)	Tích cực: 8 | Tiêu cực: 14
Vận chuyển (Transport)	Tích cực: 16 | Tiêu cực: 18
Tiền mặt (Cash)	Tích cực: 4 | Tiêu cực: 1
ĐIỂM NÓNG THIÊN TAI	
Hà Nội	65
Quảng Ninh	58
Hải Phòng	43
Yên Bái	14
Lào Cai	14
Thái Nguyên	13
Phú Thọ	11
Tuyên Quang	11
Cao Bằng	9
Bắc Ninh	7

4.4. Bão yagi – Youtube

THÔNG TIN DỰ ÁN	
Tên thảm họa	yagi
Từ khóa	bão yagi
Khoảng thời gian	2024-09-03 09:48:11 đến 2024-09-29 21:35:20
Nền tảng	YouTube
THỐNG KÊ DỮ LIỆU	
Tổng số bài viết	200
Tổng số bình luận	4162
PHÂN BỐ CẢM XÚC	
Tích cực	413 (9,5%)
Trung lập	2767 (63,4%)
Tiêu cực	1182 (27,1%)
PHÂN LOẠI THIỆT HẠI	
Con người (People)	25
Kinh tế (Economy)	8
Nhà cửa (Housing)	43
Tài sản (Belongings)	5
Hạ tầng (Infrastructure)	80
PHÂN LOẠI HÀI LÒNG	
Liên lạc (Communication)	Tích cực: 24 | Tiêu cực: 116
Chỗ ở (Shelter)	Tích cực: 73 | Tiêu cực: 216
Lương thực (Food)	Tích cực: 29 | Tiêu cực: 57
Cứu hộ (Rescue)	Tích cực: 22 | Tiêu cực: 28
Y tế và Vệ sinh (Medical & Hygiene)	Tích cực: 3 | Tiêu cực: 1
Vận chuyển (Transport)	Tích cực: 146 | Tiêu cực: 215
Tiền mặt (Cash)	Tích cực: 5 | Tiêu cực: 5
ĐIỂM NÓNG THIÊN TAI	
Hà Nội	51
Hải Phòng	47
Quảng Ninh	37
Bắc Ninh	8
Phú Thọ	6
Lào Cai	5
Thái Nguyên	4
Yên Bái	3
Tuyên Quang	1

4.5. Bão matmo – TikTok

THÔNG TIN DỰ ÁN	
Tên thảm họa	matmo
Từ khóa	bão matmo
Khoảng thời gian	2025-10-02 05:10:30 đến 2025-10-29 11:02:47
Nền tảng	TikTok
THỐNG KÊ DỮ LIỆU	
Tổng số bài viết	100
Tổng số bình luận	1075
PHÂN BỐ CẢM XÚC	
Tích cực	90 (7,7%)
Trung lập	799 (68,0%)
Tiêu cực	286 (24,3%)
PHÂN LOẠI THIỆT HẠI	
Con người (People)	1
Kinh tế (Economy)	1
Nhà cửa (Housing)	3
Tài sản (Belongings)	2
Hạ tầng (Infrastructure)	3
PHÂN LOẠI HÀI LÒNG	
Liên lạc (Communication)	Tích cực: 1 | Tiêu cực: 16
Chỗ ở (Shelter)	Tích cực: 6 | Tiêu cực: 29
Lương thực (Food)	Tích cực: 6 | Tiêu cực: 23
Cứu hộ (Rescue)	Tích cực: 9 | Tiêu cực: 3
Y tế và Vệ sinh (Medical & Hygiene)	Tích cực: 0 | Tiêu cực: 1
Vận chuyển (Transport)	Tích cực: 11 | Tiêu cực: 28
Tiền mặt (Cash)	Tích cực: 4 | Tiêu cực: 1
ĐIỂM NÓNG THIÊN TAI	
Thái Nguyên	15
Quảng Ninh	11
Hà Nội	8
Hải Phòng	5
Yên Bái	3
Phú Thọ	2
Bắc Ninh	1

V. Các công nghệ & Kỹ thuật lập trình được áp dụng

5.1. Trong lập trình hướng đối tượng

5.1.1. Tính Đóng Gói (Encapsulation)

Là một nguyên lý tối quan trọng trong OOP, tính Đóng gói (**Encapsulation**) giúp cô lập dữ liệu và che giấu các chi tiết thực thi phức tạp bên trong đối tượng. Bằng cách chỉ bộc lộ những phương thức tương tác cần thiết ra bên ngoài, Encapsulation không chỉ đảm bảo an toàn hệ thống mà còn giúp giảm thiểu sự ràng buộc giữa các thành phần độc lập.

Xuyên suốt quá trình phát triển dự án, tính Đóng gói đã được nhóm vận dụng một cách triệt để và đa tầng. Thay vì chỉ giới hạn ở mức biến và hàm, nhóm đã thiết lập các ranh giới đóng gói chặt chẽ ở quy mô lớn hơn, bao quát toàn diện từ cấp độ lớp, module cho đến các package.

5.1.1.1. Tính đóng gói trong preprocessor

Trong kiến trúc của hệ thống tiền xử lý (Preprocessor), các thuộc tính nền tảng như danh sách bộ lọc (processors trong PreProcessPipeline), bộ từ dừng (stopWords kiểu HashSet) hay từ điển chuẩn hóa lóng (teencode_map) đều được đóng gói nghiêm ngặt bằng phạm vi truy cập private

```
private final List<PreProcessor> processors;

private Set<String> stopWords;

private static final Map<String, String> TEENCODE_MAP = new HashMap<>();
```

Vì các tập dữ liệu như stopWords hay TEENCODE là "trái tim" của thuật toán làm sạch văn bản, trong khi processors quyết định quy trình vận hành của đường ống xử lý. Nếu để public, bất kỳ lớp nào bên ngoài cũng có quyền can thiệp trực tiếp (ví dụ: vô tình gọi lệnh stopWords.clear() làm xóa sạch từ điển, hoặc tự ý chèn thêm một bộ lọc rác vào processors). Việc đóng gói private giúp tạo ra một bức tường lửa, ngăn chặn tuyệt đối các do việc thao tác dữ liệu bừa bãi từ bên ngoài gây ra.

Bên cạnh đó còn giúp che giấu các chi tiết cài đặt bên trong và kiểm soát lỗi của hệ thống dễ dàng hơn (ví dụ: kiểm tra if (processor == null) thì từ chối thêm vào danh sách, tránh lỗi văng app NullPointerException sau này)

Không chỉ đóng gói các thuộc tính, những phương thức cần thiết cũng cần được bảo vệ, ví dụ quá trình thêm StopWords và lưu vào bộ nhớ RAM cũng cần những điều kiện nhất định của coder khi truyền từ file bên ngoài vào

```
private void loadStopWords(String filePath){}
```

5.1.1.2. Tính đóng gói trong service

Trong các lớp dịch vụ AI (như GeminiAIService hay GroqAIService), các thuộc tính lõi bao gồm API_KEY, API_URL và CLIENT được thiết lập ở mức độ truy cập khắt khe nhất (private static final)

```
private static final String API_KEY;

private static final String API_URL;

private static final HttpClient CLIENT;
```

Bởi đây đều là "chìa khóa" định danh để hệ thống giao tiếp với máy chủ AI (Google/Groq) và thường gắn liền với hạn mức sử dụng (quota) hoặc thẻ tín dụng. Việc đóng gói private biến chìa khóa này thành "tài sản tuyệt mật" của riêng lớp Service đó, độ bảo mật được đặt lên mức tối cao.

5.1.1.3. Tính đóng gói trong model

- **Áp dụng rộng rãi:** Được thể hiện rõ ràng trong tất cả các lớp thực thể biểu diễn dữ liệu mạng xã hội và kết quả phân tích (như SocialItem, Post, Comment, AnalysisResult, CountNum, SentimentResult).
- **Kiểm soát quyền truy cập thuộc tính:** Toàn bộ các thuộc tính nghiệp vụ nhạy cảm hoặc quan trọng (ví dụ: positiveCount, negativeCount trong SentimentResult hay comments trong Post) đều được khai báo phạm vi truy cập là private. Các đối tượng bên ngoài chỉ có thể đọc hoặc thay đổi trạng thái của các thuộc tính này thông qua các phương thức getter và setter được

định nghĩa rõ ràng. Riêng các thuộc tính có tính kế thừa cao tại lớp cha như id, rawContent (ở SocialItem) hay value (ở AnalysisResult) được thiết lập ở mức protected nhằm cho phép các lớp con trực tiếp sử dụng nhưng vẫn ngăn chặn sự truy cập tự do từ các lớp bên ngoài hệ thống.

- **Bảo vệ tính toàn vẹn của dữ liệu:**
    - Trong lớp Post, thuộc tính danh sách comments được khai báo là private final. Điều này đảm bảo tham chiếu đến danh sách không bị ghi đè hoặc thay đổi ngoài ý muốn từ bên ngoài. Việc thêm bình luận mới vào bài viết bắt buộc phải thông qua phương thức addComment(Comment), giúp kiểm soát chặt chẽ mối liên kết dữ liệu giữa các thực thể.
    - Trong lớp SentimentResult, phương thức trợ giúp updateTotalValue() được đặt phạm vi truy cập là private. Thiết kế này đảm bảo mỗi khi các chỉ số tích cực hoặc tiêu cực thay đổi qua các phương thức setter, hệ thống sẽ tự động tính toán lại và đồng bộ thuộc tính value một cách khép kín, ngăn chặn việc can thiệp thủ công từ bên ngoài làm sai lệch kết quả thống kê.

5.1.1.4. Tính đóng gói trong data

- **Áp dụng trong kiến trúc thu thập:** Thể hiện chủ yếu qua các lớp lõi trừu tượng đóng vai trò định hình luồng xử lý (như BaseCollector, BaseHtmlScraper và BaseRestApiCollector).
- **Che giấu chi tiết xử lý kỹ thuật phức tạp:** Các thuộc tính điều khiển hoạt động thu thập (như maxCount, logger trong BaseCollector) được định cấu hình truy cập ở mức hạn chế (protected hoặc private). Các công cụ bóc tách định dạng JSON nội bộ của BaseRestApiCollector (như các phương thức tiện ích getStr, getInt, getObj) cũng được giới hạn phạm vi truy cập, chỉ phục vụ cho mục đích xử lý dữ liệu nội bộ trong phân hệ thu thập tin.
- **Đóng gói logic vận hành của bộ cào dữ liệu:**
    - Trong lớp BaseHtmlScraper, các tiến trình xử lý chi tiết như tải trang HTML, phân tích cú pháp chi tiết bài viết (scrapeArticleDetail), trích xuất ngày tháng bằng biểu thức chính quy (extractDate) hay lọc tiêu đề dự phòng (extractTitleFromLink) đều được thiết lập là các phương thức private.
    - Các lớp con kế thừa (như VnExpressCollector, DanTriCollector) không cần biết và không thể can thiệp vào cách thức Jsoup kết nối hay

phân tích cú pháp HTML. Chúng chỉ cần cung cấp thông tin cấu hình tối thiểu là các CSS Selector (thông qua các phương thức trừu tượng được mở ra). Toàn bộ logic nghiệp vụ phức tạp của một Web Scraper đã được bao bọc an toàn và khép kín bên trong lớp cha.

5.1.1.5. Tính đóng gói trong analysis

```
private static final Pattern NEGATION_PATTERN =
        Pattern.compile("(?i)(không|chưa|chẳng|đừng)\\s*$");
```

Biến NEGATION_PATTERN được khai báo là private static final. Điều này đảm bảo rằng biểu thức chính quy (Regex) cốt lõi của hệ thống không thể bị bất kỳ lớp nào khác vô tình sửa đổi.

```
private final Map<String, List<String>> locationDictionary =
        DictionaryConfig.getLocationKeywords();
```

Lớp LocationAnalysis giấu kín từ điển địa danh của nó trong biến private final Map<String, List<String>> locationDictionary.

```
protected int checkKeyword(String lowerContent, String keyword)
protected boolean containsAny(String lowerContent, List<String> keywords)
```

Các phương thức hỗ trợ như checkKeyword và containsAny trong lớp BaseKeywordAnalysis được đặt là protected thay vì public. Điều này có nghĩa là chúng chỉ được phục vụ riêng cho các lớp con kế thừa nó, không bộc lộ ra ngoài cho các phần khác của hệ thống sử dụng bừa bãi.

5.1.2. Tính Kế Thừa (Inheritance)

Đóng vai trò là công cụ cốt lõi để giải quyết bài toán trùng lặp mã nguồn, tính Kế thừa (**Inheritance**) cho phép các thực thể con (Subclass) thừa hưởng và mở rộng trực tiếp các thuộc tính, phương thức từ thực thể cha (Superclass). Thay vì phải định nghĩa lại từ đầu, dự án đã tận dụng nguyên lý này để thiết lập mối quan hệ phân cấp rõ ràng.

5.1.2.1. Kế thừa trong exception & các package liên quan

Trong gói exception, tính Kế thừa được ứng dụng triệt để qua việc xây dựng một hệ thống phân cấp lỗi đa tầng. Lớp cha **DataAnalyzerException** đóng vai trò nền tảng, định nghĩa sẵn các đặc tính và cấu trúc chung cho toàn bộ ngoại lệ. Các lớp con mang tính đặc thù (như **DataStorageException, NetworkConnectionException, InvalidDataException**...) được thiết kế kế thừa trực tiếp từ lớp cha này. Cấu trúc này không chỉ giúp loại bỏ việc lặp lại mã nguồn khởi tạo (tuân thủ nguyên lý DRY), mà còn tạo ra một bộ khung vững chắc, dễ dàng mở rộng thêm các loại lỗi mới trong tương lai.

Một quyết định thiết kế kiến trúc quan trọng trong hệ thống là việc lớp gốc **DataAnalyzerException** được định nghĩa kế thừa trực tiếp từ **RuntimeException** của Java, thay vì Exception tiêu chuẩn. Quyết định này biến toàn bộ hệ thống phân cấp lỗi của dự án thành dạng **Unchecked Exception**.

Nhờ chiến lược này, hệ thống đạt được hai lợi ích cốt lõi:

- Thứ nhất, giải phóng mã nguồn khỏi sự "ô nhiễm" bởi các khối try-catch hay từ khóa throws rải rác ở khắp các tầng trung gian (như Data hay Service), giúp đảm bảo tiêu chí Clean Code.
- Thứ hai, nó cho phép cơ chế 'Bong bóng lỗi' (Error Bubbling) hoạt động trơn tru, đẩy toàn bộ các ngoại lệ từ tầng đáy hệ thống lên thẳng tầng Giao diện (UI) để xử lý tập trung và đưa ra cảnh báo trực quan cho người dùng.

5.1.2.2. Kế thừa trong ui

```
public class MainUI extends Application{}
```

- Tại tầng giao diện, tính Kế thừa được thể hiện rõ nét thông qua việc lớp trung tâm **MainUI** kế thừa trực tiếp từ lớp nền tảng **Application** của framework JavaFX.

- Việc kế thừa này không chỉ đáp ứng yêu cầu cấu trúc của thư viện, mà còn giúp ứng dụng tự động thừa hưởng toàn bộ cơ chế quản lý vòng đời phức tạp (Lifecycle) từ hệ thống – bao gồm các pha khởi tạo, hiển thị và đóng ứng dụng.

- Nhờ nền tảng có sẵn này, lớp MainUI chỉ cần tập trung ghi đè (override) phương thức start() để thiết lập cửa sổ làm việc chính (Stage) và điều hướng các màn hình, giảm thiểu tối đa việc phải tự xử lý các tương tác đồ họa rườm rà với hệ điều hành.

5.1.2.3. Kế thừa trong model

- **Áp dụng trong cấu trúc thực thể và kết quả phân tích:** Nhằm chia sẻ các thuộc tính dùng chung, tránh việc khai báo lặp lại mã nguồn và thiết lập mối quan hệ phân cấp đối tượng rõ ràng từ mức tổng quát đến mức cụ thể.

- **Phân cấp thực thể mạng xã hội (SocialItem, Post, Comment):** Lớp cha trừu tượng SocialItem định nghĩa sẵn các thuộc tính nền tảng của một thực thể mạng xã hội (id, rawContent, cleanContent, timestamp, likeCount). Lớp Post và Comment kế thừa trực tiếp để tự động sở hữu toàn bộ các thuộc tính này mà không cần khai báo lại. Việc thiết lập giá trị ban đầu cho lớp cha được thực hiện tường minh thông qua cơ chế gọi ngược hàm dựng bằng từ khóa super(...) ở các lớp con.

- **Tái sử dụng cấu trúc kết quả phân tích:**
    - Lớp AnalysisResult đóng vai trò định hình một kết quả phân tích dữ liệu tổng quát gồm nhãn mô tả (label), giá trị (value) và đơn vị đo lường (unit).

    - Lớp CountNum kế thừa để chuyên biệt hóa cho việc hiển thị số lượng đếm, tự động định cấu hình đơn vị mặc định là "Count" và thực hiện ép kiểu giá trị số thực từ lớp cha về số nguyên (int).

    - Lớp SentimentResult kế thừa và mở rộng thêm các thuộc tính riêng (positiveCount, negativeCount), đồng thời tái sử dụng thuộc

tính value của lớp cha để lưu trữ tổng số lượng bài viết tích cực và tiêu cực sau khi tự động đồng bộ.

5.1.2.4. Kế thừa trong data

- **Thiết lập phân cấp đa mức (Multi-level Inheritance):** Giúp phân tách rõ ràng trách nhiệm xử lý công nghệ ở từng cấp của hệ thống thu thập dữ liệu, giữ cho các lớp triển khai thực tế ở mức cuối cùng luôn đạt được sự tinh gọn tối đa.
- **Chuyên biệt hóa công nghệ ở lớp trung gian:** Các lớp trừu tượng trung gian đóng vai trò giải quyết các bài toán kỹ thuật chung cho từng nhóm công nghệ thu thập dữ liệu:
    - Lớp BaseHtmlScraper kế thừa từ BaseCollector để đóng gói toàn bộ hạ tầng xử lý Web Scraping tĩnh, bao gồm việc kết nối và tải tài liệu HTML bằng Jsoup, tự động phân trang bằng chỉ số, bóc tách thời gian bằng Regex và thiết lập độ trễ lịch sự (Polite Delay) để phòng chống bị chặn IP.
    - Lớp BaseRestApiCollector kế thừa từ BaseCollector để đóng gói hạ tầng gọi REST API, bao gồm việc gửi yêu cầu HTTP đi kèm Header xác thực bảo mật, xử lý phân trang động bằng Token/Cursor, bóc tách và chuyển đổi dữ liệu JSON bằng Gson.

- **Tối giản hóa mã nguồn ở các lớp con cuối cùng:** Các bộ cào cụ thể (DanTriCollector, VnExpressCollector, VietNamNetCollector kế thừa từ BaseHtmlScraper; YouTubeCollector, TikTokCollector kế thừa từ BaseRestApiCollector) chỉ cần tập trung vào nghiệp vụ đặc thù của nguồn tin. Chúng được giải phóng hoàn toàn khỏi các đoạn mã kết nối mạng hay phân tích dữ liệu thô phức tạp, chỉ cần cung cấp các thông tin cấu hình tối thiểu (như CSS Selectors đối với báo mạng, hoặc cấu trúc URL API đối với mạng xã hội) thông qua việc triển khai các phương thức trừu tượng được chỉ định từ lớp cha.

5.1.2.5. Kế thừa từ analysis

```
public class DamageCategoryAnalysis extends BaseKeywordAnalysis<CountNum>
public class LocationAnalysis extends BaseKeywordAnalysis<CountNum>
public class SatisfactionAnalysis extends BaseKeywordAnalysis<SentimentResult>
public class SentimentOverTimeAnalysis extends
BaseKeywordAnalysis<SentimentResult>
```

- Các lớp như DamageCategoryAnalysis, LocationAnalysis, SatisfactionAnalysis, và SentimentOverTimeAnalysis đều dùng từ khóa extends BaseKeywordAnalysis.
- Nhờ sự kế thừa này, không có lớp phân tích cụ thể nào phải viết lại đoạn mã dùng Regex để tìm từ khóa hay xử lý từ phủ định. Tất cả đều được kế thừa hai phương thức checkKeyword() và containsAny() từ lớp cha BaseKeywordAnalysis

5.1.3. Tính Đa Hình (Polymorphism)

Tính Đa hình (**Polymorphism**) là cơ chế cốt lõi mang lại sự linh hoạt tối đa cho hệ thống hướng đối tượng, cho phép các thực thể khác nhau có thể phản hồi cùng một thông điệp theo những cách thức riêng biệt. Trong dự án, thế mạnh của nguyên lý này được thể hiện sâu sắc qua việc lập trình hướng giao diện (Programming to Interface)

5.1.3.1. Đa hình trong preprocessor

Tính Đa hình được thể hiện qua giao diện **PreProcessor** với phương thức chuẩn hóa process(). Nhờ cơ chế này, các bộ lọc khác nhau (như LowerCaseProcessor, StopWordsRemover…) có thể tự do triển khai logic làm sạch văn bản theo cách riêng nhờ cơ chế **ghi đè (Override)**. Điều này cho phép hệ thống dễ dàng cắm thêm các bộ lọc mới mà không phải sửa đổi mã nguồn cốt lõi, tuân thủ tuyệt đối nguyên lý Đóng/Mở.

```
public interface PreProcessor {
    String process(String content);
}
```

Bên cạnh đó, trong lớp **PreProcessPipeline**, tính **đa hình lúc biên dịch** (Compile-time Polymorphism) được thể hiện qua việc nạp chồng phương thức **execute()**. Thay vì phải tạo ra hai hàm với tên gọi cồng kềnh như executePost() và executeComment(), hệ thống định nghĩa hai phương thức trùng tên execute nhưng **khác nhau** về **kiểu dữ liệu tham số đầu vào** (Post và Comment). Điều này giúp giao diện sử dụng của lớp Pipeline trở nên nhất quán, trực quan và linh hoạt hơn.

Tùy thuộc vào việc đối tượng truyền vào là bài đăng hay bình luận, trình biên dịch sẽ tự động điều hướng luồng xử lý đến đúng phương thức tương ứng.

```
public void execute(Post post){}

public void execute(Comment cmt){}
```

5.1.3.2. Đa hình trong service

Giao diện **AIClient** đóng vai trò là một "bản hợp đồng" giao tiếp chuẩn mực giữa hệ thống và các nhà cung cấp mô hình ngôn ngữ lớn (LLM). Bằng cách khai báo các phương thức trừu tượng như getSummaryFromAI() hay isAvailable(), hệ thống cho phép tạo ra nhiều lớp thực thi (Implementations) hoàn toàn khác biệt (ví dụ: GeminiClient, GroqClient…)

```
public interface AIClient {}
```

5.1.3.3. Đa hình trong analysis

```
processContent(contentToAnalyze, p.getTimestamp(), result);
```

Hàm processContent() được khai báo ảo ở lớp cha, nhưng khi chạy thực tế, mỗi lớp con lại có một cách "biến hình" khác nhau:

- Lớp LocationAnalysis ghi đè hàm này để đếm tần suất địa danh CountNum.
- Lớp SatisfactionAnalysis ghi đè hàm này để chấm điểm cảm xúc SentimentResult (cộng/trừ điểm tích cực, tiêu cực).

5.1.3.4. Đa hình trong model

- **Áp dụng trong cấu trúc dữ liệu và hiển thị báo cáo:** Cho phép các thực thể cùng chia sẻ các phương thức nghiệp vụ chung nhưng tự quyết định hành vi phản hồi chi tiết tại thời điểm hệ thống thực thi.
- **Đa hình lúc biên dịch thông qua nạp chồng hàm khởi tạo (Constructor Overloading):**
    - Lớp CountNum cung cấp nhiều hàm khởi tạo khác nhau: một phiên bản nhận đầy đủ nhãn và số lượng đếm để thiết lập kết quả tùy chỉnh, một phiên bản chỉ nhận số lượng và tự gán nhãn mặc định là "Total".
    - Lớp SentimentResult hỗ trợ cả hàm khởi tạo không tham số (gọi ngược lên lớp cha thông qua super()) và hàm khởi tạo đầy đủ tham số để phục vụ linh hoạt cho từng kịch bản tạo lập đối tượng trong quá trình phân

tích sắc thái cảm xúc.

- **Đa hình lúc chạy thông qua ghi đè phương thức (Method Overriding):** Các lớp con như CountNum và SentimentResult cùng ghi đè phương thức toString() kế thừa từ lớp tổ tiên Object. Khi hệ thống yêu cầu in thông tin hoặc ghi log, mỗi đối tượng tự động xuất ra chuỗi ký tự được định dạng chuyên biệt theo cấu trúc nghiệp vụ riêng của nó (hiển thị số lượng đơn thuần đối với CountNum, hoặc hiển thị chi tiết tỉ lệ tích cực/tiêu cực theo ngày đối với SentimentResult).

5.1.3.5. Đa hình trong data

- **Đa hình thông qua giao diện chung (Interface Polymorphism):** Toàn bộ các bộ thu thập dữ liệu (như FileCollector, DanTriCollector, YouTubeCollector, TikTokCollector,...) đều triển khai chung giao diện IDataCollector. Thiết kế này cho phép phân hệ quản lý cấp cao kiểm soát đồng nhất mọi bộ cào dưới một kiểu tham chiếu duy nhất là IDataCollector, dễ dàng thực thi tiến trình thu thập thông qua phương thức collect(...) mà không cần phụ thuộc vào chi tiết công nghệ của từng nền tảng cụ thể.

- **Đa hình lúc chạy thông qua cơ chế liên kết động (Dynamic Binding):**
  - Áp dụng mẫu thiết kế Template Method trong lớp trừu tượng BaseCollector. Phương thức collect của lớp cha gọi hai phương thức trừu tượng là fetchRawPosts và fetchCommentsForPost. Khi hệ thống vận hành, JVM sẽ tự động nhận diện đối tượng bộ cào thực tế đang hoạt động để liên kết động và thực thi chính xác các phương thức đã được ghi đè ở lớp con tương ứng.

  - Lớp VietNamNetCollector ghi đè phương thức trợ giúp preFilterDate của lớp cha BaseHtmlScraper. Khi duyệt qua danh sách bài viết, hệ thống sẽ thực thi hành vi lọc ngày đăng chuyên biệt của VietNamNet để loại bỏ sớm các bài viết quá hạn, thay vì trả về giá trị trống như hành vi mặc định của lớp cha.

- **Đa hình lúc biên dịch qua nạp chồng phương thức (Method Overloading):** Trong lớp BaseRestApiCollector, phương thức hỗ trợ phân trang collectPaginated được nạp chồng với hai chữ ký (signature) khác nhau. Một phiên bản sử dụng giới hạn số lượng mặc định từ cấu hình

(this.maxCount), phiên bản còn lại cho phép các lớp con truyền giới hạn số lượng tùy biến trực tiếp tại thời điểm gọi API, giúp tối ưu hóa khả năng tái sử dụng mã nguồn cho các nền tảng mạng xã hội khác nhau.

5.1.4. Tính Trừu Tượng (Abstraction)

Tính Trừu tượng (**Abstraction**) đóng vai trò là lăng kính lọc bỏ những chi tiết thực thi rườm rà, giúp lập trình viên chỉ cần tập trung vào các đặc tính và hành vi cốt lõi của đối tượng. Bằng cách che giấu đi sự phức tạp của logic tầng thấp, nguyên lý này tạo ra một bản thiết kế tổng thể mạch lạc và dễ dàng quản lý.

5.1.4.1 Trừu tượng trong analysis

```
public interface AnalysisTask<T extends AnalysisResult> {
    void execute(Post data, Map<String, T> result);
}
```

Lớp AnalysisTask là một interface định nghĩa một "bản hợp đồng" với phương thức void execute(Post data, Map<String, T> result);. Bất kỳ ai gọi hàm này không cần biết bên trong bài viết được phân tích ra sao, chỉ cần biết đầu vào là Post và đầu ra được lưu vào result.

```
public abstract class BaseKeywordAnalysis<T extends AnalysisResult>
        implements AnalysisTask<T>
```

- Lớp BaseKeywordAnalysis được khai báo là abstract class. Nó định nghĩa sẵn một bộ khung (hàm execute làm nhiệm vụ nối chuỗi văn bản và duyệt qua các bình luận) nhưng cố tình chừa lại một phương thức protected abstract void processContent(...).
- Đây là sự trừu tượng hóa: lớp cha không biết cụ thể phải phân tích từ khóa gì, nên nó ép các lớp con phải tự định nghĩa chi tiết phần này.

5.1.4.2. Trừu tượng trong model

- **Ngăn chặn khởi tạo đối tượng không xác định:** Đảm bảo tính đúng đắn về mặt ngữ nghĩa và logic nghiệp vụ của hệ thống thông qua việc quản lý các thực thể ở dạng khái quát.

- **Định hình thực thể khái quát thông qua lớp trừu tượng (Abstract Class):**

    - Lớp SocialItem được khai báo là một lớp trừu tượng (abstract class). Trong thực tế nghiệp vụ, không có thực thể nào tồn tại độc lập dưới tên gọi chung là "đối tượng mạng xã hội", mà nó bắt buộc phải là một "bài viết" (Post) hoặc một "bình luận" (Comment) cụ thể. Việc đặt lớp SocialItem là trừu tượng giúp ngăn chặn hoàn toàn việc khởi tạo trực tiếp đối tượng này bằng từ khóa new.

    - Lớp trừu tượng SocialItem đóng vai trò làm bộ khung (blueprint) định nghĩa các thuộc tính và hành vi chung nhất của các thực thể mạng xã hội, nhường quyền định danh thực tế và mở rộng cấu trúc dữ liệu cho các lớp con ở tầng dưới.

5.1.4.3. Trừu tượng trong data

- **Phân tách ranh giới hành vi thông qua giao diện (Interface Abstraction):** Giao diện IDataCollector định nghĩa một hợp đồng hành vi (contract) thuần túy cho toàn bộ phân hệ thu thập dữ liệu. Nó chỉ công bố hai phương thức cốt lõi là initialize và collect mà hoàn toàn ẩn đi mọi chi tiết cài đặt kỹ thuật bên trong. Điều này giúp các phân hệ khác tương tác với bộ thu thập dữ liệu một cách độc lập, hoàn toàn không phụ thuộc vào công nghệ cào của từng nguồn tin.
- **Trừu tượng hóa quy trình xử lý công nghệ thông qua các lớp trừu tượng:**

    - Lớp BaseCollector trừu tượng hóa toàn bộ quy trình đồng bộ hóa bài viết và bình luận bằng cách khai báo hai phương thức trừu tượng là fetchRawPosts và fetchCommentsForPost. Quy trình kiểm soát luồng chung được thiết lập sẵn, nhưng phương thức lấy dữ liệu cụ thể từ mạng xã hội hay báo điện tử được trừu tượng hóa để các lớp con tự quyết định.
    - Lớp BaseHtmlScraper trừu tượng hóa cấu trúc DOM của các trang tin điện tử bằng các phương thức abstract trả về CSS Selector như

getTitleSelector(), getContentSelector(), getDateSelector(). Các bộ cào báo điện tử cụ thể không cần quan tâm đến cách Jsoup phân tích cú pháp HTML, chúng chỉ cần cung cấp "tọa độ" CSS phù hợp để hoàn thành tiến trình cào tin.

- Lớp BaseRestApiCollector trừu tượng hóa luồng phân trang và gọi API thông qua phương thức abstract buildSearchUrl, kết hợp việc truyền vào các hàm chức năng (Function, BiFunction) để các lớp con tự định nghĩa luồng ánh xạ dữ liệu JSON tương ứng với cấu trúc phản hồi của từng nền tảng API khác nhau.

5.1.5. Các mẫu thiết kế áp dụng (Design Patterns)

- **Template Method Pattern:** Áp dụng tại **BaseCollector.collect().** Phương thức này được đánh dấu là final để cố định quy trình gom dữ liệu:
  Thu thập bài đăng thô → Duyệt từng bài đăng →Kiểm tra gián đoạn luồng →Nạp bình luận chi tiết → Lọc khoảng thời gian an toàn.
  Các lớp con không được thay đổi quy trình này và bắt buộc phải triển khai chi tiết hai phương thức trừu tượng bước nhỏ là **fetchRawPosts** và **fetchCommentsForPost**.

- **Strategy Pattern:** Áp dụng cho các thuật toán tiền xử lý văn bản thô. Giao diện PreProcessor đóng vai trò là Strategy gốc, các bộ lọc cụ thể là các Concrete Strategy. Người dùng có thể linh hoạt thay đổi, thêm bớt chiến lược làm sạch văn bản mà không làm vỡ kiến trúc lớp.

- **Pipeline Pattern: PreProcessPipeline** quản lý tập hợp tuần tự các **PreProcessor.** Chuỗi ký tự đầu ra của bộ xử lý trước được chuyển làm đầu vào của bộ xử lý sau, tạo nên một dây chuyền làm sạch dữ liệu khép kín.

- **Singleton Pattern:** Áp dụng tại **ThemeManager**. Nhằm đảm bảo chỉ có duy nhất một bộ điều phối giao diện sáng/tối tồn tại xuyên suốt vòng đời ứng dụng, tránh xung đột CSS trên các View khác nhau.

- **Factory Method Pattern:** Thể hiện ở các phương thức khởi tạo tĩnh trong **CollectorConfig.getCollector()** và **AIConfig.getAIClient()**, đóng gói logic kiểm tra điều kiện và khởi tạo đối tượng phức tạp ra khỏi tầng giao diện.

5.2. Các công nghệ, thuật toán hay khác

5.2.1. Máy trạng thái phân tích CSV chuẩn RFC 4180 (Custom CSV State Machine)

Trong FileCollector, để giải quyết triệt để lỗi vỡ cấu trúc cột của phương thức split(",") truyền thống khi gặp nội dung văn bản chứa dấu phẩy nằm trong dấu nháy kép (ví dụ: "Bão Yagi tàn phá rất nặng, mong mọi người an toàn"), hệ thống triển khai thuật toán Máy trạng thái (State Machine) qua việc duyệt tuần tự từng ký tự trên dòng chữ:

- Nếu gặp dấu " đơn: Đảo ngược trạng thái cờ hiệu inQuotes = !inQuotes.

- Nếu gặp dấu " đôi liên tiếp: Xem là dấu nháy kép trốn (escaped quotes) và ghi nhận vào bộ đệm.

- Nếu gặp dấu phẩy , và inQuotes == false: Xác định đây là điểm cắt cột thực sự, tách cột an toàn mà không làm rách nội dung văn bản gốc.

5.2.2. Thuật toán trích lọc bối cảnh thiên tai (Contextual Filtering)

- Trong LocationAnalysis, để tránh việc thu thập sai lệch thông tin địa phương nhiễu (Ví dụ post: *"Hà Nội mùa này đẹp quá"* không liên quan đến thiên tai), hệ thống áp dụng kỹ thuật kiểm tra chéo bối cảnh.

- Hệ thống chỉ tiến hành quét và ghi nhận điểm nóng địa phương nếu trong văn bản đồng thời xuất hiện ít nhất một từ khóa nằm trong danh sách thiệt hại (AFFECTED_PEOPLE, DAMAGED_INFRA, HOUSES_DAMAGED) hoặc danh sách cảm xúc tiêu cực (NEGATIVE_WORDS).

5.2.3. Duyệt đối sánh từ khóa thông minh bằng Regex biên từ (Unicode Word Boundary)

Trong BaseKeywordAnalysis, để tránh nhận diện nhầm các từ khóa chứa nhau (ví dụ: tìm kiếm từ "cô" nhưng lại khớp nhầm vào từ "công", "công an"), hệ thống áp dụng Regex biên từ có hỗ trợ ký tự Unicode đầy đủ:
(?Ui)(?<![\p{L}\p{N}]) + Pattern.quote(keyword) + (?![\p{L}\p{N}])

- (?Ui): Kích hoạt chế độ không phân biệt chữ hoa chữ thường Unicode.

- (?<![\p{L}\p{N}]) và (?![\p{L}\p{N}]): Đảm bảo ngay trước và ngay sau từ khóa không được phép là một ký tự chữ hoặc số Unicode, bảo vệ từ khóa đứng độc lập tuyệt đối.

5.2.4. Tối ưu hóa hạn ngạch gọi mạng (Batch Stats Request)

Trong YouTubeCollector, thay vì thực hiện N cuộc gọi API riêng lẻ lên máy chủ Google để lấy thống kê lượt tương tác cho N video (gây tốn hạn ngạch gọi mạng và làm chậm hệ thống), lớp này thu gom toàn bộ danh sách ID video thu được trên trang hiện tại, nối chúng lại bằng dấu phẩy và thực hiện **duy nhất một cuộc gọi gộp** (Batch request) lên tài nguyên /videos?part=statistics&id=id1,id2... để lấy toàn bộ dữ liệu thống kê một lượt.

5.2.5. Đa luồng bất đồng bộ chống đóng băng giao diện (Asynchronous Multithreading)

- Tại DataService, tác vụ thu thập dữ liệu qua mạng là một tiến trình tốn thời gian (I/O Blocking). Nếu chạy trên luồng giao diện chính (JavaFX Application Thread), màn hình sẽ ngay lập tức bị đơ cứng (Not Responding).

- Hệ thống giải quyết bằng cách đẩy toàn bộ tác vụ cào dữ liệu và chạy Pipeline làm sạch sang một luồng riêng (currentCollectionThread). Khi cần cập nhật tiến độ hoặc thông báo lỗi quay ngược lại màn hình GUI, hệ thống

bọc mã lệnh cập nhật giao diện trong phương thức Platform.runLater() để xếp hàng thực thi an toàn trên luồng đồ họa chính.

VI. Triển khai, hướng dẫn sử dụng & Minh họa chương trình

6.1. Môi trường triển khai và yêu cầu hệ thống

- **Hệ điều hành:** Microsoft Windows, macOS hoặc Linux.
- **Bộ công cụ phát triển Java (JDK):** Yêu cầu JDK phiên bản 17 trở lên. Đường dẫn thư mục JDK cần được cấu hình chính xác vào biến môi trường hệ thống.

Cấu trúc cây thư mục của dự án được tổ chức phân tách rõ ràng giữa mã nguồn chính, thư viện liên kết ngoài và phân vùng kết xuất biên dịch tự động:

```
Thư mục gốc/
├── out/                  <-- Thư mục kết xuất biên dịch tự động của IDE
└── OOP_Prj/              <-- Thư mục chứa mã nguồn dự án chính
    ├── bin/              <-- Thư mục chứa bytecode biên dịch thủ công (.class)
    ├── lib/              <-- Thư mục chứa các file thư viện liên kết (.jar)
    ├── src/              <-- Thư mục chứa toàn bộ mã nguồn (.java)
    └── build.bat         <-- Kịch bản tự động hóa khởi chạy nhanh trên Windows
```

- **out/ (Output Folder):** Thư mục kết xuất biên dịch và đóng gói tự động do các môi trường phát triển tích hợp (như IntelliJ IDEA, Eclipse) tạo dựng tập trung ngoài thư mục dự án gốc.
- **OOP_Prj/ (Project Folder):** Thư mục chứa toàn bộ tài nguyên cốt lõi phục vụ phát triển ứng dụng:
    - **src/ (Source):** Chứa mã nguồn chương trình viết bằng Java chia theo các gói nghiệp vụ (model, data, preprocessor, analysis, ui, app).
    - **lib/ (Libraries):** Chứa các file thư viện liên kết cục bộ dạng .jar (như Jsoup, GSON, Apache POI, OpenCSV…) giúp biên dịch trực tiếp không phụ thuộc vào kết nối mạng.
    - **bin/ (Binaries):** Chứa các tệp tin lớp đã được biên dịch thành công sang dạng Java bytecode (.class) từ các lệnh biên dịch dòng lệnh.

- **build.bat:** Kịch bản lệnh tự động hóa biên dịch và chạy nhanh dành riêng cho môi trường Windows.

Triển khai chương trình thheo giải pháp chung nhất và ổn định nhất trên cả Windows, macOS lẫn Linux:

- Mở một IDE bất kỳ (như **IntelliJ IDEA**, **Eclipse** hoặc **NetBeans**).
- Nhập (Import) thư mục dự án chính **OOP_Prj** vào IDE.
- IDE sẽ tự động nhận diện thư mục mã nguồn src/ và thư mục chứa các file thư viện .jar trong lib/.
- Mở tệp tin src/app/Main.java, nhấp chuột phải và chọn **Run** để khởi chạy chương trình. (IDE sẽ tự động sinh và quản lý thư mục kết xuất out/).

6.2. Hướng dẫn sử dụng chi tiết

Ứng dụng được thiết kế tối giản, tập trung toàn bộ luồng điều phối trên một màn hình điều khiển trung tâm giúp người dùng vận hành dễ dàng qua các bước cơ bản sau:

Bước 1: Tiếp cận Trang chủ và lựa chọn giao diện (Theme Mode)

Khi ứng dụng khởi chạy, màn hình Trang chủ chào mừng xuất hiện ở chế độ nền tối (Dark Mode). Người dùng có ba lựa chọn tương tác tại đây:

- Nhấp chọn nút **Exit (✕ Exit)** ở giữa màn hình để đóng và thoát chương trình một cách an toàn.
- Nhấp chọn nút chuyển đổi **Light Mode (☀ Light Mode)** ở góc trên bên phải để chuyển đổi toàn bộ giao diện sang chế độ nền sáng. Hệ thống sẽ tự động hoán đổi stylesheet CSS tương ứng để đồng bộ màu chữ và phân vùng hiển thị.
- Nhấn nút **Data Analysis (📊 Data Analysis)** để chính thức bước vào bảng điều khiển trung tâm.

Bước 2: Cấu hình thông số và tiến hành thu thập dữ liệu

Tại phân vùng **Configuration** trên Dashboard, người dùng thực hiện thiết lập các tham số đầu vào:

1. **Lựa chọn nguồn dữ liệu (Source):** Chọn một trong các nguồn báo chí trực tuyến (Thanh Niên, VietNamNet, VnExpress), mạng xã hội (YouTube, TikTok) hoặc tệp ngoại tuyến (File CSV (Offline)).
    - *Nếu chọn chế độ ngoại tuyến (File CSV (Offline)):* Giao diện sẽ xuất hiện nút **Choose File**. Người dùng nhấp chọn nút này để trỏ tới tệp .csv dữ liệu thô đã đóng gói sẵn trên máy tính. Lúc này, tiến trình nạp sẽ diễn ra nhanh chóng vì không phải gửi yêu cầu qua mạng.
2. **Nhập từ khóa (Keywords):** Nhập từ khóa chủ đề (ví dụ: bão Yagi) để hệ thống tiến hành tìm kiếm và lọc nội dung (Trường này sẽ tự động khóa lại nếu chọn chế độ nạp file offline).
3. **Lựa chọn mốc thời gian (From/To):** Thiết lập khoảng thời gian giới hạn bắt đầu và kết thúc của thảm họa cứu trợ cần quét dữ liệu.
4. **Giới hạn số lượng (Max Count):** Nhập số lượng bài viết tối đa mong muốn thu thập.
5. **Kích hoạt tiến trình:** Nhấp chọn nút **Start collecting (▶ Start collecting)**. Hệ thống sẽ khởi chạy luồng cào tin bất đồng bộ ngầm. Toàn bộ vết hoạt động của bộ cào sẽ được in trực tiếp xuống khung **Logs (Tiến trình)** bên dưới theo thời gian thực. Người dùng có thể nhấn **Stop (■ Stop)** bất kỳ lúc nào để chủ động ngắt tiến trình.

Bước 3: Xuất báo cáo thống kê (CSV & Excel)

- Khi tiến trình thu thập kết thúc, nhãn trạng thái ở thanh điều phối sẽ thông báo tổng số lượng bài viết đã thu thập (ví dụ: Đã thu thập 150 kết quả). Hệ thống tự động đóng gói dữ liệu sạch này dưới dạng tệp CSV có đánh số thứ tự tăng dần ở cuối tên file (ví dụ: _1, _2...) để tránh ghi đè trong thư mục dự án.
- Đồng thời, nút **Export to Excel** màu cam sẽ được mở khóa. Người dùng nhấp chọn nút này, chỉ định vị trí lưu trên máy tính để hệ thống tự động xuất tệp báo cáo chuyên sâu định dạng .xlsx phục vụ cho việc đọc và lưu trữ số liệu thống kê.

Bước 4: Xem biểu đồ phân tích trực quan và Tóm tắt AI

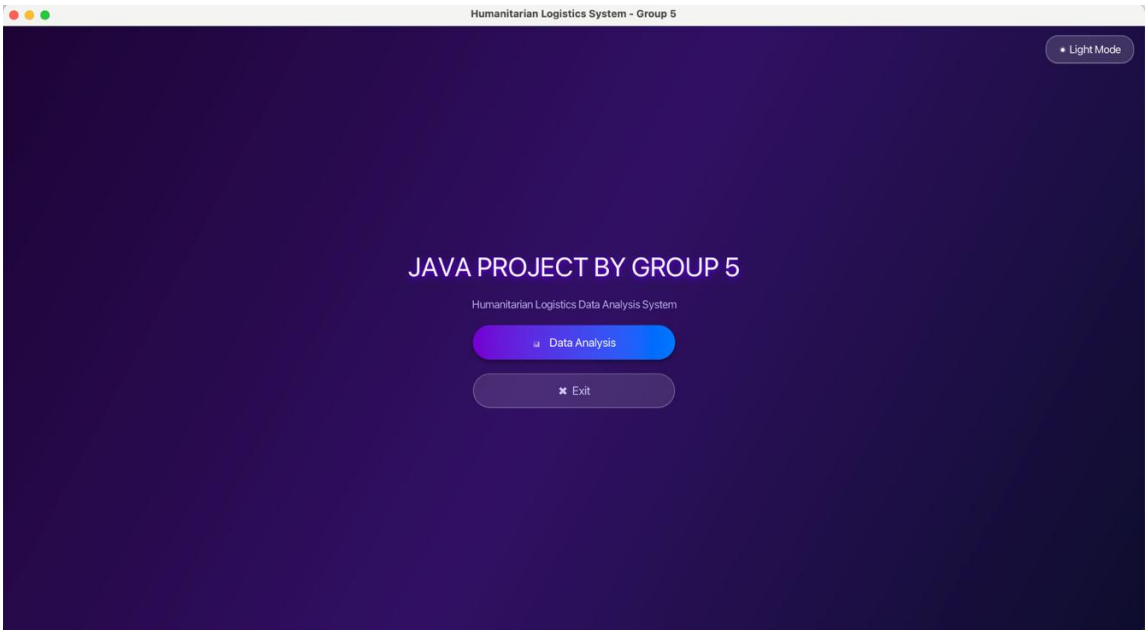
- Khi dữ liệu nạp thành công, 4 nút lối tắt biểu đồ ở chân trang Dashboard (bao gồm: *Sentiment Over Time, Damage Category, Satisfaction, Hotspot Locations*) sẽ chuyển sang trạng thái kích hoạt (mở khóa).
- Người dùng nhấp chọn một nút bất kỳ để chuyển đổi màn hình sang View biểu đồ tương ứng.
- Tại màn hình biểu đồ, người dùng lựa chọn máy chủ AI tại ComboBox thả xuống (gồm **Google Gemini** và **Groq**) rồi nhấn nút **AI Tóm tắt (✨ AI Tóm tắt)**. Hệ thống sẽ tự động gửi dữ liệu định dạng văn bản của biểu đồ lên mô hình AI và hiển thị bản phân tích xu hướng định lượng chi tiết ở khung văn bản chân trang.
- Nhấn nút **Back (⬅ Back)** ở góc trên bên trái để quay trở lại Dashboard chính. Nhấn nút **Home** trên Dashboard để quay về Trang chủ và nhấn **Exit** để đóng chương trình.

6.3. Minh họa và thuyết minh chương trình

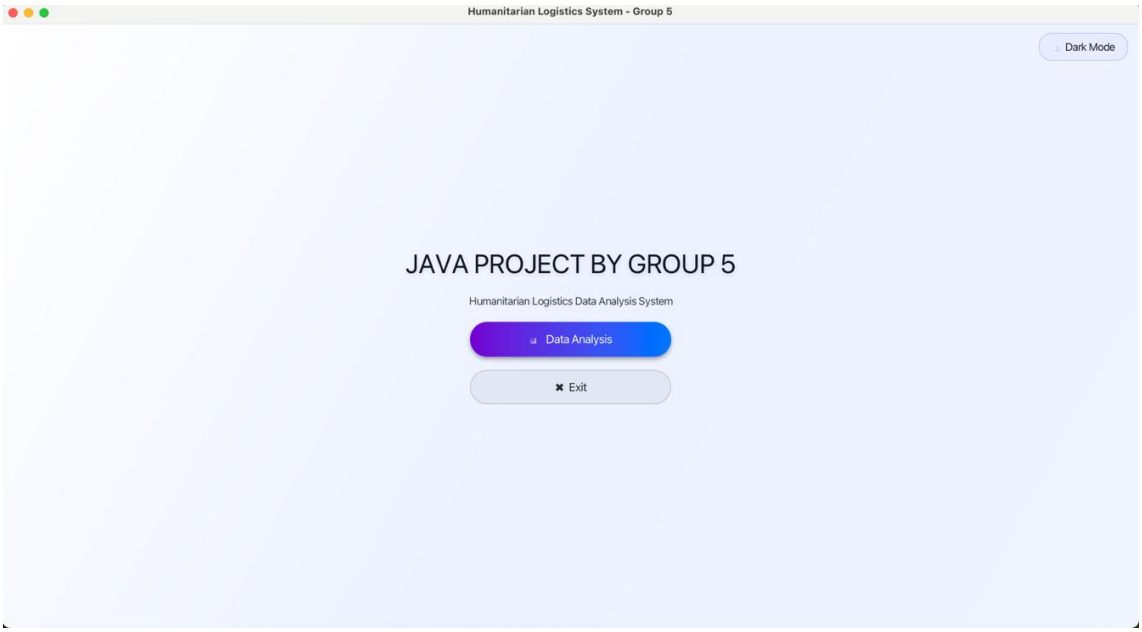
Dưới đây là các hình ảnh ghi nhận giao diện vận hành thực tế của chương trình cùng thuyết minh kỹ thuật chi tiết:

6.3.1. Giao diện màn hình Trang chủ (HomeView)

- **Hình 6.1 (Chế độ tối - Dark Mode):** Giao diện khởi đầu của ứng dụng

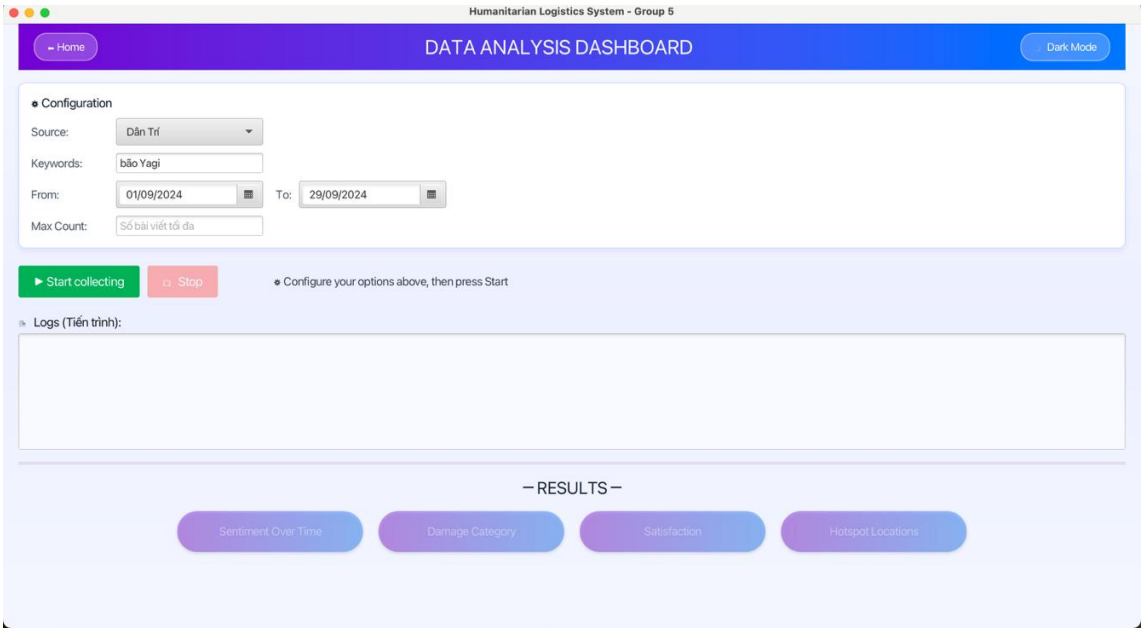


- **Hình 6.2 (Chế độ sáng - Light Mode):** Thể hiện sự linh hoạt của hệ thống ThemeManager

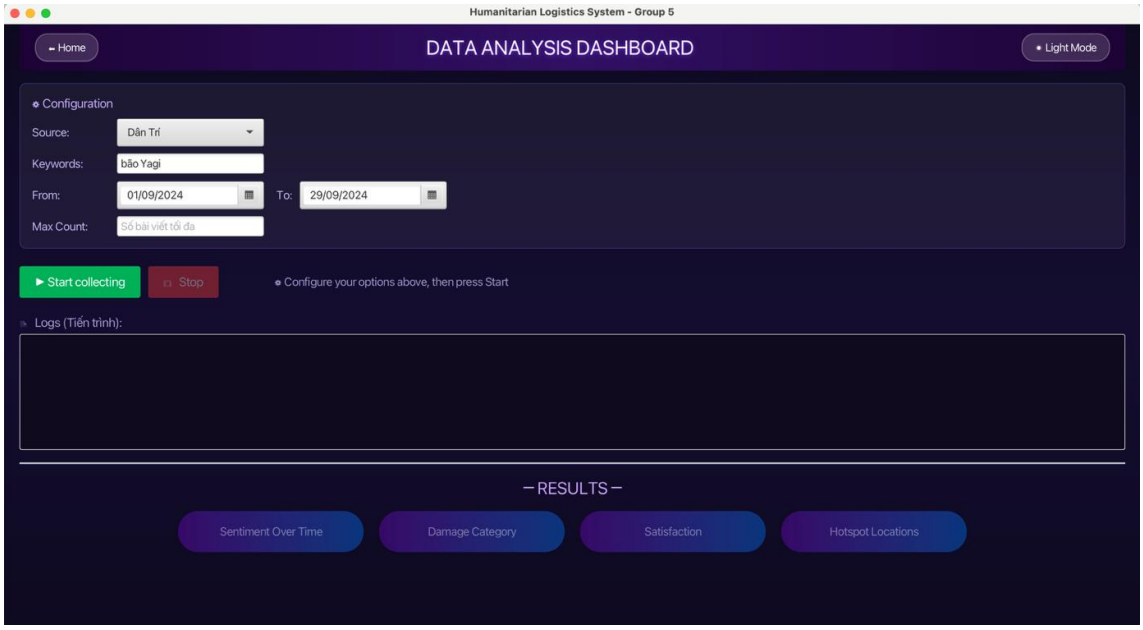


6.3.2. Giao diện bảng điều khiển trung tâm (DashboardView)

- **Hình 6.3 (Dashboard ở chế độ nền sáng - Light Mode):** Minh họa giao diện nhập liệu trực quan của hệ thống

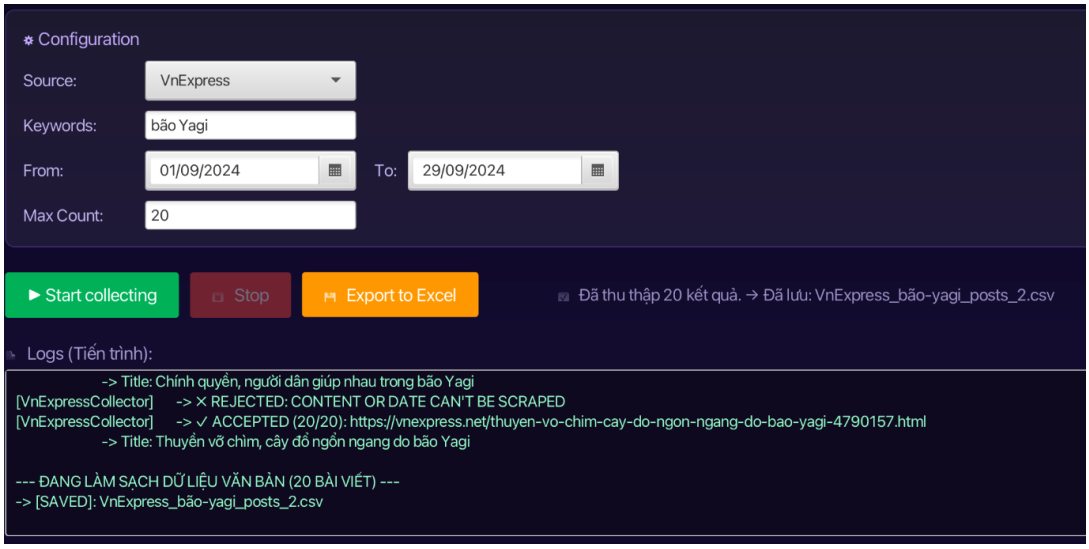


- **Hình 6.4 (Dashboard ở chế độ nền tối - Dark Mode):** Thể hiện sự đồng bộ giao diện tối toàn diện trên màn hình Dashboard. Toàn bộ các nhãn nhập liệu (Source, Keywords, From, To) tự động chuyển sang màu chữ trắng sáng để người dùng dễ đọc trên nền tối sâu của ứng dụng.

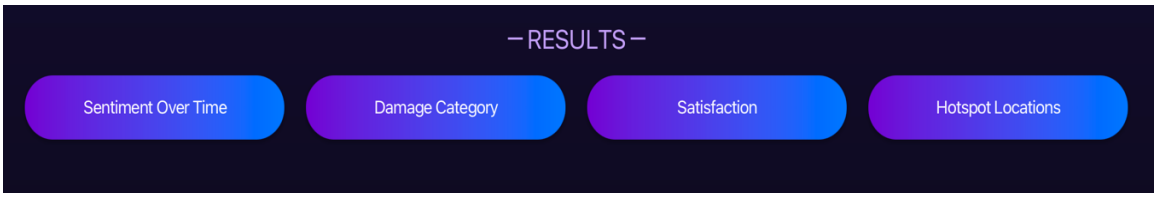


6.3.3. Kết quả thu thập và lưu trữ báo cáo ngoại tuyến

- **Hình 6.5 (Trạng thái nạp dữ liệu hoàn tất):** Minh họa trạng thái hệ thống sau khi cào tin hoặc nạp file thành công. Nhãn trạng thái in rõ dòng chữ Đã thu thập xong kết quả kèm theo sự xuất hiện của nút bấm màu cam **Export to Excel** nổi bật ở trung tâm thanh trạng thái.



- **Hình 6.6 (Hệ thống các nút lối tắt biểu đồ được mở khóa):** Minh họa 4 nút bấm biểu đồ phân tích ở chân trang Results đã được chuyển sang trạng thái kích hoạt hoàn toàn với hiệu ứng dải màu chuyển sắc xanh - tím bắt mắt, sẵn sàng dẫn dắt người dùng sang màn hình trực quan hóa đồ thị.

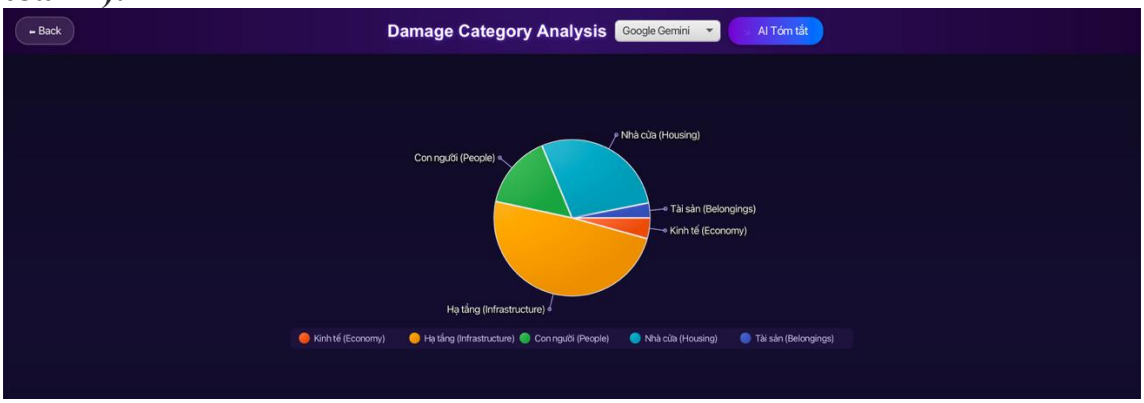


6.3.4. Trực quan hóa đồ thị và Phân tích tóm tắt bằng Generative AI

- **Hình 6.7 - Biểu đồ đường phân tích diễn biến tâm lý theo thời gian (LineChart - Bài toán 1):**



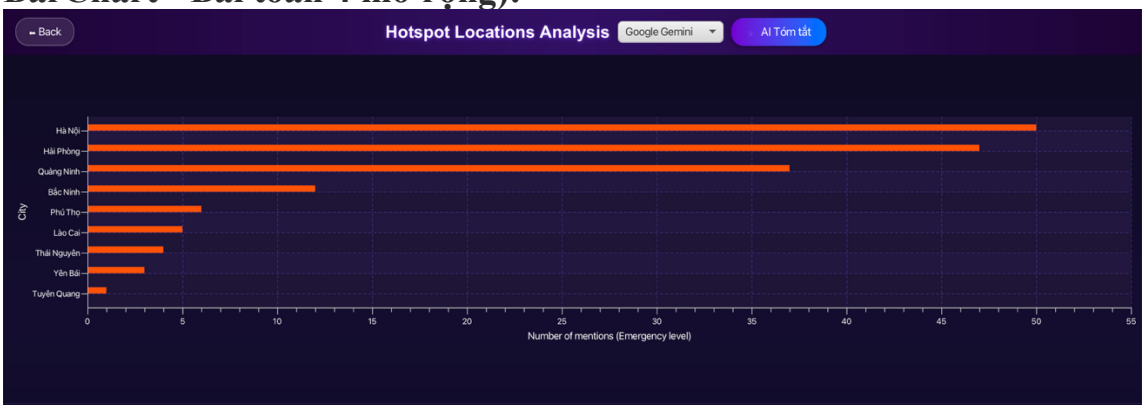
- **Hình 6.8 - Biểu đồ tròn phân loại danh mục thiệt hại (PieChart - Bài toán 2):**



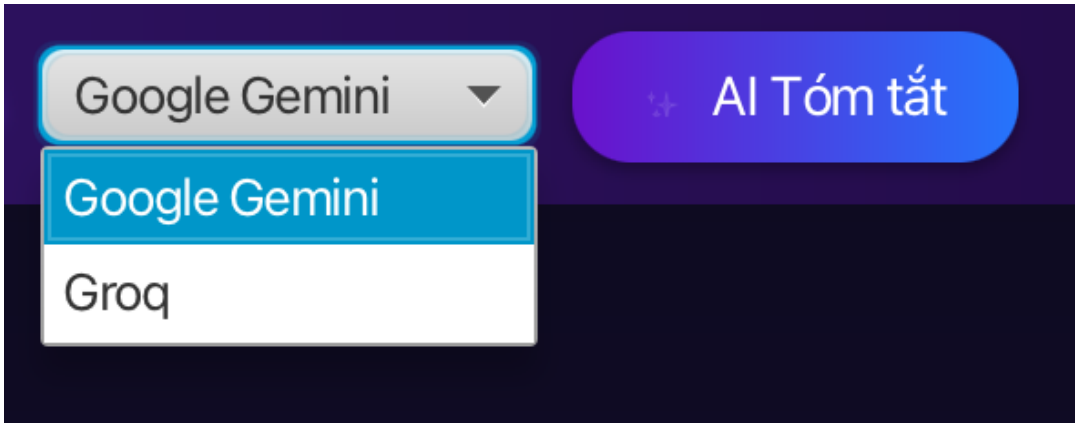
- **Hình 6.9 - Biểu đồ cột đối sánh mức độ hài lòng cứu trợ (BarChart - Bài toán 3):**



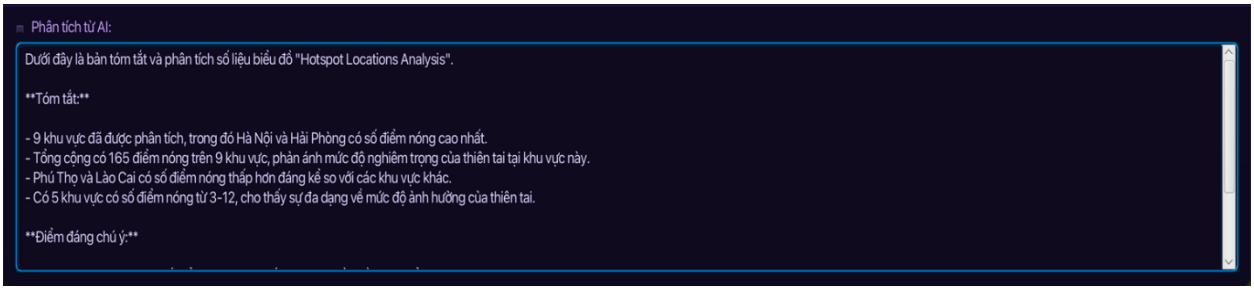
- **Hình 6.10 - Biểu đồ cột ngang xếp hạng điểm nóng thiên tai (Horizontal BarChart - Bài toán 4 mở rộng):**



- **Hình 6.11 - Cơ chế lựa chọn mô hình AI dự phòng:**
  Minh họa menu ComboBox thả xuống tích hợp tại thanh Header của màn hình biểu đồ, cho phép người dùng chủ động hoán đổi nhanh giữa hai nhà cung cấp dịch vụ độc lập là **Google Gemini** và **Groq**, thể hiện tư duy thiết kế phần mềm có tính sẵn sàng cao và tính chịu lỗi của nhóm.



- **Hình 6.12 - Kết quả tóm tắt và đánh giá số liệu tự động từ AI:**



VII. Kết luận, mở rộng & Hướng phát triển tương lai

7.1. Kết luận

Dự án đã thiết kế và xây dựng thành công một giải pháp ứng dụng công nghệ thông tin và khoa học dữ liệu mạng xã hội vào việc giải quyết bài toán thực tế cấp bách trong Logistics nhân đạo. Hệ thống đã hiện thực hóa việc đa hình hóa các nguồn thu thập thông tin trực tuyến và ngoại tuyến, thiết lập quy trình làm sạch chữ thô khép kín bằng các bộ tiền xử lý và bóc tách dữ liệu thống kê trực quan qua 4 bài toán thực tiễn về tâm lý, phân loại thiệt hại, mức độ hài lòng của công chúng và định vị điểm nóng thiên tai. Việc tích hợp thành công sức mạnh trí tuệ nhân tạo (Gemini/Groq) giúp hệ thống sở hữu năng lực tự động đọc biểu đồ vượt trội, hỗ trợ đắc lực cho các nhà điều hành cứu trợ nhân đạo đưa ra quyết định nhanh chóng và chính xác.

7.2. Hạn chế của hệ thống

Mặc dù đạt được những kết quả khả quan về mặt kỹ thuật và nghiệp vụ, hệ thống vẫn tồn tại một số điểm hạn chế cần nhìn nhận khách quan:

- **Phụ thuộc vào hạn ngạch API mạng xã hội:** Việc cào dữ liệu từ YouTube hay TikTok trực tiếp phụ thuộc lớn vào tính ổn định của các máy chủ bên thứ ba và giới hạn hạn ngạch gọi API miễn phí.

- **Đọc hiểu ngữ nghĩa ở mức độ cơ bản:** Hệ thống hiện tại phân tích tâm cảm dựa trên quét từ điển từ khóa thông minh kết hợp đảo ngữ Regex. Phương pháp này hoạt động siêu nhanh nhưng chưa thể bao quát được hết các sắc thái ngôn ngữ phức tạp như mỉa mai, châm biếm, ẩn dụ thường gặp trên mạng xã hội.

7.3. Hướng phát triển tương lai

Để phát triển hệ thống lên tầm cao mới, các hướng nghiên cứu mở rộng trong tương lai bao gồm:

- **Tích hợp mô hình học máy sâu chuyên dụng (Deep Learning NLP):** Thay thế bộ quét từ khóa bằng các mô hình xử lý ngôn ngữ tự nhiên tiếng Việt chuyên sâu học máy như **PhởBERT** để phân tích sắc thái cảm xúc đạt độ chính xác trên 95%.

- **Tích hợp công nghệ luồng dữ liệu thời gian thực (Real-time Data Streaming):** Sử dụng các giải pháp như Apache Kafka để bắt trực tiếp luồng dữ liệu cứu trợ trực tuyến ngay khi người dùng đăng bài, giúp đưa ra cảnh báo cứu nạn tức thời trên bản đồ nhiệt địa lý (Heatmap).

Tài liệu tham khảo

[1] The Apache Software Foundation, "Apache POI - The Java API for Microsoft Documents". Địa chỉ: https://poi.apache.org/.

[2] J. Hedley, "Jsoup Developer Guide: Java HTML Parser," Jsoup. Địa chỉ: https://jsoup.org/.

[3] Google, "Gemini API Documentation: Generative AI Reference Guides, Google AI for Developers". Địa chỉ: https://ai.google.dev/.

[4] Groq, "Groq API & Llama-3 Reference Manual, GroqCloud". Địa chỉ: https://groq.com/.

[5] OpenJFX, "JavaFX Documentation: Client Application Platform for Desktop, Mobile and Embedded Systems". Địa chỉ: https://openjfx.io/.

[7] Oracle, "Package java.lang" Java Platform, Standard Edition & Java Development Kit Version 21 API Specification. Địa chỉ:

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/package-summary.html.

[8] Google, "Google Java Style Guide, GitHub Pages". Địa chỉ:

https://google.github.io/styleguide/javaguide.html.

LỜI CẢM ƠN

Lời cuối, nhóm chúng em xin được gửi lời cảm ơn chân thành và sâu sắc nhất đến **Thầy Trịnh Tuấn Đạt**.

Trong suốt quá trình học tập môn Lập trình Hướng đối tượng (OOP), những kiến thức nền tảng, tư duy thiết kế hệ thống và sự tận tâm giảng dạy của Thầy chính là chiếc la bàn định hướng giúp chúng em hoàn thành đồ án này. Những nguyên lý kiến trúc, những mẫu thiết kế mà Thầy truyền đạt không chỉ là lý thuyết trên bục giảng, mà đã thực sự trở thành công cụ sắc bén để chúng em giải quyết các bài toán lập trình trong thực tế.

Đặc biệt, chúng em vô cùng trân trọng và biết ơn Thầy đã dành thời gian quý báu của mình để đọc, xem xét và đánh giá tâm huyết của chúng em qua cuốn báo cáo chi tiết này. Dù đã nỗ lực hết sức để áp dụng những kiến thức được học vào dự án, nhưng do còn nhiều hạn chế về mặt kinh nghiệm thực tiễn, cả mã nguồn lẫn báo cáo chắc chắn sẽ không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những lời nhận xét, góp ý và chỉ bảo quý báu từ Thầy để nhóm có thể rút kinh nghiệm và ngày càng hoàn thiện hơn trên con đường học tập, phát triển phần mềm phía trước.

Sau cùng, chúng em kính chúc Thầy luôn dồi dào sức khỏe, luôn giữ mãi ngọn lửa nhiệt huyết với nghề và gặt hái được thêm nhiều thành công trong sự nghiệp giảng dạy cao quý.

Trân trọng cảm ơn Thầy!

--- KẾT THÚC ---